

Джамии наблизو الأذان

Нов, пети раздел „Джамии“: карта и списък с джамиите около теб, подредени по разстояние, и следващата молитва. Докосваш джамия – виждаш адреса и колко време е пеша, и с един бутон тръгваш с любимото си приложение за навигация.

OpenStreetMap

MapLibre

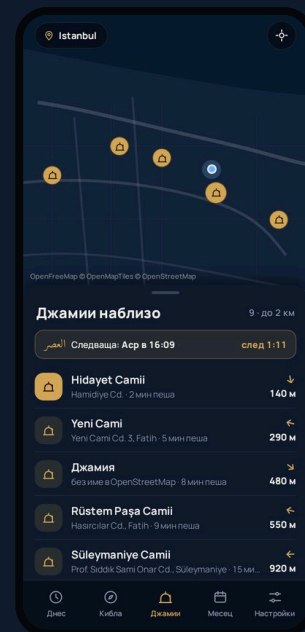
OpenFreeMap

Waze · Google Maps · Petal Maps

Без Google услуги

Jest – 258 теста

23 файла



Какво е решено в този етап

„Джамии“ е пети раздел: Днес · Кибла · Джамии · Месец · Настройки · данните – от OpenStreetMap (Overpass), картата – MapLibre + OpenFreeMap: без ключ и без Google, затова работи и на Huawei · радиус 2 км, бутон „Търси до 10 км“ · пеша до 2,5 км, по-далеч – с кола · Настройки → Навигация: Автоматично, Google Maps, Waze, HERE WeGo, Petal Maps (Huawei), Organic Maps (на iPhone „Автоматично“ е Apple Maps) · първото „Упътване“ пита „Отвори с...“ и „Запомни избора“ – изборът остава и след рестарт · Waze води само с кола · без интернет – последното търсене с часа му · до сървъра отива само район от ~1 км.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нови пакети и нов build

03 Откъде са джамиите

04 Картата

05 Упътване и изборът на навигация

06 Екраните и текстовете

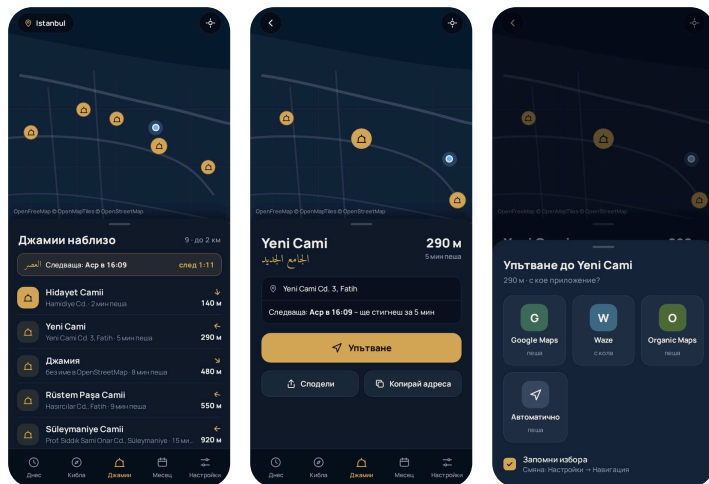
07 Тестове

08 Проверка на телефона

Стъпка 1

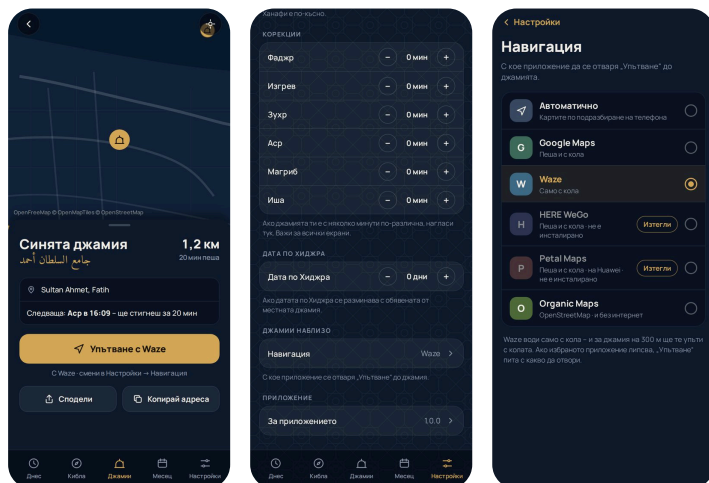
Какво прави етапът

Петият раздел в долната лента. Позицията се взима само докато екранът е отворен – както досега, без слеждане във фонов режим.



Списъкът · джамията · „Отвори с...” при първото упътване

- **Списъкът:** джамии до 2 км, най-близката – със златна икона; под името – адресът и колко минути е пеша; вдясно – разстоянието и стрелка накъде е (север е горе, както на картата).
- **Следващата молитва** отгоре – „Аср в 16:09 · след 1:11“. Изгревът се прескача – не е молитва в джамията.
- **Джамията:** името и арабското име (ако го има в OpenStreetMap), адрес, „ще стигнеш за 5 мин“ (само ако стигаш преди молитвата), „Упътване“, „Сподели“, „Копирай адреса“. Без адрес – копират се координатите.
- Android „Назад“ – от джамията обратно към списъка.



Запомнен Waze · Настройки → Джамии наблизо · изборът на навигация

Стъпка 2

Нови пакети и нов build

Картата (MapLibre) и копирането (expo-clipboard) имат native код, а в Kotlin модула има три нови функции – нужни са `npm install`, `prebuild` и нов `build`. Картата не работи в Expo Go.

Пакетите, `prebuild` и нов `build`

```
npm install
npx expo prebuild --clean
npx expo run:android
```

`--clean` е без `--platform` – така и iOS папката получава MapLibre и новите схеми за проверка на приложенията (за EAS build на iPhone по-късно).

package.json корен на проекта

- `@maplibre/maplibre-react-native` 11.5 и `expo-clipboard`.

package.json

84 реда

```
{
  "name": "ezan",
  "version": "1.0.0",
  "main": "expo-router/entry",
  "dependencies": {
    "@expo-google-fonts/amiri": "^0.4.1",
    "@expo-google-fonts/manrope": "^0.4.2",
    "@expo/metro-runtime": "~57.0.16",
    "@expo/ui": "~57.0.20",
    "@maplibre/maplibre-react-native": "^11.5.0",
    "@react-native-async-storage/async-storage": "2.2.0",
    "adhan": "^4.4.6",
    "expo": "~57.0.25",
    "expo-audio": "~57.0.5",
    "expo-background-task": "~57.0.20",
    "expo-build-properties": "~57.0.22",
    "expo-clipboard": "~57.0.2",
    "expo-constants": "~57.0.19",
    "expo-dev-client": "~57.0.19",
    "expo-document-picker": "~57.0.2",
    "expo-file-system": "~57.0.7",
    "expo-font": "~57.0.4",
    "expo-haptics": "~57.0.3",
    "expo-linear-gradient": "~57.0.2",
    "expo-linking": "~57.0.11",
    "expo-localization": "~57.0.2",
    "expo-location": "~57.0.20",
    "expo-notifications": "~57.0.21",
    "expo-router": "~57.0.23",
    "expo-sensors": "~57.0.3",
    "expo-splash-screen": "~57.0.9",
    "expo-status-bar": "~57.0.1",
    "expo-system-ui": "~57.0.4",
    "expo-task-manager": "~57.0.20",
    "expo-widgets": "~57.0.21",
    "geomagnetism": "^0.2.0",
    "react": "19.2.3",
    "react-dom": "19.2.3",
    "react-native": "0.86.3",
```

```

    "react-native-reanimated": "4.5.1",
    "react-native-safe-area-context": "~5.7.0",
    "react-native-screens": "~4.26.0",
    "react-native-svg": "15.15.4",
    "react-native-web": "~0.21.0",
    "react-native-worklets": "0.10.1",
    "zustand": "^5.0.15"
  },
  "devDependencies": {
    "@react-native/jest-preset": "^0.86.3",
    "@types/jest": "^29.5.14",
    "@types/react": "~19.2.2",
    "eslint": "^9.0.0",
    "eslint-config-expo": "~57.0.2",
    "jest": "^29.7.0",
    "jest-expo": "^57.0.5",
    "typescript": "~6.0.3"
  },
  "scripts": {
    "start": "expo start",
    "android": "expo run:android",
    "ios": "expo run:ios",
    "web": "expo start --web",
    "typecheck": "tsc --noEmit",
    "lint": "expo lint",
    "test": "jest"
  },
  "private": true,
  "jest": {
    "preset": "jest-expo",
    "testMatch": [
      "**/__tests__/**/*.test.ts"
    ],
    "globalSetup": "./jest.global-setup.js"
  },
  "expo": {
    "autolinking": {
      "android": {
        "exclude": [
          "expo-widgets"
        ]
      }
    }
  }
}

```

app.json корен на проекта

- Plugin `@maplibre/maplibre-react-native`.
- iPhone: `LSApplicationQueriesSchemes` – comgooglemaps, waze, om (без тях iPhone не казва дали Google Maps, Waze и Organic Maps са инсталирани).
- Текстът за разрешението за местоположение – „... и джамиите наблизо“ (и в `locales/*.json`).

app.json

175 реда

```

{
  "expo": {
    "name": "Adhan",
    "slug": "adhan",

```

```

"version": "1.0.0",
"orientation": "portrait",
"icon": "./assets/icon.png",
"userInterfaceStyle": "dark",
"ios": {
  "supportsTablet": false,
  "bundleIdentifier": "com.ilkoadamov.adhan",
  "infoPlist": {
    "CFBundleAllowMixedLocalizations": true,
    "LSApplicationQueriesSchemes": ["comgooglemaps", "waze", "om"]
  }
},
"android": {
  "adaptiveIcon": {
    "backgroundColor": "#0F1B2D",
    "foregroundImage": "./assets/android-icon-foreground.png",
    "backgroundImage": "./assets/android-icon-background.png",
    "monochromeImage": "./assets/android-icon-monochrome.png"
  },
  "predictiveBackGestureEnabled": false,
  "package": "com.ilkoadamov.adhan",
  "permissions": [
    "android.permission.SCHEDULE_EXACT_ALARM"
  ],
  "blockedPermissions": [
    "android.permission.READ_EXTERNAL_STORAGE",
    "android.permission.WRITE_EXTERNAL_STORAGE",
    "android.permission.RECORD_AUDIO"
  ]
},
"web": {
  "favicon": "./assets/favicon.png"
},
"plugins": [
  "expo-router",
  [
    "expo-splash-screen",
    {
      "backgroundColor": "#0F1B2D",
      "image": "./assets/splash-icon.png",
      "imageWidth": 160
    }
  ],
  "expo-font",
  [
    "expo-build-properties",
    {
      "android": {
        "cmakeVersion": "3.31.6"
      }
    }
  ],
  [
    "expo-localization",
    {
      "supportedLocales": {
        "ios": [
          "en",
          "bg",
          "tr"
        ],
      }
    }
  ],

```

```

        "android": [
            "en",
            "bg",
            "tr"
        ]
    }
},
[
    "expo-location",
    {
        "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place
where you are and the mosques nearby.",
        "locationAlwaysAndWhenInUsePermission": false,
        "locationAlwaysPermission": false,
        "motionUsagePermission": false
    }
],
"expo-background-task",
[
    "expo-notifications",
    {
        "icon": "./assets/notification-icon.png",
        "color": "#D4A857",
        "sounds": [
            "./assets/sounds/ezan_chime.wav",
            "./assets/sounds/chime_soft.wav",
            "./assets/sounds/takbir_makkah.wav",
            "./assets/sounds/takbir_madinah.wav",
            "./assets/sounds/takbir_aqsa.wav",
            "./assets/sounds/takbir_istanbul.wav",
            "./assets/sounds/notify_airport.wav",
            "./assets/sounds/notify_popup.wav",
            "./assets/sounds/notify_right.wav",
            "./assets/sounds/notify_modern.wav",
            "./assets/sounds/notify_magical.wav",
            "./assets/sounds/notify_chime_bright.wav",
            "./assets/sounds/notify_doorbell.wav",
            "./assets/sounds/notify_celestial.wav",
            "./assets/sounds/notify_bell_ding.wav",
            "./assets/sounds/notify_crisp_ding.wav",
            "./assets/sounds/notify_shimmer.wav",
            "./assets/sounds/notify_smoke_alarm.wav",
            "./assets/sounds/notify_swoop.wav",
            "./assets/sounds/notify_retro.wav",
            "./assets/sounds/notify_robot.wav",
            "./assets/sounds/notify_three_beep.wav",
            "./assets/sounds/notify_machine_beep.wav",
            "./assets/sounds/notify_water_drop.wav",
            "./assets/sounds/notify_hang_up.wav",
            "./assets/sounds/notify_standard.wav"
        ]
    }
],
[
    "expo-audio",
    {
        "microphonePermission": false,
        "recordAudioAndroid": false,
        "enableBackgroundPlayback": false,
        "enableBackgroundRecording": false
    }
]

```

```

    }
  ],
  "./plugins/withFullSounds",
  [
    "expo-widgets",
    {
      "widgets": [
        {
          "name": "PrayerWidget",
          "displayName": "Adhan",
          "description": "The next prayer and the time left",
          "ios": {
            "supportedFamilies": [
              "systemSmall",
              "systemMedium",
              "accessoryCircular",
              "accessoryRectangular",
              "accessoryInline"
            ]
          },
          "android": null
        },
        {
          "name": "PrayerArabicWidget",
          "displayName": "Adhan · Arabic",
          "description": "The next prayer in Arabic and the time left",
          "ios": {
            "supportedFamilies": [
              "accessoryCircular"
            ]
          },
          "android": null
        }
      ]
    }
  ],
  "@maplibre/maplibre-react-native"
],
"scheme": "adhan",
"backgroundColor": "#0F1B2D",
"experiments": {
  "typedRoutes": true
},
"locales": {
  "bg": "./locales/bg.json",
  "en": "./locales/en.json",
  "tr": "./locales/tr.json"
}
}
}

```

Стъпка 3

Откъде са джамиите

OpenStreetMap – свободната карта на света. Безплатно, без ключ, без регистрация.

mosques.ts `src/domain`

- Заявката към Overpass: `amenity=place_of_worship + religion=muslim` около точката.

- **Поверителност:** до сървъра отиват координати, закръглени до ~1 км, и радиус +1,5 км. Точното място остава в телефона – резултатите се филтрират тук.
- **Името:** на езика на приложението (`name: bg` , `name: tr` ...), иначе местното; арабското – отделно. Адресът – от `addr:*` .
- **Разстояние, посока, време:** пеша 4,8 км/ч, с кола ~30 км/ч в града, ×1,3 за завоите. Точното време дава навигацията.

src/domain/mosques.ts

158 реда

```
import { distanceKm } from './geo';

/**
 * Джамииите наблизно (етап 12) – данните от OpenStreetMap (Overpass API):
 * `amenity=place_of_worship` + `religion=muslim` – джамии и месджиди (и по-малките
 * молитвени помещения, ако са отбелязани). Тук е само чистата логика: заявката, разчитането
 * на отговора, разстоянието, посоката и времето пеша. Мрежата и кешът – src/services/mosques.ts.
 */

/** Първо търсим до 2 км; „Търси до 10 км“ – по-далечните. */
export const NEAR_RADIUS_M = 2000;
export const FAR_RADIUS_M = 10000;
/** До това разстояние упътването е пеша, над него – с кола. */
export const WALK_LIMIT_M = 2500;

export interface Mosque {
  /** „node/123“, „way/456“ – уникално в OSM. */
  id: string;
  /** Името на езика на приложението, иначе местното; null – няма име в OSM. */
  name: string | null;
  /** Името на арабски (name:ar), ако го има и се различава. */
  arabic: string | null;
  lat: number;
  lon: number;
  /** „Yeni Cami Cd. 3, Fatih“ – от addr:*; null – няма адрес. */
  address: string | null;
}

export interface NearMosque extends Mosque {
  /** Разстояние по права линия, м. */
  distanceM: number;
  /** Посока от теб към джамията, градуси от север. */
  bearing: number;
  /** Пеша (до WALK_LIMIT_M) или с кола. */
  mode: 'walk' | 'drive';
  /** Приблизително време, мин. */
  minutes: number;
}

/**
 * Какво отива към Overpass: районът е закръглен до ~1 км (2 знака след запятаята)
 * и радиусът е увеличен с 1,5 км – точното място остава в телефона, а резултатите
 * се филтрират тук по истинското разстояние.
 */
export function privacyArea(lat: number, lon: number, radiusM: number): { lat: number; lon: number; radiusM: number } {
  return { lat: Math.round(lat * 100) / 100, lon: Math.round(lon * 100) / 100, radiusM: radiusM + 1500 };
}

/** Заявката към Overpass (Overpass QL). `out center` дава точка и за сградите (way). */
```

```

export function overpassQuery(lat: number, lon: number, radiusM: number): string {
  const a = privacyArea(lat, lon, radiusM);
  return (
    `[out:json][timeout:25];` +
    `nwr["amenity"="place_of_worship"]["religion"="muslim"](around:${a.radiusM},${a.lat},${a.lon});` +
    `out center tags;`
  );
}

interface OsmElement {
  type: string;
  id: number;
  lat?: number;
  lon?: number;
  center?: { lat: number; lon: number };
  tags?: Record<string, string>;
}

/** Адресът от addr:* - улица и номер, квартал/град. */
function addressOf(tags: Record<string, string>): string | null {
  const street = [tags['addr:street'], tags['addr:housenumber']].filter(Boolean).join(' ');
  const area = tags['addr:suburb'] ?? tags['addr:district'] ?? tags['addr:city'] ?? tags['addr:place'];
  const line = [street || tags['addr:place'], street ? area : null].filter(Boolean).join(', ');
  return line || null;
}

/** Отговорът на Overpass → джамии. `lang` - езикът на приложението (за name:bg / name:en / name:tr). */
export function parseOverpass(json: unknown, lang: string): Mosque[] {
  const elements = (json as { elements?: OsmElement[] })?.elements;
  if (!Array.isArray(elements)) return [];
  const out: Mosque[] = [];
  const seen = new Set<string>();
  for (const e of elements) {
    const lat = e.lat ?? e.center?.lat;
    const lon = e.lon ?? e.center?.lon;
    if (typeof lat !== 'number' || typeof lon !== 'number') continue;
    const id = `${e.type}/${e.id}`;
    if (seen.has(id)) continue;
    seen.add(id);
    const tags = e.tags ?? {};
    const name = tags[`name:${lang}`] ?? tags.name ?? tags['name:en'] ?? null;
    const ar = tags['name:ar'] ?? null;
    out.push({
      id,
      name: name?.trim() || null,
      arabic: ar && ar !== name ? ar.trim() : null,
      lat,
      lon,
      address: addressOf(tags),
    });
  }
  return out;
}

/** Посоката от точка А към В, градуси от север (0-360). */
export function bearingDeg(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const rad = Math.PI / 180;
  const y = Math.sin((lon2 - lon1) * rad) * Math.cos(lat2 * rad);
  const x =
    Math.cos(lat1 * rad) * Math.sin(lat2 * rad) - Math.sin(lat1 * rad) * Math.cos(lat2 * rad) *
    Math.cos((lon2 - lon1) * rad);

```

```

    return (Math.atan2(y, x) / rad + 360) % 360;
}

/**
 * Приблизително време: улиците не са права линия - *1,3; пеша 4,8 км/ч (80 м/мин),
 * с кола в града ~30 км/ч (500 м/мин). Точното време дава приложението за навигация.
 */
export function travelMinutes(distanceM: number, mode: 'walk' | 'drive'): number {
    const road = distanceM * 1.3;
    return Math.max(1, Math.round(road / (mode === 'walk' ? 80 : 500)));
}

/** Джамиите около точката до `radiusM`, подредени по разстояние. */
export function nearMosques(all: readonly Mosque[], lat: number, lon: number, radiusM: number):
NearMosque[] {
    return all
        .map((m) => {
            const distanceM = Math.round(distanceKm(lat, lon, m.lat, m.lon) * 1000);
            const mode: 'walk' | 'drive' = distanceM <= WALK_LIMIT_M ? 'walk' : 'drive';
            return { ...m, distanceM, bearing: bearingDeg(lat, lon, m.lat, m.lon), mode, minutes:
travelMinutes(distanceM, mode) };
        })
        .filter((m) => m.distanceM <= radiusM)
        .sort((a, b) => a.distanceM - b.distanceM);
}

/** „350 м“, „1,4 км“, „12 км“ (на английски - с точка и „м/км“, на турски - със запетая и „м/км“). */
export function formatDistance(m: number, lang: string): string {
    const unitM = lang === 'bg' ? 'м' : 'm';
    const unitKm = lang === 'bg' ? 'км' : 'km';
    if (m < 1000) return `${Math.round(m / 10) * 10} ${unitM}`;
    const km = m / 1000;
    const text = km < 10 ? km.toFixed(1) : String(Math.round(km));
    return `${lang === 'en' ? text : text.replace('.', ',')} ${unitKm}`;
}

/** Може ли кешираният резултат да се ползва за това място и радиус. */
export function cacheCovers(
    cache: { lat: number; lon: number; radiusM: number; at: number } | null | undefined,
    lat: number,
    lon: number,
    radiusM: number,
    now: number,
    maxAgeMs = 24 * 3600_000,
): boolean {
    if (!cache) return false;
    if (now - cache.at > maxAgeMs) return false;
    // кешът е за кръг около закръглената точка с радиус cache.radiusM (вече с +1,5 км)
    const d = distanceKm(cache.lat, cache.lon, lat, lon) * 1000;
    return d + radiusM <= cache.radiusM;
}

```

mosques.ts src/services

- Два сървъра: overpass-api.de, при грешка или претоварване – overpass.kumi.systems. Изчакване до 20 сек.
- Кеш в паметта на телефона за 24 ч. – само името, арабското име и адреса (в Истанбул до 10 км са стотици джамии).

- Без интернет – последното търсене за района, колкото и старо да е, с „Няма интернет. Показвам последното търсене тук – от днес, 14:20“.
- Не по-често от веднъж в минута за същия район – Overpass е безплатен, да не го натоварваме.

src/services/mosques.ts

195 реда

```
import AsyncStorage from '@react-native-async-storage/async-storage';

import { cacheCovers, overpassQuery, parseOverpass, privacyArea, type Mosque } from '@domain/mosques';

/**
 * Джамииите наблизко (етап 12) – мрежата и кешът.
 *
 * - Overpass API (OpenStreetMap): първо overpass-api.de, при грешка – overpass.kumi.systems.
 * - До сървъра отива само закръглено място (~1 км) – виж privacyArea.
 * - Кеш в паметта на телефона: последните търсения за 24 ч. Без интернет се показва
 * последното търсене за района, колкото и старо да е (с часа му).
 * - Пазят се само нужните тагове (име, арабско име, адрес) – в Истанбул до 10 км са стотици джамии.
 */

const ENDPOINTS = ['https://overpass-api.de/api/interpreter',
'https://overpass.kumi.systems/api/interpreter'];
const CACHE_KEY = 'ezan.mosques.v1';
const TIMEOUT_MS = 20_000;
const MAX_ENTRIES = 3;
/** По-често от това не питаме сървъра за същия район (Overpass е безплатен – да не го натоварваме). */
const MIN_INTERVAL_MS = 60_000;
const USER_AGENT = 'Ezan/1.0 (com.ilkoadamov.adhan; prayer times app)';

const KEEP_TAGS = /^(name(:?(bg|en|tr|ar))?)|addr:(street|housenumber|suburb|district|city|place))$/;

interface RawElement {
  type: string;
  id: number;
  lat?: number;
  lon?: number;
  center?: { lat: number; lon: number };
  tags?: Record<string, string>;
}

interface CacheEntry {
  /** Закръгленият център и радиусът на заявката (с +1,5 км). */
  lat: number;
  lon: number;
  radiusM: number;
  at: number;
  elements: RawElement[];
}

export interface MosquesResult {
  mosques: Mosque[];
  /** Кога са взети данните от сървъра. */
  at: number;
  /** Без връзка – показва се старо търсене. */
  offline: boolean;
}

export type MosquesError = 'offline' | 'server';

export class MosquesFetchError extends Error {
```

```

    constructor(public reason: MosquesError) {
        super(reason);
    }
}

let memory: CacheEntry[] | null = null;
let lastRequest = 0;
const inFlight = new Map<string, Promise<CacheEntry>>();

async function readCache(): Promise<CacheEntry[]> {
    if (memory) return memory;
    try {
        const raw = await AsyncStorage.getItem(CACHE_KEY);
        const parsed = raw ? (JSON.parse(raw) as CacheEntry[]) : [];
        memory = Array.isArray(parsed) ? parsed : [];
    } catch {
        memory = [];
    }
    return memory;
}

async function writeCache(entry: CacheEntry): Promise<void> {
    const list = await readCache();
    // по-новото търсене за същия район замества старото
    const rest = list.filter((e) => !(e.lat === entry.lat && e.lon === entry.lon && e.radiusM <=
entry.radiusM));
    memory = [entry, ...rest].slice(0, MAX_ENTRIES);
    try {
        await AsyncStorage.setItem(CACHE_KEY, JSON.stringify(memory));
    } catch {
        // паметта е пълна – остава само в RAM до затваряне на приложението
    }
}

function slim(elements: RawElement[]): RawElement[] {
    return elements.map((e) => {
        const tags: Record<string, string> = {};
        for (const [k, v] of Object.entries(e.tags ?? {})) if (KEEP_TAGS.test(k)) tags[k] = v;
        const out: RawElement = { type: e.type, id: e.id, tags };
        if (typeof e.lat === 'number') {
            out.lat = e.lat;
            out.lon = e.lon;
        } else if (e.center) out.center = e.center;
        return out;
    });
}

async function post(url: string, query: string): Promise<RawElement[]> {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), TIMEOUT_MS);
    try {
        const res = await fetch(url, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded; charset=UTF-8',
                Accept: 'application/json',
                'User-Agent': USER_AGENT,
            },
            body: `data=${encodeURIComponent(query)}`,
            signal: controller.signal,
        });
    }
}

```

```

    if (!res.ok) throw new MosquesFetchError('server');
    const json = (await res.json()) as { elements?: RawElement[]; remark?: string };
    // „runtime error: Query timed out“ идва с код 200 и празен списък – това е грешка, не „няма джамии“
    if (!Array.isArray(json.elements) || (json.remark && json.elements.length === 0)) {
        throw new MosquesFetchError('server');
    }
    return json.elements;
} catch (e) {
    if (e instanceof MosquesFetchError) throw e;
    // AbortError (изтекло време) и „Network request failed“ – няма връзка или е твърде бавна
    throw new MosquesFetchError('offline');
} finally {
    clearTimeout(timer);
}
}

async function download(lat: number, lon: number, radiusM: number): Promise<CacheEntry> {
    const area = privacyArea(lat, lon, radiusM);
    const query = overpassQuery(lat, lon, radiusM);
    lastRequest = Date.now();
    let error = new MosquesFetchError('offline');
    // първият сървър е претоварен или не отговаря → вторият
    for (const url of ENDPOINTS) {
        try {
            const elements = await post(url, query);
            const entry: CacheEntry = { ...area, at: Date.now(), elements: slim(elements) };
            await writeCache(entry);
            return entry;
        } catch (e) {
            error = e as MosquesFetchError;
        }
    }
    throw error;
}

function toResult(entry: CacheEntry, lang: string, offline: boolean): MosquesResult {
    return { mosques: parseOverpass(entry, lang), at: entry.at, offline };
}

/**
 * Джамииите около точката до `radiusM`.
 * 1. Пресен кеш (до 24 ч.) за района → от него, без мрежа.
 * 2. Иначе – от Overpass (и в кеша).
 * 3. Без връзка / грешка на сървъра → последното търсене за района, колкото и старо да е.
 * 4. Няма и такова → MosquesFetchError ('offline' или 'server').
 * `force` – „Опитай пак“: пропуска кеша (но не по-често от веднъж на минута).
 */
export async function loadMosques(
    lat: number,
    lon: number,
    radiusM: number,
    lang: string,
    force = false,
): Promise<MosquesResult> {
    const now = Date.now();
    const cache = await readCache();
    const fresh = cache.find((e) => cacheCovers(e, lat, lon, radiusM, now));
    const tooSoon = now - lastRequest < MIN_INTERVAL_MS;
    if (fresh && (!force || tooSoon)) return toResult(fresh, lang, false);

    const area = privacyArea(lat, lon, radiusM);

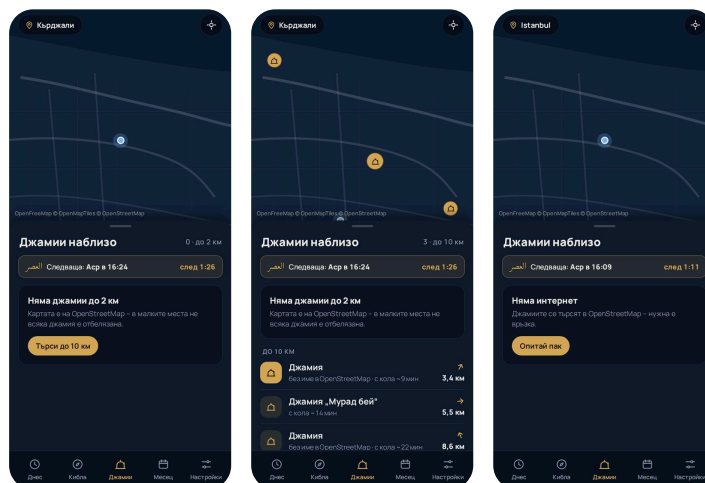
```

```

const key = `${area.lat},${area.lon},${area.radiusM}`;
let job = inFlight.get(key);
if (!job) {
  job = download(lat, lon, radiusM).finally(() => inFlight.delete(key));
  inFlight.set(key, job);
}
try {
  return toResult(await job, lang, false);
} catch (e) {
  const old = cache.find((x) => cacheCovers(x, lat, lon, radiusM, now, Infinity));
  if (old) return toResult(old, lang, true);
  throw e;
}
}

/** Последното търсене около точката (за мигновено показване, докато се зарежда новото). */
export async function cachedMosques(lat: number, lon: number, radiusM: number, lang: string):
Promise<MosquesResult | null> {
  const cache = await readCache();
  const old = cache.find((x) => cacheCovers(x, lat, lon, radiusM, Date.now(), Infinity));
  return old ? toResult(old, lang, false) : null;
}

```



Няма до 2 км · „Търси до 10 км“ · без интернет и без кеш

Ограничение: в OpenStreetMap не всяка джамия е отбелязана – в Турция и големите градове покритието е добро, в малки български села – по-слабо. Ако липсва джамия, всеки може да я добави на openstreetmap.org.

Стъпка 4

Картата

MapLibre рисува картата сам – не му трябва Google Play Services. Плочките са от OpenFreeMap (безплатни, без ключ), тъмен стил.

MosqueMap.tsx src/components/mosques

- Най-близките 20 джамии – със златна икона; останалите – златни точки (бързо и при стотици джамии).

- Камерата: ти и петте най-близки; избраната джамия – отблизо. Картата не се завърта – север е горе, за да съвпадат стрелките в списъка.
- Долу вляво – „OpenFreeMap © OpenMapTiles © OpenStreetMap“: задължителното указание на източниците.

src/components/mosques/MosqueMap.tsx

182 реда

```
import {
  Camera,
  GeoJSONSource,
  Layer,
  Map as MapView,
  Marker,
  type CameraRef,
  type LngLatBounds,
} from '@maplibre/maplibre-react-native';
import { useEffect, useMemo, useRef, useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';

import type { NearMosque } from '@domain/mosques';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { MosqueIcon } from '../icons';

import type { MosqueMapProps } from './mapTypes';

/**
 * Картата на джамиите (етап 12): MapLibre + OpenFreeMap (тъмен стил, без ключ и без Google -
 * работи и на Huawei). Най-близките джамии са с икона, останалите – златни точки.
 * Картата не се завърта (север е горе) – стрелките в списъка сочат спрямо север.
 */
const STYLE_URL = 'https://tiles.openfreemap.org/styles/dark';
/** Толкова най-близки джамии са с икона (истински изгледи); останалите – слой с точки (бързо). */
const ICON_PINS = 20;

export function MosqueMap({ center, me, mosques, selectedId, onSelect, recenter, bottomInset, attribution,
label }: MosqueMapProps) {
  const camera = useRef<CameraRef>(null);
  const [ready, setReady] = useState(false);

  const selected = mosques.find((m) => m.id === selectedId) ?? null;
  const pins = mosques.slice(0, ICON_PINS);
  const dots = useMemo<GeoJSON.FeatureCollection>(
    () => ({
      type: 'FeatureCollection',
      features: mosques.slice(ICON_PINS).map((m) => ({
        type: 'Feature',
        id: m.id,
        properties: { id: m.id },
        geometry: { type: 'Point', coordinates: [m.lon, m.lat] },
      })),
    }),
    [mosques],
  );

  // камерата: избраната джамия отблизо; иначе – ти и най-близките джамии
  useEffect(() => {
    const cam = camera.current;
    if (!ready || !cam) return;

```

```

const padding = { top: 90, bottom: bottomInset + 30, left: 40, right: 40 };
if (selected) {
  cam.easeTo({ center: [selected.lon, selected.lat], zoom: 16, padding: { ...padding, top: 60 },
duration: 600 });
  return;
}
const near = mosques.slice(0, 5);
if (near.length === 0) {
  cam.easeTo({ center: [center.lon, center.lat], zoom: 14, padding, duration: 600 });
  return;
}
cam.fitBounds(boundsOf([center, ...near]), { padding, duration: 600 });
}, [ready, selected, mosques, center, recenter, bottomInset]);

return (
  <View style={StyleSheet.absoluteFill} accessible accessibilityLabel={label}>
    <MapView
      style={StyleSheet.absoluteFill}
      mapStyle={STYLE_URL}
      attribution={false}
      logo={false}
      compass={false}
      touchRotate={false}
      touchPitch={false}
      onDidFinishLoadingMap={() => setReady(true)}
    >
      <Camera ref={camera} initialViewState={{ center: [center.lon, center.lat], zoom: 14 }} minZoom={9}
maxZoom={18.5} />

      <GeoJSONSource
        id="mosques"
        data={dots}
        onPress={(e) => {
          const id = e.nativeEvent.features[0]?.properties?.id;
          if (typeof id === 'string') onSelect(id);
        }}
      >
        <Layer
          type="circle"
          id="mosque-dots"
          paint={{
            'circle-radius': 6,
            'circle-color': colors.gold,
            'circle-opacity': 0.85,
            'circle-stroke-width': 1.5,
            'circle-stroke-color': colors.goldInk,
          }}
        />
      </GeoJSONSource>

      {pins.map((m) => (
        <Marker key={m.id} id={m.id} lngLat={[m.lon, m.lat]} onPress={() => onSelect(m.id)} selected=
{m.id === selectedId}>
          <Pin mosque={m} selected={m.id === selectedId} nearest={m === mosques[0]} />
        </Marker>
      ))}
      {selected && !pins.includes(selected) && (
        <Marker key={`sel-${selected.id}`} lngLat={[selected.lon, selected.lat]} selected>
          <Pin mosque={selected} selected nearest={false} />
        </Marker>
      )}
    )}
  </View>
)

```

```

    {me && (
      <Marker key="me" lngLat={[me.lon, me.lat]}>
        <View style={styles.meHalo}>
          <View style={styles.me} />
        </View>
      </Marker>
    )}
  </MapView>
  <View style={[styles.attributionBox, { bottom: bottomInset + 6 }]} pointerEvents="none">
    <Text style={styles.attribution}>{attribution}</Text>
  </View>
</View>
);
}

function Pin({ selected, nearest }: { mosque: NearMosque; selected: boolean; nearest: boolean }) {
  const size = selected ? 36 : nearest ? 30 : 26;
  return (
    <View
      style={[
        styles.pin,
        { width: size, height: size, borderRadius: size / 2 },
        selected && styles.pinSelected,
        !selected && !nearest && styles.pinDim,
      ]}
    >
      <MosqueIcon size={selected ? 20 : 15} color={colors.goldInk} strokeWidth={2} />
    </View>
  );
}

function boundsOf(points: { lat: number; lon: number }[]): LngLatBounds {
  let w = Infinity;
  let s = Infinity;
  let e = -Infinity;
  let n = -Infinity;
  for (const p of points) {
    w = Math.min(w, p.lon);
    e = Math.max(e, p.lon);
    s = Math.min(s, p.lat);
    n = Math.max(n, p.lat);
  }
  return [w, s, e, n];
}

const styles = StyleSheet.create({
  pin: {
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: colors.gold,
    borderWidth: 1.5,
    borderColor: 'rgba(26,20,8,0.6)',
  },
  pinSelected: { borderWidth: 4, borderColor: 'rgba(212,168,87,0.35)' },
  pinDim: { backgroundColor: 'rgba(212,168,87,0.88)' },
  meHalo: {
    width: 26,
    height: 26,
    borderRadius: 13,
    backgroundColor: 'rgba(120,190,255,0.22)',
  },
});

```

```

    alignItems: 'center',
    justifyContent: 'center',
  },
  me: { width: 13, height: 13, borderRadius: 7, backgroundColor: '#6FB6FF', borderWidth: 2.5, borderColor:
colors.text },
  attributionBox: { position: 'absolute', left: 10 },
  attribution: {
    fontFamily: fonts.medium,
    fontSize: 9.5,
    color: 'rgba(242,239,232,0.6)',
  },
});

```

MosqueMap.web.tsx src/components/mosques

- Само за прегледа на екраните в браузъра – MapLibre няма уеб версия за React Native.

src/components/mosques/MosqueMap.web.tsx

96 реда

```

import { Pressable, StyleSheet, Text, useWindowDimensions, View } from 'react-native';
import Svg, { Path } from 'react-native-svg';

import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { MosqueIcon } from '../icons';

import type { MosqueMapProps } from './mapTypes';

/**
 * Уеб версията (само за прегледа на екраните): MapLibre RN няма уеб част, затова точките
 * се рисуват върху схематичен фон в същия мащаб. В приложението е истинската карта.
 */
export function MosqueMap({ center, me, mosques, selectedId, onSelect, bottomInset, attribution, label,
height }: MosqueMapProps) {
  const { width } = useWindowDimensions();
  const selected = mosques.find((m) => m.id === selectedId) ?? null;
  const usable = height - bottomInset - 90;

  // мащаб: избраната отблизо, иначе - ти и 5-те най-близки
  const focus = selected ? [selected] : [center, ...mosques.slice(0, 5)];
  const k = Math.cos((center.lat * Math.PI) / 180);
  const xs = focus.map((p) => p.lon * k);
  const ys = focus.map((p) => p.lat);
  const spanX = Math.max(...xs) - Math.min(...xs) || 0.004;
  const spanY = Math.max(...ys) - Math.min(...ys) || 0.004;
  const scale = selected ? 60000 : Math.min((width - 80) / spanX, usable / spanY);
  const cx = selected ? selected.lon * k : (Math.max(...xs) + Math.min(...xs)) / 2;
  const cy = selected ? selected.lat : (Math.max(...ys) + Math.min(...ys)) / 2;
  const toXY = (lat: number, lon: number) => ({
    x: width / 2 + (lon * k - cx) * scale,
    y: 90 + usable / 2 - (lat - cy) * scale,
  });

  return (
    <View style={[StyleSheet.absoluteFill, styles.bg]} accessibilityLabel={label}>
      <Svg width={width} height={height} style={StyleSheet.absoluteFill}>
        <Path d={`M0 0 H${width} V110 C${width * 0.8} 104 ${width * 0.6} 92 0 46 Z`} fill="#0A1A2E" />
        <Path d={`M20 120 C90 130 160 150 250 170 S${width - 30} 200 ${width} 205`}
stroke="rgba(255,255,255,0.16)" strokeWidth={5} fill="none" />
      </Svg>
    </View>
  );
}

```

```

        <Path d={`M0 230 C80 222 170 230 240 250 S${width - 60} 290 ${width - 30} 330`}
stroke="rgba(255,255,255,0.13)" strokeWidth={4} fill="none" />
        <Path d="M150 112 L170 430 M70 140 L95 430" stroke="rgba(255,255,255,0.09)" strokeWidth={3}
fill="none" />
        <Path d={`M0 330 C100 320 200 340 ${width} 380`} stroke="rgba(255,255,255,0.09)" strokeWidth={3}
fill="none" />
        <Path d={`M0 180 H${width} M0 280 H${width} M40 120 V430 M120 120 V430 M190 120 V430 M320 140
V430`} stroke="rgba(255,255,255,0.035)" strokeWidth={1} fill="none" />
    </Svg>
    {mosques.slice(0, 60).map((m, i) => {
      const { x, y } = toXY(m.lat, m.lon);
      if (x < -20 || x > width + 20 || y < -20 || y > height) return null;
      const sel = m.id === selectedId;
      const size = sel ? 36 : i === 0 ? 30 : i < 20 ? 26 : 12;
      return (
        <Pressable
          key={m.id}
          onPress={() => onSelect(m.id)}
          style={[styles.pin, { left: x - size / 2, top: y - size / 2, width: size, height: size,
borderRadius: size / 2 }, sel && styles.sel]}
          >
          {size > 12 && <MosqueIcon size={sel ? 20 : 15} color={colors.goldInk} strokeWidth={2} />}
        </Pressable>
      );
    })}
    {me &&
      (() => {
        const { x, y } = toXY(me.lat, me.lon);
        return (
          <View style={[styles.meHalo, { left: x - 13, top: y - 13 }]}>
            <View style={styles.me} />
          </View>
        );
      })()
    }
    <Text style={[styles.attribution, { bottom: bottomInset + 6 }]}>{attribution}</Text>
  </View>
);
}

const styles = StyleSheet.create({
  bg: { backgroundColor: '#13233A', overflow: 'hidden' },
  pin: {
    position: 'absolute',
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: colors.gold,
    borderWidth: 1.5,
    borderColor: 'rgba(26,20,8,0.6)',
  },
  sel: { borderWidth: 4, borderColor: 'rgba(212,168,87,0.35)' },
  meHalo: {
    position: 'absolute',
    width: 26,
    height: 26,
    borderRadius: 13,
    backgroundColor: 'rgba(120,190,255,0.22)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  me: { width: 13, height: 13, borderRadius: 7, backgroundColor: '#6FB6FF', borderWidth: 2.5, borderColor:
colors.text },

```

```
attribution: { position: 'absolute', left: 10, fontFamily: fonts.medium, fontSize: 9.5, color:
'rgba(242,239,232,0.6)' },
});
```

Без интернет новите части на картата не се зареждат (сиво), но списъкът от кеша и „Упътване“ се показват.

Стъпка 5

Упътване и изборът на навигация

Собствена навигация няма – Езан дава координатите на приложението, което ти харесва.

ПРИЛОЖЕНИЕТО	ПЕША	КАК СЕ ОТВАРЯ
Автоматично	да	Android – картите по подразбиране (geo:); iPhone – Apple Maps
Google Maps	да	google.navigation: / comgooglemaps://
Waze	не	waze.com/ul – винаги с кола
HERE WeGo	да	share.here.com
Petal Maps	да	petalmaps:// – само Android (Huawei)
Organic Maps	да	om:// – работи и без интернет

- **Първия път** „Упътване“ пита „Отвори с...“ – само инсталираните приложения и „Автоматично“. „Запомни избора“ е отменено.
- **Изборът се пази завинаги** – с другите настройки, и след затваряне на приложението и рестарт на телефона. Смяна: Настройки → Джамии наблизно → Навигация.
- **Изтрито приложение:** „Упътване“ разбира, че го няма, и пита отново („Waze вече не е на телефона...“).
- **Неинсталираните** в Настройки са с „Изтегли“ – Google Play / App Store (на Huawei без Google Play – уеб страницата).
- **Режимът:** до 2,5 км – пеша, по-далеч – с кола. Waze няма пешеходна навигация – и за 300 м води с колата (написано под списъка).

navApps.ts src/domain

- Приложенията и връзките за всяко – проверени по документацията им.

src/domain/navApps.ts

174 реда

```
/**
 * Приложенията за навигация (етап 12) – с кое се отваря „Упътване“ до джамия.
 * Собствена навигация няма: даваме координатите на избраното приложение.
 *
 * Връзките са проверени по документацията на всяко приложение (10.2026):
 * - Google Maps – google.navigation: (Android), comgooglemaps:// (iPhone), резерва
https://www.google.com/maps/dir/
 * - Waze – https://waze.com/ul?ll=...&navigate=yes; няма режим пеша (само с кола)
 * - HERE WeGo – https://share.here.com/r/mylocation/...?m=w/d
 * - Petal Maps (Huawei, само Android) – petalmaps://route?daddr=...&type=walk|drive&coordinateType=0
 * (без coordinateType=0 координатите се четат в китайската система GCJ02)
 * - Organic Maps – om://route?sll=...&saddr=...&dll=...&daddr=...&type=pedestrian|vehicle
```

```

* (иска и началната точка, параметрите – точно в този ред)
* - Apple Maps (iPhone) – https://maps.apple.com/directions?destination=...&mode=walking/driving (iOS 18.4+)
* - Автоматично: Android – geo: (телефонът пита или отваря картите по подразбиране); iPhone – Apple Maps
*/

export type NavAppId = 'auto' | 'google' | 'waze' | 'here' | 'petal' | 'organic' | 'apple';
export type NavPlatform = 'android' | 'ios';
export type TravelMode = 'walk' | 'drive';

export interface NavApp {
  id: NavAppId;
  /** Името (марките не се превеждат); за 'auto' – от текстовете на приложението. */
  name: string;
  /** Буквата на плочката, докато няма истинска икона. */
  letter: string;
  /** Тъмен цвят на плочката. */
  color: string;
  /** Може ли да води пеша. */
  walking: boolean;
  platforms: readonly NavPlatform[];
  /** Android: пакетът – за проверка дали е инсталирано и за отваряне точно в него. */
  androidPackage?: string;
  /** iPhone: схемата за проверка дали е инсталирано (LSApplicationQueriesSchemes). */
  iosScheme?: string;
  /** App Store. */
  iosStore?: string;
}

export const NAV_APPS: readonly NavApp[] = [
  { id: 'auto', name: '', letter: 'A', color: '#3A4A63', walking: true, platforms: ['android', 'ios'] },
  {
    id: 'google',
    name: 'Google Maps',
    letter: 'G',
    color: '#3E6B5A',
    walking: true,
    platforms: ['android', 'ios'],
    androidPackage: 'com.google.android.apps.maps',
    iosScheme: 'comgooglemaps',
    iosStore: 'https://apps.apple.com/app/id585027354',
  },
  {
    id: 'waze',
    name: 'Waze',
    letter: 'W',
    color: '#3F6A86',
    walking: false,
    platforms: ['android', 'ios'],
    androidPackage: 'com.waze',
    iosScheme: 'waze',
    iosStore: 'https://apps.apple.com/app/id323229106',
  },
  {
    id: 'here',
    name: 'HERE WeGo',
    letter: 'H',
    color: '#4B4F7A',
    walking: true,
    platforms: ['android', 'ios'],
    androidPackage: 'com.here.app.maps',
    iosStore: 'https://apps.apple.com/app/id955837609',
  },

```

```

    },
    {
      id: 'petal',
      name: 'Petal Maps',
      letter: 'P',
      color: '#7A4B4B',
      walking: true,
      platforms: ['android'],
      androidPackage: 'com.huawei.maps.app',
    },
    {
      id: 'organic',
      name: 'Organic Maps',
      letter: 'O',
      color: '#4F6B3A',
      walking: true,
      platforms: ['android', 'ios'],
      androidPackage: 'app.organicmaps',
      iosScheme: 'om',
      iosStore: 'https://apps.apple.com/app/id1567437057',
    },
    { id: 'apple', name: 'Apple Maps', letter: 'A', color: '#4A5568', walking: true, platforms: ['ios'] },
  ],

  export function navAppsFor(platform: NavPlatform): NavApp[] {
    // на iPhone „Автоматично“ е Apple Maps – отделен ред „Apple Maps“ би бил същото
    return NAV_APPS.filter((a) => a.platforms.includes(platform) && !(platform === 'ios' && a.id === 'apple'));
  }

  export function navApp(id: NavAppId): NavApp {
    return NAV_APPS.find((a) => a.id === id) ?? NAV_APPS[0];
  }

  export interface Destination {
    lat: number;
    lon: number;
    name: string;
  }

  export interface Origin {
    lat: number;
    lon: number;
    name: string;
  }

  const c = (n: number) => n.toFixed(6);
  const enc = encodeURIComponent;

  /**
   * Връзката за упътване. Режимът: пеша за близките джамии, с кола за далечните;
   * приложение без режим пеша (Waze) – винаги с кола.
   */
  export function navUrl(
    id: NavAppId,
    platform: NavPlatform,
    to: Destination,
    mode: TravelMode,
    from?: Origin | null,
  ): string {
    const ll = `${c(to.lat)},${c(to.lon)}`;

```

```

const walk = mode === 'walk';
switch (id) {
  case 'google':
    return platform === 'android'
      ? `google.navigation:q=${ll}&mode=${walk ? 'w' : 'd'}`
      : `comgooglemaps://?daddr=${ll}&directionsmode=${walk ? 'walking' : 'driving'}`;
  case 'waze':
    return `https://waze.com/ul?ll=${ll}&navigate=yes&utm_source=ezan`;
  case 'here':
    return `https://share.here.com/r/mylocation/${ll},${enc(to.name)}?m=${walk ? 'w' : 'd'}`;
  case 'petal':
    return `petalmaps://route?daddr=${ll}&type=${walk ? 'walk' : 'drive'}&coordinateType=0&utm_source=com.ilkoadamov.adhan`;
  case 'organic':
    // без началната точка Organic Maps отхвърля връзката - тогава само показва мястото
    return from
      ? `om://route?sll=${c(from.lat)},${c(from.lon)}&saddr=${enc(from.name)}&dll=${ll}&daddr=${enc(to.name)}&type=${walk ? 'pedestrian' : 'vehicle'}`
      : `om://map?ll=${ll}&n=${enc(to.name)}`;
  case 'apple':
    return `https://maps.apple.com/directions?destination=${ll}&mode=${walk ? 'walking' : 'driving'}`;
  case 'auto':
  default:
    return platform === 'android'
      ? `geo:0,0?q=${ll}(${enc(to.name)})`
      : `https://maps.apple.com/directions?destination=${ll}&mode=${walk ? 'walking' : 'driving'}`;
}
}

/** Режимът за приложението: Waze - само с кола. */
export function modeFor(id: NavAppId, wanted: TravelMode): TravelMode {
  return navApp(id).walking ? wanted : 'drive';
}

/** Къде да се изтегли приложението. */
export function storeUrl(app: NavApp, platform: NavPlatform): string | null {
  if (platform === 'android') return app.androidPackage ? `market://details?id=${app.androidPackage}` : null;
  return app.iosStore ?? null;
}

/** Резерва, ако магазинът не се отвори (няма Google Play - Huawei). */
export function storeWebUrl(app: NavApp): string | null {
  return app.androidPackage ? `https://play.google.com/store/apps/details?id=${app.androidPackage}` : null;
}

```

navigation.ts src/services

- Проверка дали е инсталирано, отваряне точно в избраното приложение, магазинът, връзката за „Сподели“.

src/services/navigation.ts

124 реда

```

import { Linking, Platform } from 'react-native';

import { appIcon, isAppInstalled, openInApp } from '../../modules/adhan-native';

import {
  modeFor,

```

```

navApp,
navAppsFor,
navUrl,
storeUrl,
storeWebUrl,
type Destination,
type NavApp,
type NavAppId,
type NavPlatform,
type Origin,
type TravelMode,
} from '@domain/navApps';

/**
 * Отваряне на „Упътване“ в избраното приложение за навигация (етап 12).
 * Android: точно в приложението (Intent с пакет) – проверката „инсталирано ли е“ е в native частта
 * (пакетите са изброени в <queries> в AndroidManifest.xml).
 * iPhone: по схемата на приложението (LSApplicationQueriesSchemes в app.json).
 */

export const navPlatform: NavPlatform = Platform.OS === 'ios' ? 'ios' : 'android';

/** Приложенията за този телефон. */
export function availableNavApps(): NavApp[] {
  return navAppsFor(navPlatform);
}

/**
 * Инсталирано ли е. „Автоматично“ и Apple Maps – винаги.
 * HERE WeGo на iPhone няма схема за проверка – връзката е https и без приложението се отваря в браузъра.
 */
export async function isInstalled(id: NavAppId): Promise<boolean> {
  const app = navApp(id);
  if (id === 'auto' || id === 'apple') return true;
  if (navPlatform === 'android') {
    if (!app.androidPackage) return false;
    return isAppInstalled(app.androidPackage) ?? true; // стар build без native частта – не знаем
  }
  if (!app.iosScheme) return id === 'here';
  return Linking.canOpenURL(`${app.iosScheme}://`).catch(() => false);
}

/** Кои приложения са инсталирани – за списъка в Настройки и листа „Отвори с...“ */
export async function installedMap(): Promise<Partial<Record<NavAppId, boolean>>> {
  const apps = availableNavApps();
  const values = await Promise.all(apps.map((a) => isInstalled(a.id)));
  return Object.fromEntries(apps.map((a, i) => [a.id, values[i]]));
}

const icons = new Map<string, string | null>();

/** Истинската икона на приложението (Android) като data: URI; null – буквата на плочката. */
export function navAppIcon(id: NavAppId): string | null {
  const pkg = navApp(id).androidPackage;
  if (navPlatform !== 'android' || !pkg) return null;
  if (!icons.has(pkg)) {
    const b64 = appIcon(pkg, 96);
    icons.set(pkg, b64 ? `data:image/png;base64,${b64}` : null);
  }
  return icons.get(pkg) ?? null;
}

```

```

async function open(url: string, pkg: string | null): Promise<boolean> {
  if (navPlatform === 'android') {
    const r = openInApp(url, pkg);
    if (r !== null) return r;
  }
  try {
    await Linking.openURL(url);
    return true;
  } catch {
    return false;
  }
}

/**
 * Отваря упътването. 'missing' – приложението вече го няма (изтрито) → питаме отново.
 * Режимът: пеша за близките джамии, с кола за далечните; Waze – винаги с кола.
 */
export async function openDirections(
  id: NavAppId,
  to: Destination,
  wanted: TravelMode,
  from?: Origin | null,
): Promise<'ok' | 'missing'> {
  const app = navApp(id);
  const mode = modeFor(id, wanted);
  if (id !== 'auto' && id !== 'apple' && !(await isInstalled(id))) return 'missing';

  const url = navUrl(id, navPlatform, to, mode, from);
  if (await open(url, app.androidPackage ?? null)) return 'ok';

  if (id === 'auto') {
    // на телефона няма нито едно приложение за карти – Google Maps в брауъра
    const travel = mode === 'walk' ? 'walking' : 'driving';
    const web = `https://www.google.com/maps/dir/?api=1&destination=${to.lat.toFixed(6)},${to.lon.toFixed(6)}&travelmode=${travel}`;
    return (await open(web, null)) ? 'ok' : 'missing';
  }
  return 'missing';
}

/** Магазинът за приложението (Google Play / App Store); без Google Play (Huawei) – уеб страницата. */
export async function openStore(id: NavAppId): Promise<void> {
  const app = navApp(id);
  const url = storeUrl(app, navPlatform);
  if (!url) return;
  try {
    await Linking.openURL(url);
  } catch {
    const web = storeWebUrl(app);
    if (web) await Linking.openURL(web).catch(() => {});
  }
}

/** Връзка към мястото за „Сподели“ – отваря се в картите на всеки телефон. */
export function placeLink(lat: number, lon: number): string {
  return `https://www.google.com/maps/search/?api=1&query=${lat.toFixed(6)},${lon.toFixed(6)}`;
}

```

settings.ts src/store

- `navApp` и `navRemembered` – изборът; `forgetNavApp` – когато приложението е изтрито.

src/store/settings.ts

212 реда

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,
  DEFAULT_METHOD,
} from '@config/defaults';
import {
  methodForCountry,
  type AsrMadhab,
  type CalculationMethodId,
  type HighLatRuleId,
  type MethodChoice,
} from '@domain/calc';
import type { AppLocation } from '@domain/location';
import type { NavAppId } from '@domain/navApps';
import { getPlace, PLACES_COUNT, placeToLocation } from '@domain/places';
import { countryOf } from '@domain/resolve';
import type { PrayerId } from '@domain/prayers';
import type { TimesOptions } from '@domain/times';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни
/** Възможните напомняния преди молитвата (минути); 0 = без напомняне. */
export const REMINDER_CHOICES = [0, 5, 10, 15, 20, 30] as const;

const NO_OFFSETS: Record<PrayerId, number> = {
  fajr: 0,
  sunrise: 0,
  dhuhr: 0,
  asr: 0,
  maghrib: 0,
  isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
  /** true – мястото идва от GPS; false – избрано ръчно. */
  autoLocation: boolean;
  /** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
  gpsLocation: AppLocation | null;
  /** Ръчно избраното място. */
  manualLocation: AppLocation | null;

  /** 'auto' – според държавата (виж methodForCountry). */
  method: MethodChoice;
  madhab: AsrMadhab;
  highLatRule: HighLatRuleId;
  offsets: Record<PrayerId, number>;
  hijriAdjust: number;
}
```

```

/** Напомняне X минути преди молитвата; 0 = без. */
reminderMinutes: number;
/** Вибрация при известие (Android; на iPhone решава телефонът). */
vibrate: boolean;
/** С кое приложение се отваря „Упътване“ до джамия (етап 12). */
navApp: NavAppId;
/** Изборът е запомнен (листа „Отвори с...“ или Настройки → Навигация); false – питаме при „Упътване“. */
navRemembered: boolean;

setAutoLocation: (on: boolean) => void;
setGpsLocation: (loc: AppLocation) => void;
chooseLocation: (loc: AppLocation) => void;
setMethod: (m: MethodChoice) => void;
setMadhab: (m: AsrMadhab) => void;
setHighLatRule: (r: HighLatRuleId) => void;
changeOffset: (id: PrayerId, delta: number) => void;
resetOffsets: () => void;
changeHijriAdjust: (delta: number) => void;
setReminderMinutes: (n: number) => void;
setVibrate: (on: boolean) => void;
/** Запомня приложението за навигация – остава и след затваряне на приложението и рестарт. */
setNavApp: (id: NavAppId) => void;
/** Забравя избора (избраното приложение е изтрито) – следващото „Упътване“ пак пита. */
forgetNavApp: () => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
      hijriAdjust: 0,
      reminderMinutes: 0,
      vibrate: true,
      navApp: 'auto',
      navRemembered: false,

      setAutoLocation: (on) => set({ autoLocation: on }),
      setGpsLocation: (loc) => set({ gpsLocation: loc }),
      // ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
      chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
      setMethod: (method) => set({ method }),
      setMadhab: (madhab) => set({ madhab }),
      setHighLatRule: (highLatRule) => set({ highLatRule }),
      changeOffset: (id, delta) =>
        set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
      resetOffsets: () => set({ offsets: NO_OFFSETS }),
      changeHijriAdjust: (delta) =>
        set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
      setReminderMinutes: (reminderMinutes) => set({ reminderMinutes }),
      setVibrate: (vibrate) => set({ vibrate }),
      setNavApp: (navApp) => set({ navApp, navRemembered: true }),
      forgetNavApp: () => set({ navApp: 'auto', navRemembered: false }),
    }),
    {
      name: 'ezan.settings',
    }
  )
)

```

```

// 2 (етап 5): методът по подразбиране е „Автоматично“ – старият „Диянет“ по подразбиране
// става „Автоматично“ (за Турция и Европа пак е Диянет)
// 3 (етап 10): турски език – запазените места в България получават и турското име
version: 3,
migrate: (persisted, version) => {
    const state = persisted as Partial<SettingsState>;
    if (version < 2 && state.method === 'Turkey') state.method = 'auto';
    if (version < 3) {
        state.manualLocation = withTurkishName(state.manualLocation);
        state.gpsLocation = withTurkishName(state.gpsLocation);
    }
    return state as SettingsState;
},
storage: createJSONStorage(() => AsyncStorage),
// новите полета (reminderMinutes, vibrate, navApp, navRemembered) липсват в старите записи – тогава
// важат стойностите по подразбиране
partialize: ({
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
    reminderMinutes,
    vibrate,
    navApp,
    navRemembered,
}) => ({
    reminderMinutes,
    vibrate,
    navApp,
    navRemembered,
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
}),
},
),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {
    const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
    return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

type TimesParts = Pick<SettingsState, 'method' | 'madhab' | 'highLatRule' | 'offsets'> & { location:
AppLocation };

/** Опциите за часовете и техният „отпечатък“ `key` – по него се решава кога да се преизчисли. */
export function buildTimesOptions({ location, method, madhab, highLatRule, offsets }: TimesParts): {
    options: TimesOptions;
    key: string;
} {

```

```

const options: TimesOptions = { location, method: resolveMethod(method, location), madhab, highLatRule,
offsets };
const key = JSON.stringify([
  location.id,
  location.latitude,
  location.longitude,
  location.source,
  location.muftiShift,
  options.method,
  madhab,
  highLatRule,
  offsets,
]);
return { options, key };
}

/** „Автоматично“ → методът за държавата на мястото; иначе избраният. */
export function resolveMethod(method: MethodChoice, location: AppLocation): CalculationMethodId {
  return method === 'auto' ? methodForCountry(countryOf(location)) : method;
}

/** Същото, направо от състоянието – без React (ползва се и от известията във фонов режим). */
export function selectTimesOptions(s: SettingsState): { options: TimesOptions; key: string } {
  return buildTimesOptions({ ...s, location: selectLocation(s) });
}

/** Място в България, запазено преди етап 10: наново от списъка – с турското име. */
function withTurkishName<T extends AppLocation | null | undefined>(loc: T): T {
  if (!loc || !loc.id.startsWith('bg-')) return loc;
  const index = Number(loc.id.slice(3));
  if (!Number.isInteger(index) || index < 0 || index >= PLACES_COUNT) return loc;
  try {
    const fresh = placeToLocation(getPlace(index));
    return fresh.names.bg === loc.names.bg ? ({ ...loc, names: fresh.names, detail: fresh.detail } as T) :
loc;
  } catch {
    return loc;
  }
}

```

AdhanNativeModule.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- `isAppInstalled` – има ли пакета на телефона.
- `openInApp` – отваря връзката точно в това приложение (Intent с пакет); без пакет – каквото избере телефонът.
- `appIcon` – истинската икона на приложението за списъка (иначе – цветна плочка с буква).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AdhanNativeModule.kt

243 реда

```

package expo.modules.adhannative

import android.app.AlarmManager
import android.app.NotificationManager
import android.content.ActivityNotFoundException
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.PowerManager

```

```

import android.provider.Settings
import expo.modules.kotlin.Promise
import expo.modules.kotlin.exception.Exceptions
import expo.modules.kotlin.modules.Module
import expo.modules.kotlin.modules.ModuleDefinition

/**
 * Native модульът на Езан (само Android):
 * - етап 4: точни известия (Android 12+);
 * - етап 5: алармата с пълния езан (AlarmManager.setAlarmClock + foreground service +
 * екран „Аларма“), аларма на цял екран (Android 14+), работа на заден план
 * (оптимизация на батерията) и местоположение без Google услуги (Huawei);
 * - етап 6: звуците; етап 8: widget-ите (PrayerWidgets.kt); етап 12: навигацията до джамия.
 */
class AdhanNativeModule : Module() {
    private val context: Context
    get() = appContext.reactContext ?: throw Exceptions.ReactContextLost()

    private val packageUri: Uri
    get() = Uri.parse("package:${context.packageName}")

    private fun open(intent: Intent, fallback: Intent? = null) {
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        try {
            context.startActivity(intent)
        } catch (e: ActivityNotFoundException) {
            val f = fallback ?: Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
            context.startActivity(f.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
        }
    }

    override fun definition() = ModuleDefinition {
        Name("AdhanNative")

        /* ----- точни аларми (етап 4) */

        // Може ли приложението да планира точни аларми. До Android 12 – винаги да.
        // От Android 14 разрешението „Аларми и напомняния“ е изключено по подразбиране.
        Function("canScheduleExactAlarms") {
            if (Build.VERSION.SDK_INT < Build.VERSION_CODES.S) {
                true
            } else {
                val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
                alarmManager.canScheduleExactAlarms()
            }
        }

        // Отваря системния екран „Аларми и напомняния“ за Езан.
        // Ако производителят го е махнал – екрана с информация за приложението.
        Function("openExactAlarmSettings") {
            val intent =
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
                    Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM, packageUri)
                } else {
                    Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
                }
            open(intent)
        }

        /* ----- алармата с езана (етап 5) */
    }
}

```

```

// Заменя всички аларми (JSON масив, виж AlarmData). Връща колко са планирани.
Function("setAlarms") { json: String ->
    AlarmScheduler.setAlarms(context, json)
}

// Запазеният списък (JSON) – за „Проверка на известията“.
Function("getAlarms") {
    AlarmScheduler.storedJson(context)
}

// Последните звъннали аларми (JSON): планиран и истински час.
Function("getAlarmHistory") {
    AlarmScheduler.historyJson(context)
}

// Пробна аларма след delayMs.
Function("testAlarm") { json: String, delayMs: Double ->
    AlarmScheduler.scheduleTest(context, json, delayMs.toLong())
}

Function("stopAlarm") {
    AlarmService.command(context, AlarmService.ACTION_STOP)
}

/* ----- звуци (етап 6) */

// Преслушване: вграден звук (res/raw) или свой файл (file://...).
Function("previewSound") { source: String ->
    SoundPreview.play(context, source)
}

Function("stopPreview") {
    SoundPreview.stop()
}

// Свой кратък звук: откъс до maxSec (срез на пауза, заглъхване) в Notifications/Ezan.
AsyncFunction("prepareShortSound") { uri: String, title: String, maxSec: Double ->
    val r = ShortSound.prepare(context, uri, title, maxSec)
    mapOf(
        "uri" to r.uri,
        "duration" to r.duration,
        "originalDuration" to r.originalDuration,
        "trimmed" to r.trimmed,
    )
}

Function("deleteShortSound") { uri: String ->
    ShortSound.delete(context, uri)
}

// Канал за известие със свой звук (content://...).
Function("createSoundChannel") { id: String, name: String, uri: String, alarm: Boolean, vibrate: Boolean, pattern: List<Double> ->
    ShortSound.createChannel(context, id, name, uri, alarm, vibrate, pattern.map { it.toLong() }.toLongArray())
}

// Дължината на звуков файл в секунди; -1 – не е звук или не може да се прочете.
AsyncFunction("getAudioDuration") { uri: String ->
    SoundSource.durationSec(context, uri)
}

```

```

/* ----- аларма на цял екран (Android 14+) */

Function("canUseFullScreenIntent") {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        true
    } else {
        val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        nm.canUseFullScreenIntent()
    }
}

Function("openFullScreenIntentSettings") {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        open(Intent(Settings.ACTION_MANAGE_APP_USE_FULL_SCREEN_INTENT, packageUri))
    } else {
        open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
    }
}

/* ----- работа на заден план */

// true - телефонът не ограничава Езан (оптимизацията на батерията е изключена за него).
Function("isIgnoringBatteryOptimizations") {
    val pm = context.getSystemService(Context.POWER_SERVICE) as PowerManager
    pm.isIgnoringBatteryOptimizations(context.packageName)
}

// Настройките на Езан в телефона (оттам: Батерия → Без ограничения).
Function("openAppSettings") {
    open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
}

// Списъкът „Оптимизация на батерията“ на Android.
Function("openBatteryOptimizationSettings") {
    open(Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATIONS_SETTINGS))
}

/* ----- навигация до джамия (етап 12) */

// Инсталирано ли е приложението (пакетите са изброени в <queries> в AndroidManifest.xml).
Function("isAppInstalled") { pkg: String ->
    try {
        context.packageManager.getPackageInfo(pkg, 0)
        true
    } catch (e: Exception) {
        false
    }
}

// Отваря връзката точно в това приложение (или – без пакет – в каквото избере телефонът).
// false – няма такова приложение.
Function("openInApp") { url: String, pkg: String? ->
    val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url)).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    if (!pkg.isNullOrEmpty()) intent.setPackage(pkg)
    try {
        context.startActivity(intent)
        true
    } catch (e: ActivityNotFoundException) {
        false
    } catch (e: Exception) {

```

```

        false
    }
}

// Иконата на приложението като PNG (base64) - за списъка с навигации. null - няма го.
Function("appIcon") { pkg: String, size: Int ->
    try {
        val drawable = context.packageManager.getApplicationIcon(pkg)
        val px = size.coerceIn(24, 256)
        val bitmap = android.graphics.Bitmap.createBitmap(px, px, android.graphics.Bitmap.Config.ARGB_8888)
        val canvas = android.graphics.Canvas(bitmap)
        drawable.setBounds(0, 0, px, px)
        drawable.draw(canvas)
        val out = java.io.ByteArrayOutputStream()
        bitmap.compress(android.graphics.Bitmap.CompressFormat.PNG, 100, out)
        bitmap.recycle()
        android.util.Base64.encodeToString(out.toByteArray(), android.util.Base64.NO_WRAP)
    } catch (e: Exception) {
        null
    }
}

/* ----- widget-и (етап 8) */

// Кадрите на widget-ите (JSON от src/domain/widget.ts): записва ги и прерисува widget-ите.
Function("setWidgetData") { json: String ->
    PrayerWidgets.setData(context, json)
}

// Колко widget-а на Езан има на началния екран.
Function("getWidgetCount") {
    PrayerWidgets.count(context)
}

/* ----- местоположение без Google (Huawei) */

Function("hasGoogleServices") {
    NativeLocation.hasGoogleServices(context)
}

AsyncFunction("getCurrentLocation") { timeoutMs: Double, promise: Promise ->
    val ctx = context
    if (!NativeLocation.hasPermission(ctx)) {
        promise.resolve(null)
        return@AsyncFunction
    }
    NativeLocation.current(ctx, timeoutMs.toLong()) { location ->
        promise.resolve(location?.let { NativeLocation.toMap(it) })
    }
}
}
}
}

```

AndroidManifest.xml modules/adhan-native/android/src/main

- `<queries>` с петте пакета и `geo:` – от Android 11 без тях приложението не „вижда“ другите

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Алармата с пълния езан (етап 5) -->
  <uses-permission android:name="android.permission.WAKE_LOCK" />
  <uses-permission android:name="android.permission.VIBRATE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
  <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT" />
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

  <!-- Android 11+: за да види дали на телефона има Google Play Services (Huawei без Google) -->
  <queries>
    <package android:name="com.google.android.gms" />
    <!-- Етап 12: кои приложения за навигация са инсталирани (Настройки → Навигация) -->
    <package android:name="com.google.android.apps.maps" />
    <package android:name="com.waze" />
    <package android:name="com.here.app.maps" />
    <package android:name="com.huawei.maps.app" />
    <package android:name="app.organicmaps" />
    <!-- „Автоматично“: има ли изобщо приложение за карти (geo:) -->
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="geo" />
    </intent>
  </queries>

  <application>
    <!-- Екранът „Аларма“ върху заключения екран; отделна задача, за да не вадя цялото приложение -->
    <activity
      android:name="expo.modules.adhannative.AlarmActivity"
      android:exported="false"
      android:showWhenLocked="true"
      android:turnScreenOn="true"
      android:excludeFromRecents="true"
      android:launchMode="singleInstance"
      android:taskAffinity="${applicationId}.alarm"
      android:theme="@android:style/Theme.Material.NoActionBar" />

    <!-- Пуска езана и вибрацията, докато свърши (или „Спри“) -->
    <service
      android:name="expo.modules.adhannative.AlarmService"
      android:exported="false"
      android:foregroundServiceType="mediaPlayback" />

    <!-- Будилникът на Android (setAlarmClock) вика този receiver в часа на молитвата -->
    <receiver
      android:name="expo.modules.adhannative.AlarmReceiver"
      android:exported="false" />

    <!-- След рестарт, обновяване на приложението, смяна на часа или на разрешението – алармите наново -->
    <receiver
      android:name="expo.modules.adhannative.BootReceiver"
      android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
        <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
        <action android:name="android.intent.action.TIME_SET" />
        <action android:name="android.intent.action.TIMEZONE_CHANGED" />
      </intent-filter>
    </receiver>
  </application>
</manifest>

```

```

        <action android:name="android.app.action.SCHEDULE_EXACT_ALARM_PERMISSION_STATE_CHANGED" />
    </intent-filter>
</receiver>
<!-- Widget-ите (етап 8): „Следваща“ 2×2, „Денят“ 4×2, „Лента“ 4×1 -->
<receiver
    android:name="expo.modules.adhannative.NextPrayerWidget"
    android:label="@string/ezan_widget_next_name"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_next_info" />
</receiver>
<receiver
    android:name="expo.modules.adhannative.DayWidget"
    android:label="@string/ezan_widget_day_name"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_day_info" />
</receiver>
<receiver
    android:name="expo.modules.adhannative.BarWidget"
    android:label="@string/ezan_widget_bar_name"
    android:exported="false">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>
    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/ezan_widget_bar_info" />
</receiver>

<!-- Новият кадър на widget-ите в часа на всяка молитва и в полунощ -->
<receiver
    android:name="expo.modules.adhannative.WidgetRefreshReceiver"
    android:exported="false" />
</application>
</manifest>

```

Стъпка 6

Екраните и текстовете

Новите и променените екрани.

mosques.tsx src/app/(tabs)

- Разделът „Джамии“: карта, списък, джамията, „Отвори с...“. Без разрешение за GPS – търси около избраното място, с бутон „Разреши“.

src/app/(tabs)/mosques.tsx

751 реда

```

import * as Clipboard from 'expo-clipboard';
import * as Location from 'expo-location';

```

```

import { useFocusEffect } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import {
  ActivityIndicator,
  BackHandler,
  FlatList,
  Linking,
  Pressable,
  Share,
  StyleSheet,
  Text,
  useWindowDimensions,
  View,
} from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { ArrowUpIcon, ChevronIcon, CopyIcon, LocateIcon, MosqueIcon, NavigateIcon, PinIcon, ShareIcon }
from '@components/icons';
import { MosqueMap } from '@components/mosques/MosqueMap';
import { navAppName } from '@components/mosques/navLabel';
import { NavSheet } from '@components/mosques/NavSheet';
import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { formatHM } from '@domain/format';
import { FAR_RADIUS_M, formatDistance, NEAR_RADIUS_M, nearMosques, type NearMosque } from
'@domain/mosques';
import type { NavAppId } from '@domain/navApps';
import { PRAYERS } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n, type Lang, type Strings } from '@i18n';
import { currentPosition } from '@services/location';
import { cachedMosques, loadMosques, MosquesFetchError, type MosquesError, type MosquesResult } from
'@services/mosques';
import { openDirections, placeLink } from '@services/navigation';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

/** Листът със списъка застъпва картата с толкова (заоблените ъгли). */
const OVERLAP = 28;

type Origin = { lat: number; lon: number; gps: boolean };
type Phase = 'locating' | 'loading' | 'done' | 'error';

/**
 * „Джамии наблизо“ (етап 12): карта отгоре, списък по разстояние отдолу, следващата молитва.
 * Докосване на джамия → детайли с „Упътване“ в избраното приложение за навигация.
 * Позицията се взима наново при всяко отваряне на екрана (без фоново следене).
 */
export default function MosquesScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t, pick } = useI18n();
  const m = t.mosques;
  const { height: winH } = useWindowDimensions();
  const mapH = Math.round(Math.min(Math.max(winH * 0.44, 280), 440));

  const location = useSettings(selectLocation);
  const autoLocation = useSettings((s) => s.autoLocation);
  const navApp = useSettings((s) => s.navApp);

```

```

const navRemembered = useSettings((s) => s.navRemembered);
const setNavApp = useSettings((s) => s.setNavApp);
const forgetNavApp = useSettings((s) => s.forgetNavApp);

const [origin, setOrigin] = useState<Origin | null>(null);
const [noGps, setNoGps] = useState<'denied' | 'unavailable' | null>(null);
const [radius, setRadius] = useState<NEAR_RADIUS_M>;
const [result, setResult] = useState<MosquesResult | null>(null);
const [phase, setPhase] = useState<Phase>('locating');
const [error, setError] = useState<MosquesError | null>(null);
const [reload, setReload] = useState(0);
const [selectedId, setSelectedId] = useState<string | null>(null);
const [recenter, setRecenter] = useState(0);
const [sheet, setSheet] = useState<{ missing: NavAppId | null } | null>(null);
const [toast, setToast] = useState<{ id: number; text: string } | null>(null);
const toastId = useRef(0);
const say = (text: string) => setToast({ id: ++toastId.current, text });

const applyPosition = (p: Awaited<ReturnType<typeof currentPosition>>) => {
  if (p.ok) {
    setOrigin({ lat: p.latitude, lon: p.longitude, gps: true });
    setNoGps(null);
  } else {
    // без GPS – около избраното място за часовете
    const { latitude, longitude } = useSettings.getState().gpsLocation ??
useSettings.getState().manualLocation ?? location;
    setOrigin({ lat: latitude, lon: longitude, gps: false });
    setNoGps(p.reason);
  }
  setRecenter((x) => x + 1);
};

const locate = () => {
  setPhase('locating');
  currentPosition().then(applyPosition);
};

// позицията – при отваряне на раздела
useEffect(() => {
  currentPosition().then(applyPosition);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

useEffect(() => {
  if (!origin) return;
  let live = true;
  const force = reload > 0;
  (async () => {
    // последното търсене – веднага, докато дойде новото
    const cached = await cachedMosques(origin.lat, origin.lon, radius, lang);
    if (!live) return;
    if (cached) setResult(cached);
    setPhase('loading');
    try {
      const r = await loadMosques(origin.lat, origin.lon, radius, lang, force);
      if (!live) return;
      setResult(r);
      setError(null);
      setPhase('done');
    } catch (e) {
      if (!live) return;
      setError(e instanceof MosquesFetchError ? e.reason : 'offline');
    }
  })();
}, [origin, radius, lang, force]);

```

```

        setPhase('error');
    }
})();
return () => {
    live = false;
};
}, [origin, radius, lang, reload]);

const list = result && origin ? nearMosques(result.mosques, origin.lat, origin.lon, radius) : [];
const selected = list.find((x) => x.id === selectedId) ?? null;
const noneNear = radius === FAR_RADIUS_M && !list.some((x) => x.distanceM <= NEAR_RADIUS_M);

// Android „Назад“: от детайлите - към списъка
useFocusEffect(
    useCallback(() => {
        const sub = BackHandler.addEventListener('hardwareBackPress', () => {
            if (!selectedId) return false;
            setSelectedId(null);
            return true;
        });
        return () => sub.remove();
    }, [selectedId]),
);

// следващата молитва (без изгрева - не е молитва в джамията)
const now = useNow(15_000);
const schedule = usePrayerSchedule(now);
const next = schedule.next.id === 'sunrise' ? (schedule.today.find((p) => p.id === 'dhuhr') ??
schedule.next) : schedule.next;
const leftMin = Math.max(0, Math.floor((next.time.getTime() - now.getTime()) / 60_000));
const leftText = `${Math.floor(leftMin / 60)}:${String(leftMin % 60).padStart(2, '0')}`;

const name = (x: NearMosque) => x.name ?? m.unnamed;
const timeText = (x: NearMosque) => (x.mode === 'walk' ? m.walk(x.minutes) : m.drive(x.minutes));
const placeLabel = origin && !origin.gps ? pick(location.names) : autoLocation ? pick(location.names) :
m.myLocation;

const destination = (x: NearMosque) => ({ lat: x.lat, lon: x.lon, name: name(x) });
const from = origin?.gps ? { lat: origin.lat, lon: origin.lon, name: m.myLocation } : null;

const directions = async () => {
    if (!selected) return;
    if (!navRemembered) {
        setSheet({ missing: null });
        return;
    }
    const r = await openDirections(navApp, destination(selected), selected.mode, from);
    if (r === 'missing') {
        // избраното приложение е изтрито - питаме отново
        forgetNavApp();
        setSheet({ missing: navApp });
    }
};

const pickApp = async (id: NavAppId, remember: boolean) => {
    setSheet(null);
    if (!selected) return;
    if (remember) setNavApp(id);
    const r = await openDirections(id, destination(selected), selected.mode, from);
    if (r === 'missing') {
        if (remember) forgetNavApp();
    }
};

```

```

    setSheet({ missing: id });
  }
};

const share = () => {
  if (!selected) return;
  const message = [name(selected), selected.address, placeLink(selected.lat,
selected.lon)].filter(Boolean).join('\n');
  Share.share({ message }).catch(() => {});
};

const copy = async () => {
  if (!selected) return;
  const text = selected.address ?? `${selected.lat.toFixed(6)}, ${selected.lon.toFixed(6)}`;
  await Clipboard.setStringAsync(text).catch(() => {});
  say(m.copied);
};

const allowLocation = async () => {
  const perm = await Location.getForegroundPermissionsAsync();
  if (perm.status !== 'granted' && !perm.canAskAgain) Linking.openSettings().catch(() => {});
  else locate();
};

const retry = () => setReload((x) => x + 1);
const busy = phase === 'locating' || phase === 'loading';

/* ----- ЛИСТЪТ: СПИСКЪК ----- */

const header = (
  <View>
    <View style={styles.headRow}>
      <Text style={styles.title} accessibilityRole="header">
        {m.title}
      </Text>
      {busy && result ? (
        <ActivityIndicator size="small" color={colors.gold} />
      ) : result && list.length > 0 ? (
        <Text style={styles.count}>{m.count(list.length, radiusText(radius, lang))}</Text>
      ) : null}
    </View>

    <NextChip arabic={PRAYERS[next.id].arabic} label={m.nextAt(t.prayers[next.id], formatHM(next.time))}
left={m.nextIn(leftText)} t={t} />

    {noGps && (
      <Banner>
        <Text style={styles.bannerText}>{m.noGps(pick(location.names))}</Text>
        {noGps === 'denied' && <SmallButton label={m.allow} onPress={allowLocation} />}
      </Banner>
    )}
    {result?.offline && (
      <Banner>
        <Text style={styles.bannerText}>
          <Text style={styles.bannerBold}>{m.offlineTitle} </Text>
          {m.offlineText(whenText(result.at, now, m))}
        </Text>
      </Banner>
    )}

    {!result && busy && (

```

```

        <View style={styles.loading}>
            <ActivityIndicator color={colors.gold} />
            <Text style={styles.loadingText}>{phase === 'locating' ? m.locating : m.loading}</Text>
        </View>
    )}
    {!result && phase === 'error' && (
        <Card
            title={error === 'server' ? m.errorServer : m.errorOffline}
            text={error === 'server' ? m.errorServerText : m.errorOfflineText}
            action={{ label: m.retry, onPress: retry }}
        />
    )}

    {result && radius === NEAR_RADIUS_M && list.length === 0 && (
        <Card
            title={m.noneNear(radiusText(NEAR_RADIUS_M, lang))}
            text={m.noneHint}
            action={{ label: m.searchFar(radiusText(FAR_RADIUS_M, lang)), onPress: () =>
setRadius(FAR_RADIUS_M) }}
        />
    )}
    {result && radius === FAR_RADIUS_M && noneNear && (
        <Card
            title={list.length ? m.noneNear(radiusText(NEAR_RADIUS_M, lang)) :
m.noneFar(radiusText(FAR_RADIUS_M, lang))}
            text={m.noneHint}
        />
    )}
    {result && radius === FAR_RADIUS_M && noneNear && list.length > 0 && (
        <Text style={styles.section}>{m.within(radiusText(FAR_RADIUS_M, lang)).toUpperCase()}</Text>
    )}
</View>
);

const footer =
    result && radius === NEAR_RADIUS_M && list.length > 0 ? (
        <Pressable
            onPress={() => setRadius(FAR_RADIUS_M)}
            accessibilityRole="button"
            style={({ pressed }) => [styles.farBtn, pressed && styles.pressed]}
        >
            <Text style={styles.farText}>{m.searchFar(radiusText(FAR_RADIUS_M, lang))}</Text>
        </Pressable>
    ) : null;

return (
    <View style={styles.root}>
        <View style={[styles.map, { height: mapH + OVERLAP }]}>
            {origin && (
                <MosqueMap
                    center={origin}
                    me={origin.gps ? origin : null}
                    mosques={list}
                    selectedId={selectedId}
                    onSelect={setSelectedId}
                    recenter={recenter}
                    bottomInset={OVERLAP}
                    height={mapH + OVERLAP}
                    attribution={m.attribution}
                    label={m.allMap(list.length)}
                />
            )}
        </View>
    </View>
);

```

```

    })
    <View style={[styles.mapTop, { top: insets.top + 8 }]} pointerEvents="box-none">
      {selected ? (
        <Pressable
          onPress={() => setSelectedId(null)}
          accessibilityRole="button"
          accessibilityLabel={m.back}
          style={({ pressed }) => [styles.round, pressed && styles.pressed]}
        >
          <ChevronIcon direction="left" size={20} color={colors.text} />
        </Pressable>
      ) : (
        <View style={styles.chip}>
          <PinIcon color={colors.gold} />
          <Text style={styles.chipText} numberOfLines={1}>
            {placeLabel}
          </Text>
        </View>
      )}
    <Pressable
      onPress={() => {
        setSelectedId(null);
        locate();
      }}
      accessibilityRole="button"
      accessibilityLabel={m.locate}
      style={({ pressed }) => [styles.round, pressed && styles.pressed]}
    >
      <LocateIcon color={colors.text} />
    </Pressable>
  </View>
</View>

<View style={[styles.sheet, { top: mapH, paddingBottom: tabBarHeight }]}>
  <View style={styles.handle} />
  {selected ? (
    <Detail
      mosque={selected}
      name={name(selected)}
      dist={formatDistance(selected.distanceM, lang)}
      time={timeText(selected)}
      nextLine={m.nextAt(t.prayers[next.id], formatHM(next.time))}
      reach={selected.minutes < leftMin ? m.reach(selected.minutes) : null}
      appLabel={navRemembered ? m.directions(navAppName(navApp, t)) : m.directionsAuto}
      appHint={navRemembered ? (navApp === 'auto' ? m.withAuto : m.withApp(navAppName(navApp, t))) :
null}

      onDirections={directions}
      onShare={share}
      onCopy={copy}
      t={t}
    />
  ) : (
    <FlatList
      data={result ? list : []}
      keyExtractor={(x) => x.id}
      ListHeaderComponent={header}
      ListFooterComponent={footer}
      contentContainerStyle={styles.listContent}
      showsVerticalScrollIndicator={false}
      initialNumToRender={12}
      renderItem={({ item, index }) => (

```

```

        <MosqueRow
            name={name(item)}
            sub={[item.address ?? (item.name ? null : m.unnamedSub),
timeText(item)].filter(Boolean).join(' · ')}
            dist={formatDistance(item.distanceM, lang)}
            bearing={item.bearing}
            nearest={index === 0}
            allY={m.allYRow(name(item), formatDistance(item.distanceM, lang), timeText(item))}
            onPress={() => setSelectedId(item.id)}
        />
    )}
/>
})
</View>

{/* монтира се при всяко отваряне - „Запомни избора“ отново е отметнато */}
{selected && sheet && (
    <NavSheet
        visible
        title={name(selected)}
        distance={formatDistance(selected.distanceM, lang)}
        mode={selected.mode}
        missing={sheet.missing}
        onPick={pickApp}
        onClose={() => setSheet(null)}
    />
)}
<Toast message={toast} bottom={tabBarHeight + 16} />
</View>
);
}

/** „2 км“, „10 км“ - радиусът на търсенето (цяло число). */
function radiusText(r: number, lang: Lang): string {
    return `${r / 1000} ${lang === 'bg' ? 'км' : 'km'}`;
}

/** „днес, 14:20“, „вчера, 09:05“, „3.10, 18:40“. */
function whenText(at: number, now: Date, m: Strings['mosques']): string {
    const d = new Date(at);
    const day = (x: Date) => new Date(x.getFullYear(), x.getMonth(), x.getDate()).getTime();
    const diff = Math.round((day(now) - day(d)) / 86_400_000);
    const date = diff === 0 ? m.today : diff === 1 ? m.yesterday : `${d.getDate()}.${String(d.getMonth() + 1).padStart(2, '0')}`;
    return `${date}, ${formatHM(d)}`;
}

function NextChip({ arabic, label, left, t }: { arabic: string; label: string; left: string; t: Strings })
{
    return (
        <View style={styles.next} accessible accessibilityLabel={`${t.mosques.next} ${label}, ${left}`}>
            <Text style={styles.nextArabic}>{arabic}</Text>
            <Text style={styles.nextText} numberOfLines={1}>
                {t.mosques.next} <Text style={styles.nextBold}>{label}</Text>
            </Text>
            <Text style={styles.nextLeft}>{left}</Text>
        </View>
    );
}

function MosqueRow({

```

```

name,
sub,
dist,
bearing,
nearest,
ally,
onPress,
}: {
  name: string;
  sub: string;
  dist: string;
  bearing: number;
  nearest: boolean;
  ally: string;
  onPress: () => void;
}) {
  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      accessibilityLabel={ally}
      style={({ pressed }) => [styles.row, pressed && styles.pressed]}
    >
      <View style={[styles.badge, nearest && styles.badgeNear]}>
        <MosqueIcon size={18} strokeWidth={2} color={nearest ? colors.goldInk : colors.gold} />
      </View>
      <View style={styles.rowText}>
        <Text style={styles.rowName} numberOfLines={1}>
          {name}
        </Text>
        <Text style={styles.rowSub} numberOfLines={1}>
          {sub}
        </Text>
      </View>
      <View style={styles.rowEnd}>
        <View style={{ transform: [{ rotate: `${Math.round(bearing)}deg` }] }}>
          <ArrowUpIcon color={colors.gold} />
        </View>
        <Text style={styles.rowDist}>{dist}</Text>
      </View>
    </Pressable>
  );
}

function Detail({
  mosque,
  name,
  dist,
  time,
  nextLine,
  reach,
  appLabel,
  appHint,
  onDirections,
  onShare,
  onCopy,
  t,
}): {
  mosque: NearMosque;
  name: string;
  dist: string;

```

```

time: string;
nextLine: string;
reach: string | null;
appLabel: string;
appHint: string | null;
onDirections: () => void;
onShare: () => void;
onCopy: () => void;
t: Strings;
}) {
  const m = t.mosques;
  return (
    <View style={styles.detail}>
      <View style={styles.detailHead}>
        <View style={styles.detailNames}>
          <Text style={styles.detailName} accessibilityRole="header">
            {name}
          </Text>
          <Text>
            {mosque.arabic && (
              <Text style={styles.detailArabic} importantForAccessibility="no" accessibilityElementsHidden>
                {mosque.arabic}
              </Text>
            )}
          </Text>
        </View>
        <View style={styles.detailDist}>
          <Text style={styles.detailDistBig}>{dist}</Text>
          <Text style={styles.detailDistSub}>{time}</Text>
        </View>
      </View>

      <View style={styles.infoBox}>
        <View style={styles.infoRow}>
          <PinIcon color={colors.muted} />
          <Text style={styles.infoText, !mosque.address && styles.infoMuted}>{mosque.address ??
m.noAddress}</Text>
        </View>
        <View style={[styles.infoRow, styles.infoBorder]}>
          <Text style={styles.infoText}>
            {m.next} <Text style={styles.infoBold}>{nextLine}</Text>
            {reach ? ` - ${reach}` : ''}
          </Text>
        </View>
      </View>

      <Pressable onPress={onDirections} accessibilityRole="button" style={({ pressed }) => [styles.go,
pressed && styles.goPressed]}>
        <NavigateIcon color={colors.goldInk} />
        <Text style={styles.goText}>{appLabel}</Text>
      </Pressable>
      {appHint && <Text style={styles.goHint}>{appHint}</Text>}

      <View style={styles.actions}>
        <Pressable onPress={onShare} accessibilityRole="button" style={({ pressed }) => [styles.action,
pressed && styles.pressed]}>
          <ShareIcon color={colors.text} />
          <Text style={styles.actionText}>{m.share}</Text>
        </Pressable>
        <Pressable onPress={onCopy} accessibilityRole="button" style={({ pressed }) => [styles.action,
pressed && styles.pressed]}>
          <CopyIcon color={colors.text} />
          <Text style={styles.actionText}>{mosque.address ? m.copyAddress : m.copyCoords}</Text>
        </Pressable>
      </View>
    </View>
  );
}

```

```

        </Pressable>
      </View>
    </View>
  );
}

function Banner({ children }: { children: React.ReactNode }) {
  return <View style={styles.banner}>{children}</View>;
}

function SmallButton({ label, onPress }: { label: string; onPress: () => void }) {
  return (
    <Pressable onPress={onPress} accessibilityRole="button" style={({ pressed }) => [styles.smallBtn,
    pressed && styles.pressed]}>
      <Text style={styles.smallBtnText}>{label}</Text>
    </Pressable>
  );
}

function Card({ title, text, action }: { title: string; text: string; action?: { label: string; onPress: ()
=> void } }) {
  return (
    <View style={styles.card}>
      <Text style={styles.cardTitle}>{title}</Text>
      <Text style={styles.cardText}>{text}</Text>
      {action && (
        <Pressable onPress={action.onPress} accessibilityRole="button" style={({ pressed }) =>
[styles.cardBtn, pressed && styles.pressed]}>
          <Text style={styles.cardBtnText}>{action.label}</Text>
        </Pressable>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  pressed: { opacity: 0.7 },
  map: { position: 'absolute', top: 0, left: 0, right: 0, backgroundColor: '#13233A', overflow: 'hidden' },
  mapTop: {
    position: 'absolute',
    left: 16,
    right: 16,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    gap: 10,
  },
  chip: {
    flexShrink: 1,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 8,
    paddingLeft: 11,
    paddingRight: 14,
    borderRadius: 999,
    backgroundColor: 'rgba(6,10,20,0.62)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.08)',
  },
});

```

```

chipText: { flexShrink: 1, fontFamily: fonts.bold, fontSize: 13, color: colors.text },
round: {
  width: 40,
  height: 40,
  borderRadius: 20,
  alignItems: 'center',
  justifyContent: 'center',
  backgroundColor: 'rgba(6,10,20,0.62)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.08)',
},
sheet: {
  position: 'absolute',
  left: 0,
  right: 0,
  bottom: 0,
  backgroundColor: colors.base,
  borderTopLeftRadius: 22,
  borderTopRightRadius: 22,
  paddingTop: 8,
  shadowColor: '#000',
  shadowOpacity: 0.35,
  shadowRadius: 15,
  shadowOffset: { width: 0, height: -10 },
  elevation: 12,
},
handle: { width: 38, height: 4, borderRadius: 2, backgroundColor: 'rgba(255,255,255,0.18)', alignSelf:
'center', marginBottom: 12 },
listContent: { paddingHorizontal: 16, paddingBottom: 16 },
headRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center',
paddingHorizontal: 2, paddingBottom: 10 },
title: { fontFamily: fonts.extrabold, fontSize: 20, color: colors.text },
count: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.muted, ...tabularNums },
next: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 8,
  paddingHorizontal: 12,
  paddingVertical: 8,
  borderRadius: 12,
  backgroundColor: 'rgba(212,168,87,0.12)',
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.35)',
  marginBottom: 8,
},
nextArabic: { fontFamily: fonts.arabic, fontSize: 16, lineHeight: 22, color: colors.gold },
nextText: { flexShrink: 1, fontFamily: fonts.semibold, fontSize: 12.5, color: colors.text, ...tabularNums
},
nextBold: { fontFamily: fonts.extrabold },
nextLeft: { marginLeft: 'auto', fontFamily: fonts.extrabold, fontSize: 12.5, color: colors.gold,
...tabularNums },
banner: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 10,
  padding: 12,
  borderRadius: 12,
  backgroundColor: 'rgba(227,154,75,0.12)',
  borderWidth: 1,
  borderColor: 'rgba(227,154,75,0.4)',
  marginBottom: 8,
}

```

```

},
bannerText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.text },
bannerBold: { fontFamily: fonts.extrabold, color: colors.warn },
smallBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999, backgroundColor: colors.gold },
smallBtnText: { fontFamily: fonts.bold, fontSize: 12.5, color: colors.goldInk },
loading: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 18, paddingHorizontal: 4
},
loadingText: { fontFamily: fonts.semibold, fontSize: 13.5, color: colors.muted },
card: {
  marginTop: 4,
  marginBottom: 8,
  padding: 16,
  borderRadius: 16,
  backgroundColor: colors.card,
  borderWidth: 1,
  borderColor: colors.cardBorder,
  gap: 4,
},
cardTitle: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.text },
cardText: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 18, color: colors.muted },
cardBtn: { alignSelf: 'flex-start', marginTop: 8, paddingHorizontal: 14, paddingVertical: 8,
borderRadius: 999, backgroundColor: colors.gold },
cardBtnText: { fontFamily: fonts.bold, fontSize: 13, color: colors.goldInk },
section: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted, marginTop: 6,
paddingHorizontal: 4 },
row: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 12,
  minHeight: 60,
  paddingVertical: 8,
  borderBottomWidth: StyleSheet.hairlineWidth,
  borderBottomColor: 'rgba(255,255,255,0.08)',
},
badge: {
  width: 38,
  height: 38,
  borderRadius: 12,
  alignItems: 'center',
  justifyContent: 'center',
  backgroundColor: 'rgba(212,168,87,0.14)',
},
badgeNear: { backgroundColor: colors.gold },
rowText: { flex: 1, minWidth: 0 },
rowName: { fontFamily: fonts.bold, fontSize: 15, color: colors.text },
rowSub: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted, marginTop: 1, ...tabularNums },
rowEnd: { alignItems: 'flex-end', gap: 2 },
rowDist: { fontFamily: fonts.extrabold, fontSize: 13, color: colors.text, ...tabularNums },
farBtn: {
  marginTop: 12,
  alignSelf: 'center',
  paddingHorizontal: 18,
  paddingVertical: 10,
  borderRadius: 999,
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.6)',
},
farText: { fontFamily: fonts.bold, fontSize: 13.5, color: colors.gold },
detail: { paddingHorizontal: 20, paddingTop: 4 },
detailHead: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
detailNames: { flex: 1 },

```

```

detailName: { fontFamily: fonts.extrabold, fontSize: 24, lineHeight: 30, color: colors.text },
detailArabic: { fontFamily: fonts.arabic, fontSize: 20, lineHeight: 30, color: colors.gold, textAlign:
'left' },
detailDist: { alignItems: 'flex-end' },
detailDistBig: { fontFamily: fonts.extrabold, fontSize: 22, color: colors.text, ...tabularNums },
detailDistSub: { fontFamily: fonts.semibold, fontSize: 12, color: colors.muted, ...tabularNums },
infoBox: {
  marginTop: 14,
  borderRadius: 14,
  backgroundColor: colors.card,
  borderWidth: 1,
  borderColor: colors.cardBorder,
},
infoRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingHorizontal: 12, paddingVertical:
11 },
infoBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
infoText: { flex: 1, fontFamily: fonts.medium, fontSize: 13, lineHeight: 18, color: colors.text,
...tabularNums },
infoMuted: { color: colors.muted },
infoBold: { fontFamily: fonts.extrabold },
go: {
  marginTop: 16,
  height: 52,
  borderRadius: 16,
  backgroundColor: colors.gold,
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'center',
  gap: 10,
},
goPressed: { opacity: 0.85 },
goText: { fontFamily: fonts.extrabold, fontSize: 16, color: colors.goldInk },
goHint: { fontFamily: fonts.medium, fontSize: 12, color: colors.muted, textAlign: 'center', marginTop: 8
},
actions: { flexDirection: 'row', gap: 10, marginTop: 14 },
action: {
  flex: 1,
  height: 46,
  borderRadius: 14,
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'center',
  gap: 8,
  backgroundColor: 'rgba(255,255,255,0.06)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.08)',
},
actionText: { fontFamily: fonts.bold, fontSize: 13.5, color: colors.text },
});

```

navigation.tsx src/app

- Настройки → Навигация.

src/app/navigation.tsx

144 реда

```

import { router } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { AppState, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

```

```

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { NavAppTile } from '@components/mosques/NavAppTile';
import { navAppDesc, navAppName } from '@components/mosques/navLabel';
import type { NavAppId } from '@domain/navApps';
import { useI18n } from '@i18n';
import { availableNavApps, installedMap, openStore } from '@services/navigation';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

/**
 * Настройки → Навигация (етап 12): с кое приложение се отваря „Упътване“ до джамия.
 * Изборът се пази завинаги (и след рестарт на телефона). Липсващите приложения – с „Изтегли“.
 */
export default function NavigationScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const n = t.nav;
  const navApp = useSettings((s) => s.navApp);
  const remembered = useSettings((s) => s.navRemembered);
  const setNavApp = useSettings((s) => s.setNavApp);
  const [installed, setInstalled] = useState<Partial<Record<NavAppId, boolean>>>({});
  const apps = availableNavApps();

  // при връщане от магазина списъкът се обновява
  useEffect(() => {
    const check = () => installedMap().then(setInstalled);
    check();
    const sub = AppState.addEventListener('change', (st) => st === 'active' && check());
    return () => sub.remove();
  }, []);

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  return (
    <View style={styles.root}>
      <GeometricPattern />
      <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
        <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
          <ChevronIcon direction="left" size={18} color={colors.gold} />
          <Text style={styles.backText}>{n.back}</Text>
        </Pressable>
      </View>
      <ScrollView
        contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 40 }]}
        showsVerticalScrollIndicator={false}
      >
        <Text style={styles.title}>{n.title}</Text>
        <Text style={styles.intro}>{n.intro}</Text>

        <View style={styles.card} accessibilityRole="radiogroup" accessibilityLabel={n.title}>
          {apps.map((app, i) => {
            const has = installed[app.id] ?? true;

```

```

const on = remembered && navApp === app.id;
const name = navAppName(app.id, t);
return (
  <Pressable
    key={app.id}
    onPress={() => (has ? setNavApp(app.id) : openStore(app.id))}
    accessibilityRole="radio"
    accessibilityState={{ checked: on, disabled: !has }}
    accessibilityLabel={` ${name}. ${navAppDesc(app, has, t)}`}
    accessibilityHint={has ? undefined : n.download}
    style={({ pressed }) => [styles.row, i > 0 && styles.border, on && styles.rowOn, pressed &&
styles.pressed]}
  >
    <NavAppTile id={app.id} dim={!has} />
    <View style={styles.text}>
      <Text style={[styles.name, on && styles.nameOn, !has && styles.nameDim]}>{name}</Text>
      <Text style={styles.desc}>{navAppDesc(app, has, t)}</Text>
    </View>
    {!has && (
      <View style={styles.get}>
        <Text style={styles.getText}>{n.download}</Text>
      </View>
    )}
    <View style={[styles.radio, on && styles.radioOn]}>{on && <View style={styles.dot} />}
  </View>
</Pressable>
);
}}
</View>

  <Text style={styles.note}>{n.note}</Text>
</ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: { paddingHorizontal: 16, paddingBottom: 4 },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6, alignSelf: 'flex-
start' },
  backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 8 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginTop: 4 },
  intro: { fontFamily: fonts.regular, fontSize: 13.5, lineHeight: 19, color: colors.muted, marginBottom: 8
},
  card: {
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
    borderRadius: 18,
    overflow: 'hidden',
  },
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 10,
minHeight: 64 },
  rowOn: { backgroundColor: 'rgba(212,168,87,0.10)' },
  border: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { opacity: 0.7 },
  text: { flex: 1, gap: 2 },
  name: { fontFamily: fonts.bold, fontSize: 15.5, color: colors.text },
  nameOn: { fontFamily: fonts.extrabold, color: colors.gold },

```

```

nameDim: { color: 'rgba(242,239,232,0.6)' },
desc: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.muted },
get: {
  paddingHorizontal: 11,
  paddingVertical: 5,
  borderRadius: 999,
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.6)',
},
getText: { fontFamily: fonts.bold, fontSize: 12, color: colors.gold },
radio: {
  width: 22,
  height: 22,
  borderRadius: 11,
  borderWidth: 2,
  borderColor: 'rgba(242,239,232,0.35)',
  alignItems: 'center',
  justifyContent: 'center',
},
radioOn: { borderColor: colors.gold },
dot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.gold },
note: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 18, color: colors.muted, marginTop: 6,
paddingHorizontal: 4 },
});

```

NavSheet.tsx src/components/mosques

- „Отвори с...” и „Запомни избора”.

src/components/mosques/NavSheet.tsx

152 реда

```

import { useEffect, useState } from 'react';
import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import Svg, { Path } from 'react-native-svg';

import { modeFor, type NavAppId, type TravelMode } from '@/domain/navApps';
import { useI18n } from '@/i18n';
import { availableNavApps, installedMap } from '@/services/navigation';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

import { NavAppTile } from './NavAppTile';
import { navAppName } from './navLabel';

interface Props {
  visible: boolean;
  /** Името на джамията и разстоянието („350 м"). */
  title: string;
  distance: string;
  mode: TravelMode;
  /** Приложението, което вече го няма на телефона (изтрито след избора). */
  missing?: NavAppId | null;
  onPick: (id: NavAppId, remember: boolean) => void;
  onClose: () => void;
}

/**
 * „Отвори с...” (етап 12) – при първото „Упътване” или ако избраното приложение е изтрито.
 * Показват се само инсталираните приложения и „Автоматично”. „Запомни избора” е отменено.

```

```

*/
export function NavSheet({ visible, title, distance, mode, missing, onPick, onClose }: Props) {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const n = t.nav;
  const [installed, setInstalled] = useState<Partial<Record<NavAppId, boolean>> | null>(null);
  const [remember, setRemember] = useState(true);

  useEffect(() => {
    if (visible) installedMap().then(setInstalled);
  }, [visible]);

  // „Автоматично“ – последно: първо конкретните приложения
  const apps = availableNavApps().filter((a) => a.id !== 'auto' && installed?.[a.id]);
  const list = [...apps.map((a) => a.id), 'auto' as const];

  return (
    <Modal visible={visible} transparent animationType="slide" onRequestClose={onClose}
statusBarTranslucent>
      <Pressable style={styles.backdrop} onPress={onClose} accessibilityRole="button" accessibilityLabel=
{n.close} />
      <View style={[styles.sheet, { paddingBottom: insets.bottom + 24 }]} accessibilityViewIsModal>
        <View style={styles.handle} />
        <Text style={styles.title} accessibilityRole="header">
          {n.sheetTitle(title)}
        </Text>
        <Text style={styles.sub}>{n.sheetSub(distance)}</Text>
        {missing && <Text style={styles.missing}>{n.missing(navAppName(missing, t))}</Text>}

        <View style={styles.grid}>
          {installed &&
            list.map((id) => {
              const m = modeFor(id, mode);
              const name = navAppName(id, t);
              const modeText = m === 'walk' ? n.walk : n.drive;
              return (
                <Pressable
                  key={id}
                  onPress={() => onPick(id, remember)}
                  accessibilityRole="button"
                  accessibilityLabel={` ${name}, ${modeText} `}
                  style={({ pressed }) => [styles.app, pressed && styles.appPressed]}
                >
                  <NavAppTile id={id} size={48} />
                  <Text style={styles.appName} numberOfLines={1}>
                    {name}
                  </Text>
                  <Text style={styles.appMode}>{modeText}</Text>
                </Pressable>
              );
            })}
        </View>

        <Pressable
          onPress={() => setRemember((v) => !v)}
          accessibilityRole="checkbox"
          accessibilityState={{ checked: remember }}
          style={styles.remember}
        >
          <View style={[styles.box, remember && styles.boxOn]}>
            {remember && (

```

```

        <Svg width={14} height={14} viewBox="0 0 24 24">
          <Path d="M5 12.5L4.5 4.5L19 7.5" fill="none" stroke={colors.goldInk} strokeWidth={3}
strokeLinecap="round" strokeLinejoin="round" />
        </Svg>
      )}
    </View>
    <View style={styles.rememberText}>
      <Text style={styles.rememberLabel}>{n.remember}</Text>
      <Text style={styles.rememberHint}>{n.rememberHint}</Text>
    </View>
  </Pressable>
</View>
</Modal>
);
}

const styles = StyleSheet.create({
  backdrop: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, backgroundColor:
'rgba(3,6,12,0.55)' },
  sheet: {
    position: 'absolute',
    left: 0,
    right: 0,
    bottom: 0,
    backgroundColor: '#16243A',
    borderTopLeftRadius: 24,
    borderTopRightRadius: 24,
    paddingTop: 10,
    paddingHorizontal: 20,
  },
  handle: { width: 38, height: 4, borderRadius: 2, backgroundColor: 'rgba(255,255,255,0.2)', alignSelf:
'center', marginBottom: 16 },
  title: { fontFamily: fonts.extrabold, fontSize: 20, color: colors.text },
  sub: { fontFamily: fonts.medium, fontSize: 13, color: colors.muted, marginTop: 4 },
  missing: { fontFamily: fonts.semibold, fontSize: 13, lineHeight: 18, color: colors.warn, marginTop: 10 },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 12, marginTop: 18 },
  app: {
    width: '31%',
    flexGrow: 1,
    maxWidth: '32%',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 14,
    paddingHorizontal: 6,
    borderRadius: 16,
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.08)',
  },
  appPressed: { backgroundColor: 'rgba(212,168,87,0.14)', borderColor: 'rgba(212,168,87,0.6)' },
  appName: { fontFamily: fonts.bold, fontSize: 13, color: colors.text },
  appMode: { fontFamily: fonts.medium, fontSize: 11, color: colors.muted },
  remember: { flexDirection: 'row', alignItems: 'center', gap: 12, marginTop: 20 },
  box: {
    width: 22,
    height: 22,
    borderRadius: 6,
    borderWidth: 2,
    borderColor: 'rgba(242,239,232,0.45)',
    alignItems: 'center',
    justifyContent: 'center',
  },

```

```

},
boxOn: { backgroundColor: colors.gold, borderColor: colors.gold },
rememberText: { flex: 1 },
rememberLabel: { fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
rememberHint: { fontFamily: fonts.medium, fontSize: 12, color: colors.muted },
});

```

NavAppTile.tsx src/components/mosques

- Иконата на приложението или плочката с буквата.

src/components/mosques/NavAppTile.tsx

41 реда

```

import { Image, StyleSheet, Text, View } from 'react-native';

import { navApp, type NavAppId } from '@/domain/navApps';
import { navAppIcon } from '@/services/navigation';
import { fonts } from '@/theme/typography';

import { NavigateIcon } from '../icons';

/**
 * Плочката на приложението за навигация: на Android – истинската икона (ако е инсталирано),
 * иначе цветна плочка с буквата. „Автоматично“ – стрелката за навигация.
 */
export function NavAppTile({ id, size = 40, dim }: { id: NavAppId; size?: number; dim?: boolean }) {
  const app = navApp(id);
  const icon = dim ? null : navAppIcon(id);
  const radius = Math.round(size * 0.3);
  if (icon) {
    return <Image source={{ uri: icon }} style={{ width: size, height: size, borderRadius: radius }}
      accessibilityIgnoresInvertColors />;
  }
  return (
    <View
      style={[
        styles.tile,
        { width: size, height: size, borderRadius: radius, backgroundColor: app.color },
        dim && styles.dim,
      ]}
    >
      {id === 'auto' ? (
        <NavigateIcon size={Math.round(size * 0.48)} color="#F2EFE8" />
      ) : (
        <Text style={[styles.letter, { fontSize: Math.round(size * 0.42) }]}>{app.letter}</Text>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  tile: { alignItems: 'center', justifyContent: 'center' },
  dim: { opacity: 0.55 },
  letter: { fontFamily: fonts.extrabold, color: '#F2EFE8' },
});

```

`_layout.tsx` src/app/(tabs)

- Петият раздел.

src/app/(tabs)/_layout.tsx

24 реда

```
import { Tabs } from 'expo-router/js-tabs';

import { TabBar } from '@components/TabBar';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';

export default function TabsLayout() {
  const { t } = useI18n();
  return (
    <Tabs
      tabBar={({props}) => <TabBar {...props} />}
      screenOptions={{
        headerShown: false,
        sceneStyle: { backgroundColor: colors.base },
      }}
    >
      <Tabs.Screen name="index" options={{ title: t.tabs.today }} />
      <Tabs.Screen name="qibla" options={{ title: t.tabs.qibla }} />
      <Tabs.Screen name="mosques" options={{ title: t.tabs.mosques }} />
      <Tabs.Screen name="month" options={{ title: t.tabs.month }} />
      <Tabs.Screen name="settings" options={{ title: t.tabs.settings }} />
    </Tabs>
  );
}
```

`TabBar.tsx` src/components

- Иконата „Джамии“; надписите – на един ред (за „Настройки“ и „Ayarlar“).

src/components/TabBar.tsx

82 реда

```
import type { BottomTabBarProps } from 'expo-router/js-tabs';
import type { ComponentType } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { colors } from '@theme/colors';
import { FONT_SCALE, fonts } from '@theme/typography';

import { MonthIcon, MosqueIcon, QiblaIcon, SettingsIcon, TodayIcon } from './icons';

const ICONS: Record<string, ComponentType<{ color: string }>> = {
  index: TodayIcon,
  qibla: QiblaIcon,
  mosques: MosqueIcon,
  month: MonthIcon,
  settings: SettingsIcon,
};

const BAR_HEIGHT = 58;

/** Височината на лентата + долния безопасен отстъп. Екраните я ползват за padding отдолу. */
export function useTabBarHeight() {
  const insets = useSafeAreaInsets();
```

```

    return BAR_HEIGHT + Math.max(insets.bottom, 12);
  }

  /**
   * Собственная долна навигация. Позиционирана е `absolute`, за да минава
   * градиентът на екрана под нея, както в тоскур-а.
   */
  export function TabBar({ state, descriptors, navigation }: BottomTabBarProps) {
    const insets = useSafeAreaInsets();

    return (
      <View style={[styles.bar, { paddingBottom: Math.max(insets.bottom, 12) }]}>
        {state.routes.map((route, index) => {
          const focused = state.index === index;
          const { options } = descriptors[route.key];
          const label = options.title ?? route.name;
          const Icon = ICONS[route.name] ?? TodayIcon;
          const color = focused ? colors.gold : colors.muted;

          const onPress = () => {
            const event = navigation.emit({ type: 'tabPress', target: route.key, canPreventDefault: true });
            if (!focused && !event.defaultPrevented) navigation.navigate(route.name, route.params);
          };

          return (
            <Pressable
              key={route.key}
              onPress={onPress}
              style={styles.item}
              accessibilityRole="tab"
              accessibilityState={{ selected: focused }}
              accessibilityLabel={label}
            >
              <Icon color={color} />
              <Text maxFontSizeMultiplier={FONT_SCALE.dense} numberOfLines={1} style={styles.label, { color
            ]]}>{label}</Text>
            </Pressable>
          );
        })}
      </View>
    );
  }

  const styles = StyleSheet.create({
    bar: {
      position: 'absolute',
      left: 0,
      right: 0,
      bottom: 0,
      flexDirection: 'row',
      paddingTop: 8,
      paddingHorizontal: 8,
      backgroundColor: colors.tabBar,
      borderTopWidth: StyleSheet.hairlineWidth,
      borderTopColor: 'rgba(255,255,255,0.10)',
    },
    item: { flex: 1, alignItems: 'center', gap: 4, paddingVertical: 4 },
    // 5 раздела: „Настройки“ / „Ayarlar“ трябва да се побират и на тесен телефон
    label: { fontFamily: fonts.semibold, fontSize: 10.5 },
  });

```

settings.tsx src/app/(tabs)

- Групата „Джамии наблизно“ → „Навигация“.

src/app/(tabs)/settings.tsx

276 реда

```
import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useState } from 'react';
import { ActivityIndicator, Linking, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { BatteryWarning } from '@components/settings/BatteryWarning';
import { NotificationsGroup } from '@components/settings/NotificationsGroup';
import { SoundsGroup } from '@components/settings/SoundsGroup';
import { PinIcon } from '@components/icons';
import { navAppName } from '@components/mosques/navLabel';
import {
  Choice,
  Group,
  NavRow,
  Row,
  StepperRow,
  SwitchRow,
  settingsStyles,
} from '@components/settings/controls';
import { useTabBarHeight } from '@components/TabBar';
import { CALC_METHODS, HIGH_LAT_RULES, isHighLatitude, isPolar, methodForCountry, type MethodChoice } from '@domain/calc';
import { PRAYER_IDS } from '@domain/prayers';
import { countryOf } from '@domain/resolve';
import { countryName } from '@domain/worldCities';
import { useI18n } from '@/i18n';
import { refreshLocation, useLocationStatus } from '@services/location';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

export default function SettingsScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t, pick } = useI18n();
  const s = useSettings();
  const location = selectLocation(s);
  const status = useLocationStatus((x) => x.status);
  const [open, setOpen] = useState<'method' | 'asr' | 'highLat' | null>(null);

  const isCalc = location.source === 'calc';
  // „Автоматично“: държавата на мястото и методът за нея
  const country = isCalc ? countryOf(location) : null;
  const autoMethod = methodForCountry(country);
  const highLat = isCalc && isHighLatitude(location.latitude);
  const anyOffset = PRAYER_IDS.some((id) => s.offsets[id] !== 0);
  const toggle = (what: 'method' | 'asr' | 'highLat') => setOpen((cur) => (cur === what ? null : what));

  return (
    <View style={styles.root}>
      <GeometricPattern />
    </View>
  );
}
```

```

{/* списъкът свършва над долната навигация, за да не се застъпва с нея */}
<ScrollView
  style={{ marginBottom: tabBarHeight }}
  contentContainerStyle={[styles.content, { paddingTop: insets.top + 8, paddingBottom: 24 }]}
  showsVerticalScrollIndicator={false}
>
  <Text style={styles.title}>{t.settings.title}</Text>
  <BatteryWarning />

  {/* ---- Място ---- */}
  <Group label={t.settings.groupPlace}>
    <Pressable
      onPress={() => router.push('/place')}
      accessibilityRole="button"
      accessibilityLabel={` ${pick(location.names)}. ${t.settings.change}`}
      style={({ pressed }) => [styles.placeRow, pressed && styles.pressed]}
    >
      <PinIcon size={18} color={colors.gold} />
      <View style={styles.placeText}>
        <Text style={styles.placeName}>{pick(location.names)}</Text>
        {location.detail ? <Text style={styles.placeDetail}>{pick(location.detail)}</Text> : null}
        <Text style={styles.badge}>{isCalc ? t.settings.sourceCalc : t.settings.sourceMufti}</Text>
      </View>
      <Text style={styles.link}>{t.settings.change}</Text>
    </Pressable>
    <SwitchRow label={t.settings.auto} value={s.autoLocation} onChange={s.setAutoLocation} />
    {s.autoLocation && (
      <Row>
        <View style={styles.statusBox}>
          <View style={styles.statusLine}>
            {status === 'locating' && <ActivityIndicator size="small" color={colors.gold} />}
            <Text style={styles.statusText, status === 'locating' && styles.statusBusy}>
              {status === 'locating'
                ? t.settings.locating
                : status === 'denied'
                  ? t.settings.denied
                  : status === 'unavailable'
                    ? t.settings.unavailable
                    : t.settings.autoHint}
            </Text>
          </View>
          {status === 'denied' ? (
            <Pressable onPress={() => Linking.openSettings()} accessibilityRole="button">
              <Text style={styles.link}>{t.settings.openSettings}</Text>
            </Pressable>
          ) : status !== 'locating' ? (
            <Pressable onPress={() => refreshLocation()} accessibilityRole="button">
              <Text style={styles.link}>{t.settings.refresh}</Text>
            </Pressable>
          ) : null}
        </View>
      </Row>
    )}
  </Group>

  {/* ---- Известия ---- */}
  <NotificationsGroup />

  {/* ---- Звуци (етап 6) ---- */}
  <SoundsGroup />

```

```

{ /* ---- Чacobe ---- */ }
{ isCalc ? (
  <Group
    label={t.settings.groupTimes}
    note={
      isPolar(location.latitude)
        ? t.settings.polarNote(Math.abs(location.latitude).toFixed(0))
        : highLat
        ? t.settings.highLatNote(Math.abs(location.latitude).toFixed(0))
        : t.settings.asrNote
    }
  >
    <NavRow
      first
      label={t.settings.method}
      value={s.method === 'auto' ? t.settings.methodAutoValue(t.methods[autoMethod]) :
t.methods[s.method]}
      onPress={() => toggle('method')}
    />
    {open === 'method' && (
      <Choice<MethodChoice>
        options={[
          {
            value: 'auto',
            label: t.settings.methodAuto,
            desc: t.settings.methodAutoDesc(country ? countryName(country, lang === 'bg' ? 'bg' :
'en') : '-', t.methods[autoMethod]),
          },
          ...CALC_METHODS.map((m) => ({ value: m, label: t.methods[m] })),
        ]}
        value={s.method}
        onChange={(m) => {
          s.setMethod(m);
          setOpen(null);
        }}
      />
    )}
    <NavRow
      label={t.settings.asr}
      value={s.madhab === 'hanafi' ? t.settings.hanafi : t.settings.shafi}
      onPress={() => toggle('asr')}
    />
    {open === 'asr' && (
      <Choice
        options={[
          { value: 'shafi' as const, label: t.settings.shafi, desc: t.settings.shafiDesc },
          { value: 'hanafi' as const, label: t.settings.hanafi, desc: t.settings.hanafiDesc },
        ]}
        value={s.madhab}
        onChange={(m) => {
          s.setMadhab(m);
          setOpen(null);
        }}
      />
    )}
    {highLat && (
      <>
        <NavRow
          label={t.settings.highLat}
          value={t.highLatRules[s.highLatRule].name}
          onPress={() => toggle('highLat')}
        </NavRow>
      </>
    )}
  )}
}

```

```

        />
        {open === 'highLat' && (
            <Choice
                options={HIGH_LAT_RULES.map((r) => ({
                    value: r,
                    label: t.highLatRules[r].name,
                    desc: t.highLatRules[r].desc,
                }))}
                value={s.highLatRule}
                onChange={(r) => {
                    s.setHighLatRule(r);
                    setOpen(null);
                }}
            />
        )}
    </>
)}
</Group>
) : (
    <Group label={t.settings.groupTimes} note={t.settings.sourceMuftiNote}>
        <Row first>
            <Text style={settingsStyles.label}>{t.settings.sourceMufti}</Text>
        </Row>
    </Group>
)}

/* ---- Корекции ---- */
<Group label={t.settings.groupCorrections} note={t.settings.offsetsNote}>
    {PRAYER_IDS.map((id, i) => (
        <StepperRow
            key={id}
            first={i === 0}
            label={t.prayers[id]}
            valueText={t.settings.minutes(s.offsets[id])}
            highlighted={s.offsets[id] !== 0}
            onMinus={() => s.changeOffset(id, -1)}
            onPlus={() => s.changeOffset(id, 1)}
            minusLabel={t.settings.decrease(t.prayers[id])}
            plusLabel={t.settings.increase(t.prayers[id])}
        />
    ))}
    {anyOffset && (
        <Pressable onPress={s.resetOffsets} accessibilityRole="button" style={styles.resetRow}>
            <Text style={styles.link}>{t.settings.reset}</Text>
        </Pressable>
    )}
</Group>

<Group label={t.settings.hijri} note={isCalc ? t.settings.hijriNoteAbroad : t.settings.hijriNote}>
    <StepperRow
        first
        label={t.settings.hijri}
        valueText={t.settings.days(s.hijriAdjust)}
        highlighted={s.hijriAdjust !== 0}
        onMinus={() => s.changeHijriAdjust(-1)}
        onPlus={() => s.changeHijriAdjust(1)}
        minusLabel={t.settings.decrease(t.settings.hijri)}
        plusLabel={t.settings.increase(t.settings.hijri)}
    />
</Group>

```

```

    { /* ---- Джамии наблизно (етап 12): с кое приложение се отваря „Упътване“ ---- */
    <Group label={t.nav.group} note={t.nav.rowNote}>
      <NavRow
        first
        label={t.nav.row}
        value={s.navRemembered ? navAppName(s.navApp, t) : t.nav.ask}
        onPress={() => router.push('/navigation')}
      />
    </Group>

    { /* ---- Приложение: източниците и поверителността са в „За приложението“ ---- */
    <Group label={t.settings.groupApp}>
      <NavRow first label={t.settings.about} value={APP_VERSION} onPress={() => router.push('/about')}
    />
    </Group>
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { paddingHorizontal: 16, gap: 20 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginLeft: 4 },
  placeRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12 },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  placeText: { flex: 1, gap: 2 },
  placeName: { fontFamily: fonts.extrabold, fontSize: 17, color: colors.text },
  placeDetail: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.muted },
  badge: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.gold, marginTop: 2 },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
  statusBox: { flex: 1, gap: 6 },
  statusLine: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  statusText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  statusBusy: { color: colors.text },
  resetRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
});

```

about.tsx src/app

- Нова група „Картата и джамиите“ (OpenStreetMap · Overpass, OpenFreeMap) и обновен текст за поверителност: за джамиите до OpenStreetMap отива само район от ~1 км.

src/app/about.tsx

111 реда

```

import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useRef } from 'react';
import { Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { Group, Row } from '@components/settings/controls';

```

```

import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

/** „За приложението“: версия, източници на данните и поверителност. Отваря се от „Настройки“. */
export default function AboutScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const a = t.about;

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  const source = (key: keyof typeof a.sources, first?: boolean) => (
    <Row first={first}>
      <View style={styles.sourceText}>
        <Text style={styles.sourceName}>{a.sources[key].name}</Text>
        <Text style={styles.sourceDesc}>{a.sources[key].desc}</Text>
      </View>
    </Row>
  );

  return (
    <View style={styles.root}>
      <GeometricPattern />
      <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
        <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
          <ChevronIcon direction="left" size={18} color={colors.gold} />
          <Text style={styles.backText}>{a.back}</Text>
        </Pressable>
      </View>
      <ScrollView
        contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}
        showsVerticalScrollIndicator={false}
      >
        <View style={styles.hero}>
          <View style={styles.logo}>
            <Text style={styles.logoText}>أذان</Text>
          </View>
          <Text style={styles.appName}>{a.name}</Text>
          <Text style={styles.tagline}>{a.tagline}</Text>
          <Text style={styles.version}>{a.version(APP_VERSION)}</Text>
        </View>

        <Group label={a.groupTimes}>
          {source('mufti', true)}
          {source('adhan')}
        </Group>
        <Group label={a.groupPlaces}>
          {source('osm', true)}
          {source('ne')}
          {source('geonames')}
          {source('praytimes')}
        </Group>
      </ScrollView>
    </View>
  );
}

```

```

        <Group label={a.groupMap}>
            {source('overpass', true)}
            {source('openfreemap')}
        </Group>
        <Group label={a.groupPrivacy}>
            <Row first>
                <Text style={styles.privacy}>{a.privacy}</Text>
            </Row>
        </Group>
    </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
    root: { flex: 1, backgroundColor: colors.base },
    header: { paddingHorizontal: 16, paddingBottom: 4 },
    backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, alignSelf: 'flex-start', paddingVertical:
6 },
    backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
    content: { paddingHorizontal: 16, gap: 20 },
    hero: { alignItems: 'center', paddingVertical: 12, gap: 4 },
    logo: {
        width: 84,
        height: 84,
        borderRadius: 24,
        alignItems: 'center',
        justifyContent: 'center',
        backgroundColor: 'rgba(212,168,87,0.12)',
        borderWidth: 1,
        borderColor: 'rgba(212,168,87,0.45)',
        marginBottom: 10,
    },
    logoText: { fontFamily: fonts.arabicBold, fontSize: 30, color: colors.gold },
    appName: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
    tagline: { fontFamily: fonts.medium, fontSize: 13.5, color: colors.textDim },
    version: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.muted, marginTop: 2 },
    sourceText: { flex: 1, gap: 2 },
    sourceName: { fontFamily: fonts.bold, fontSize: 14.5, color: colors.text },
    sourceDesc: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
    privacy: { flex: 1, fontFamily: fonts.regular, fontSize: 13, lineHeight: 19, color: colors.textDim },
});

```

strings.ts src/i18n

- Текстовете на български и английски (`mosques` , `nav`).

src/i18n/strings.ts

1019 реда

```

import type { CalculationMethodId, HighLatRuleId } from '@domain/calc';
import type { PrayerId } from '@domain/prayers';
import type { AlertMode } from '@store/alertPrefs';
import type { BatteryBrand } from '@domain/device';

/**
 * Всички текстове на приложението на двата езика.
 * `en` е типизиран като `Strings` (формата на `bg`), затова TypeScript
 * дава грешка, ако някой ключ липсва в единия от езиците.
 */
export const bg = {

```

```

prayers: {
  fajr: 'Фаджр',
  sunrise: 'Изгрев',
  dhuhr: 'Зухр',
  asr: 'Аср',
  maghrib: 'Магриб',
  isha: 'Иша',
} as Record<PrayerId, string>,

alert: {
  off: 'без известие',
  notify: 'известие',
  adhan: 'аларма',
} as Record<AlertMode, string>,

tabs: {
  today: 'Днес',
  qibla: 'Кибла',
  mosques: 'Джамии',
  month: 'Месец',
  settings: 'Настройки',
},

hero: {
  next: 'Следваща',
  at: (time: string) => `в ${time}`,
  tomorrowAt: (time: string) => `утре в ${time}`,
  remaining: (time: string) => `Остават ${time}`,
},

nowPill: 'СЕГА',

busy: 'Зарежда...',
locating: 'Определям мястото...',
choosePlace: 'Избери място',
noPlaceBanner: 'Мястото не е определено – часовете са за София. Докосни, за да избереш място.',

ally: {
  city: (city: string) => `Град: ${city}. Смени града`,
  locating: 'Определям местоположението',
  bell: (prayer: string, mode: string) => `${prayer}: ${mode}. Докосни за смяна`,
},

toast: (prayer: string, mode: string) => `${prayer}: ${mode}`,
alarmToast: (prayer: string) => `${prayer}: аларма – звучи и при тих режим`,

placeholder: {
  qibla: 'Компас към Кябе. Идва в етап 7 от плана.',
},

month: {
  dayColumn: 'Ден',
  hijriColumn: 'Хиджра',
  today: 'Днес',
  prev: 'Предишен месец',
  next: 'Следващ месец',
  friday: 'петък',
  sourceMufti: 'Източник: Главно муфтийство на Република България',
  sourceCalc: (method: string) => `Изчислено по метод ${method}`,
},

```

```

settings: {
  title: 'Настройки',
  groupPlace: 'Място',
  groupTimes: 'Часове',
  groupCorrections: 'Корекции',
  groupApp: 'Приложение',
  about: 'За приложението',
  change: 'Смени мястото',
  auto: 'Автоматично (GPS)',
  autoHint: 'Мястото се определя при отваряне на приложението.',
  locating: 'Определям местоположението...',
  denied: 'Няма разрешение за местоположение.',
  openSettings: 'Отвори настройките на телефона',
  unavailable: 'Местоположението не може да се определи. Показва се последното известно място.',
  refresh: 'Обнови местоположението',
  sourceMufti: 'Календар на Главно мюфтийство',
  sourceMuftiNote: 'В България часовете са по официалния календар. Методът и Аср се избират само извън
България.',
  sourceCalc: 'Изчисление за мястото',
  method: 'Метод',
  methodAuto: 'Автоматично',
  methodAutoValue: (method: string) => `Авто · ${method}`,
  methodAutoDesc: (country: string, method: string) => `според държавата: ${country} → ${method}`,
  asr: 'Аср',
  shafi: 'Шафии, Малики, Ханбали',
  hanafi: 'Ханафи',
  shafiDesc: 'Сянката е равна на предмета – „първият“ Аср, като в календара на Диянет',
  hanafiDesc: 'Сянката е два пъти по-дълга – около час по-късно',
  asrNote: 'Шафии, Малики и Ханбали смятат Аср еднакво; само при Ханафи е по-късно.',
  polarNote: (lat: string) =>
    `На ${lat}° ширина има полярен ден и полярна нощ. В дните без изгрев или залез часовете са изчислени
за 65° ширина – най-близките места с нормален ден и нощ. Свърете с местната джамия.`,
  highLat: 'Северни ширини',
  highLatNote: (lat: string) =>
    `На ${lat}° ширина през лятото няма истински здрач и правилото определя Фаджр и Иша. Свърете с
местната джамия.`,
  offsetsNote: 'Ако джамията ти е с няколко минути по-различна, нагласи тук. Важи за всички екрани.',
  minutes: (n: number) => (n === 0 ? '0 мин' : `${n > 0 ? '+' : ''}${n} мин`),
  hijri: 'Дата по Хиджра',
  days: (n: number) => (n === 0 ? '0 дни' : `${n > 0 ? '+' : ''}${n} ${Math.abs(n) === 1 ? 'ден' :
'дни'}`),
  hijriNote: 'Ако датата по Хиджра се разминава с обявената от Мюфтийството.',
  hijriNoteAbroad: 'Ако датата по Хиджра се разминава с обявената от местната джамия.',
  reset: 'Нулирай корекциите',
  decrease: (label: string) => `Намали: ${label}`,
  increase: (label: string) => `Увеличи: ${label}`,
},

widget: {
  next: 'Следваща',
  tomorrowShort: (time: string) => `утре ${time}`,
  until: (name: string) => `остават до ${name}`,
  left: 'остават',
  empty: 'Отвори Езан, за да се покажат часовете.',
},

notifications: {
  prayerTitle: (prayer: string, time: string) => `${prayer} – ${time}`,
  prayerBody: (prayer: string, arabic: string, place: string) => `Време е за намаза ${prayer} (${arabic})
· ${place}`,
  sunriseBody: (place: string) => `Изгрев – край на времето за Фаджр · ${place}`,
}

```

```

nextLine: (prayer: string, time: string, tomorrow: boolean) =>
    `Следваща: ${prayer} в ${time}${tomorrow ? ' (утре)' : ''}`,
reminderTitle: (prayer: string, minutes: number, time: string) => `${prayer} след ${minutes} мин. -
${time}`,
reminderBody: (place: string) => `Време е за абдест и подготовка за намаза · ${place}`,
refreshTitle: 'Отвори Езан',
refreshBody: 'Отвори приложението, за да продължат известията за намаз.',
channelPrayer: 'Молитви',
channelAdhan: 'Аларма',
channelReminder: 'Напомняния',
group: 'Известия',
status: 'Разрешение',
granted: 'Дадено',
denied: 'Отказано',
undetermined: 'Не е дадено',
allow: 'Разреши известията',
deniedNote: 'Известията са изключени в настройките на телефона – без тях известията и алармите за намаз
няма да работят.',
webNote: 'Известията работят само в приложението на телефона.',
reminder: 'Напомняне преди молитвата',
reminderOff: 'Без напомняне',
reminderValue: (n: number) => `${n} мин. преди`,
vibrate: 'Вибрация',
exact: 'Точни известия',
exactOff: 'Изключени',
exactNote:
    'Без разрешението „Аларми и напомняния“ Android може да забави известията с няколко минути.',
exactAllow: 'Разреши „Аларми и напомняния“',
scheduled: (n: number, until: string) => `Планирани ${n} известия – до ${until}.`,
scheduledNone: 'Няма планирани известия – всички камбанки на „Днес“ са изключени.',
testTitle: 'Пробно известие',
testBody: 'Така ще изглеждат известията за намаз.',
bellDenied: 'Известията не са разрешени – известията и алармите няма да работят.',
allowedToast: '✓ Известията са разрешени',
exactBanner: 'Известията може да закъснеят с няколко минути. Докосни, за да разрешиш точни известия.',
deniedBanner: 'Известията не са разрешени – известията и алармите няма да работят. Докосни, за да ги
разрешиш.',
diagnostics: 'Проверка на известията',
diag: {
    title: 'Проверка на известията',
    subtitle: 'Дали телефонът показва известията на Езан навреме – и какво да оправиш, ако не.',
    refresh: 'Обнови',
    updated: (time: string) => `Обновено в ${time}`,
    status: 'Състояние',
    permOk: 'Известията са разрешени',
    permBad: 'Известията не са разрешени',
    exactOk: 'Точни известия („Аларми и напомняния“)',
    exactBad: 'Точните известия са изключени – може да закъсняват',
    allow: 'Разреши',
    scheduledOk: (n: number, until: string) => `Планирани ${n} известия · до ${until}`,
    scheduledNone: 'Няма планирани известия – камбанките на „Днес“ са изключени',
    test: 'Пробно известие',
    testNotification: 'Известие',
    testIn: (s: number) => `след ${s} сек.`,
    testHint: 'Идва след 10 сек. Заклучи телефона, за да го видиш и чуеш както ще идва. Звукът на
алармата се преслущва в Настройки → Звуци.',
    recent: 'Последни известия',
    recentNone: 'Още няма показани известия от Езан.',
    onTime: 'навреме',
    late: (s: number) => (s < 60 ? `+${s} сек.` : `+${Math.round(s / 6) / 10} мин.`),
    plannedShown: (planned: string, shown: string) => `планирано ${planned} · показано ${shown}`,

```

```

    upcoming: 'Следващи известия',
    kindAlarm: 'Аларма',
    kindNotification: 'Известие',
    kindReminder: 'Напомняне',
    kindOther: 'Друго',
    tech: 'Технически данни',
    device: 'Телефон',
    channelLine: (importance: number, sound: string, vibrate: boolean, usage: string) =>
        `важност ${importance} · звук ${sound} · вибрация ${vibrate ? 'да' : 'не'} · поток ${usage}`,
    reset: 'Пренасрочи всички известия',
    resetHint: 'Трие всички известия на Езан и ги планира наново по текущите настройки. Ползвай го, ако
нещо изглежда объркано.',
    resetDone: '✓ Всички известия са планирани наново.',
    fullScreenOk: 'Аларма на цял екран',
    fullScreenBad: 'Аларма на цял екран е изключена',
    batteryOk: 'Работа на заден план без ограничения',
    batteryBad: 'Телефонът ограничава работата на заден план',
    fix: 'Поправи',
},
fullScreen: 'Аларма на цял екран',
fullScreenOn: 'Разрешена',
fullScreenOff: 'Изключена',
fullScreenNote: 'Без нея при заключен телефон езанът пак звучи, но се вижда само известие – не целият
екран.',
fullScreenAllow: 'Разреши „Аларма на цял екран“',
exactOn: 'Разрешени',
background: 'Работа на заден план',
backgroundOk: 'Без ограничения',
backgroundBad: 'Ограничена',
batteryTitle: 'Алармите може да закъснеят',
batteryText: 'Телефонът може да спре приложението, за да пести батерия.',
batteryFix: 'Поправи',
},

qibla: {
    title: 'Кибла',
    fromNorth: 'от север',
    toMecca: 'до Мека',
    km: 'км',
    aligned: '✓ Обърнат си към Кибла',
    turnLabel: (direction: 'left' | 'right') => `Завърти се ${direction === 'right' ? 'надясно' :
'наляво'}`,
    starting: 'Компасът се включва...',
    quality: { high: 'висока', medium: 'средна', low: 'ниска' } as Record<'high' | 'medium' | 'low',
string>,
    accuracy: (q: string) => `Компас: ${q} точност`,
    tip: 'Дръж телефона хоризонтално, с екрана нагоре. Метал и магнити наблизо объркват компаса.',
    calibrateTitle: 'Калибрирай компаса',
    calibrateText: 'Движи телефона като осмица във въздуха 2-3 пъти. Дръж го далеч от метал, магнити и
магнитни калъфи.',
    noCompassTitle: 'Няма компас',
    noCompassText: (deg: number) =>
        `Този телефон няма магнитен сензор. Киблата е на ${deg}° от север: застани с лице на север и се
завърти надясно на ${deg}°`,
    permissionTitle: 'Нужно е разрешение',
    permissionText: 'На iPhone компасът работи само с разрешение за местоположение.',
    allowLocation: 'Разреши местоположението',
    letters: ['С', 'И', 'Ю', 'З'] as readonly string[],
    allDial: (bearing: number) => `Компас. Киблата е на ${bearing}° от север.`,
    // {t} е часът – показва се удебелен и златен
    sunToday: 'Днес в {t} слънцето е точно в посоката на Киблата',
}

```

```

sunTomorrow: 'Утре в {t} слънцето ще е точно в посоката на Киблата',
sunNow: 'Сега слънцето е точно в посоката на Киблата',
sunrise: (azimuth: number, degrees: number, side: 'left' | 'right' | 'same') =>
  side === 'same'
    ? `Изгрев на ${azimuth}° – точно в посоката на Киблата`
    : `Изгрев на ${azimuth}° – Киблата е ${degrees}° ${side === 'right' ? 'вдясно' : 'вляво'} от него`,
},

sounds: {
  group: 'Звуци',
  notifyGroup: 'Напомняне и известие',
  notifyRow: 'Звук на известията',
  noteAndroid: 'Звукът свири, когато камбанката на молитвата е „аларма“. Без „Аларми и напомняния“ – краткият звук (до 30 сек.).',
  noteIos: 'Звукът свири, когато камбанката на молитвата е „аларма“. На iPhone звукът е до 30 сек.',
  fullOffTitle: 'Пълният езан е изключен',
  fullOffText: 'Без „Аларми и напомняния“ свири само краткият звук (до 30 сек.). За пълния езан разреши „Аларми и напомняния“.',
  allow: 'Разреши',
  title: (prayer: string) => `Звук за ${prayer}`,
  notifyTitle: 'Звук на известията',
  fullSection: 'Пълен звук – аларма',
  fullHint: 'Свири докрай; „Спри“ го прекъсва.',
  shortSection: 'Кратък звук – до 30 сек.',
  shortHintAndroid: 'Когато „Аларми и напомняния“ е изключено. Своят звук се скъсява до 30 сек. – на пауза, с плавно заглъхване.',
  iosSection: 'Звук – до 30 сек.',
  iosHint: 'На iPhone звукът на известие е най-много 30 сек. По-дългите записи се скъсяват на пауза, с плавно заглъхване.',
  notifyHint: 'За известията и напомнянията преди молитвата.',
  byDefault: 'по подразбиране',
  mine: 'МОЙ',
  add: 'Добави свой звук',
  addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · от телефона',
  addShortHint: 'mp3, m4a, mp4, wav... · по-дълъг – скъсява се до 30 сек.',
  addNotifyHint: 'до 10 сек. · mp3, m4a, mp4, wav...',
  tooLongNotify: (name: string, dur: string) =>
    `„${name}“ е ${dur} – звукът за известия може да е до 10 сек. Избери по-кратък.`,
  adding: 'Добавяне...',
  workingShort: 'Изрязване на откъса...',
  workingFull: 'Добавяне на звука...',
  workingHint: 'Търси се паузата до 30-ата секунда. Може да отнеме няколко секунди.',
  applyAll: 'Същият звук за всички молитви',
  apply: 'Приложи',
  applied: '✓ Приложено за всички молитви',
  iosFuture: 'Пълният езан на iPhone ще дойде с AlarmKit (iOS 26) – етап 9.',
  added: (name: string) => `✓ Добавен: ${name}`,
  addedTrimmed: (name: string, dur: string) =>
    `✓ Добавен: ${name}. Краткият звук е до 30 сек. – ще свирят първите ${dur}, до паузата, с плавно заглъхване.`,
  trimmed: (dur: string, of: string) => `свирят първите ${dur} от ${of}`,
  tooLongFull: (name: string) => `„${name}“ е по-дълъг от 15 мин.`,
  tooBig: 'Файлът е по-голям от 60 MB.',
  unreadable: 'Този файл не може да се прочете като звук.',
  unsupported: 'Своите звуци се добавят само в приложението на телефона.',
  deleteTitle: (name: string) => `Да махна ли „${name}“?`,
  deleteText: 'Молитвите с този звук се връщат към звука по подразбиране.',
  delete: 'Махни',
  cancel: 'Отказ',
  play: (name: string) => `Пусни ${name}`,
  stop: (name: string) => `Спри ${name}`,

```

```

},

alarm: {
  app: 'Езан',
  title: (prayer: string) => `Време е за ${prayer}`,
  stop: 'Спри езана',
  stopAlarm: 'Спри алармата',
  mute: 'Заглуши звука',
  muteShort: 'Заглуши',
  close: 'Затвори',
  soundName: 'Езан',
  channel: 'Аларма за молитва',
},

battery: {
  title: 'Работа на заден план',
  lede: 'За да пести батерия, телефонът може да „приспи“ Езан и алармата да закъснее или да не звънне.
Разреши на приложението да работи без ограничения.',
  yourPhone: 'Твоят телефон',
  open: 'Отвори настройките на Езан',
  openList: 'Списък „Оптимизация на батерията“',
  now: 'Сега',
  ok: 'без ограничения ✓',
  bad: 'ограничено',
  unknown: 'не може да се провери',
  otherPhone: 'Друг телефон?',
  more: 'Още съвети за всяка марка: dontkillmyapp.com',
  brands: {
    samsung: 'Samsung',
    xiaomi: 'Xiaomi / Redmi / POCO',
    huawei: 'Huawei / Honor',
    oppo: 'OPPO / Realme / OnePlus',
    other: 'Друг',
  } as Record<BatteryBrand, string>,
  // [текст, път (по желание), бележка (по желание)]
  steps: {
    samsung: [
      ['Натисни бутона долу – отворят се настройките на Езан.'],
      ['Отвори', 'Батерия'],
      ['Избери', 'Без ограничения', 'Не „Оптимизирана“ и не „Ограничена“'],
      ['По желание:', 'Настройки → Батерия → Ограничения на фона', 'Езан да не е в „Приложения в дълбок
сън“ и „Приложения в сън“'],
    ],
    xiaomi: [
      ['Натисни бутона долу – отворят се настройките на Езан.'],
      ['Отвори', 'Пестене на батерия', 'Избери „Без ограничения“'],
      ['Включи', 'Автоматично стартиране'],
      ['По желание: в последните приложения задръж картата на Езан и я заключи (катинарче).'],
    ],
    huawei: [
      ['Отвори', 'Настройки → Батерия → Стартиране на приложения'],
      ['Намери Езан и изключи', 'Управлявай автоматично'],
      ['В прозореца включи и трите:', 'Автоматично стартиране · Вторично стартиране · Работа на заден
план'],
    ],
    oppo: [
      ['Натисни бутона долу – отворят се настройките на Езан.'],
      ['Отвори', 'Използване на батерията'],
      ['Включи', 'Разреши работа на заден план', 'и „Автоматично стартиране“, ако го има'],
    ],
    other: [

```

```

        ['Натисни бутона долу – отварят се настройките на Езан.'],
        ['Отвори', 'Батерия'],
        ['Избери', 'Без ограничения', 'На някои телефони се казва „Не оптимизирай“.'],
    ],
} as Record<BatteryBrand, readonly (readonly string[])>,
},

about: {
    title: 'За приложението',
    name: 'Езан',
    back: 'Настройки',
    version: (v: string) => `Версия ${v}`,
    tagline: 'Часове за намаз, напомняния и езан',
    groupTimes: 'Източници на часовете',
    groupPlaces: 'Данни за местата',
    groupPrivacy: 'Поверителност',
    groupMap: 'Картата и джамиите',
    sources: {
        mufti: { name: 'Главна мюфтийство на Република България', desc: 'Календар за намаз – часовете в България' },
        adhan: { name: 'adhan', desc: 'Изчисление извън България · лиценз MIT' },
        osm: { name: 'OpenStreetMap', desc: 'Населени места в България · © OpenStreetMap contributors, ODbL' },
    },
    ne: { name: 'Natural Earth', desc: 'Граница на България · public domain' },
    geonames: { name: 'GeoNames', desc: 'Градове по света – името и държавата без интернет · CC BY 4.0' },
    praytimes: { name: 'PrayTimes.org', desc: 'Записите на езан: Мека, Медина, Ал-Акса' },
    overpass: { name: 'OpenStreetMap · Overpass API', desc: 'Джамиите наблизо · © OpenStreetMap contributors, ODbL' },
    openfreemap: { name: 'OpenFreeMap', desc: 'Картата · © OpenMapTiles, данни © OpenStreetMap' },
    },
    privacy:
        'Местоположението се ползва на телефона – за да се изберат населеното място и часовете. Извън България телефонът пита своята картова услуга за името на града. За „Джамии наблизо“ до OpenStreetMap се изпраща само приблизителен район (около 1 км), а картата се зарежда от OpenFreeMap.',
    },

    mosques: {
        title: 'Джамии наблизо',
        count: (n: number, dist: string) => `${n} · до ${dist}`,
        next: 'Следваща:',
        nextAt: (prayer: string, time: string) => `${prayer} в ${time}`,
        nextIn: (left: string) => `след ${left}`,
        walk: (min: number) => `${min} мин пеша`,
        drive: (min: number) => `с кола ~${min} мин`,
        unnamed: 'Джамия',
        unnamedSub: 'без име в OpenStreetMap',
        locating: 'Търся къде си...',
        loading: 'Търся джамиите наоколо...',
        noneNear: (dist: string) => `Няма джамии до ${dist}`,
        noneFar: (dist: string) => `Няма джамии и до ${dist}`,
        noneHint: 'Картата е на OpenStreetMap – в малките места не всяка джамия е отбелязана.',
        searchFar: (dist: string) => `Търси до ${dist}`,
        within: (dist: string) => `до ${dist}`,
        offlineTitle: 'Няма интернет.',
        offlineText: (when: string) => `Показвам последното търсене тук – от ${when}. Упътването ще работи, щом има връзка.`,
        today: 'днес',
        yesterday: 'вчера',
        errorOffline: 'Няма интернет',
        errorOfflineText: 'Джамиите се търсят в OpenStreetMap – нужна е връзка.',
    },

```

```

errorServer: 'Сървърът не отговаря',
errorServerText: 'OpenStreetMap (Overpass) е претоварен. Опитай пак след минута.',
retry: 'Опитай пак',
noGps: (place: string) => `Без достъп до местоположението – търся около ${place}.`,
allow: 'Разреши',
myLocation: 'Моето място',
locate: 'Към моето място',
back: 'Назад към списъка',
reach: (min: number) => `ще стигнеш за ${min} мин`,
directions: (app: string) => `Упътване с ${app}`,
directionsAuto: 'Упътване',
withApp: (app: string) => `С ${app} · смени в Настройки → Навигация`,
withAuto: 'С картите на телефона · смени в Настройки → Навигация',
share: 'Сподели',
copyAddress: 'Копирай адреса',
copyCoords: 'Копирай координатите',
copied: 'Копирано',
noAddress: 'Няма адрес в OpenStreetMap',
allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
allyMap: (n: number) => `Карта с ${n} джамии`,
attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',
},

nav: {
  title: 'Навигация',
  back: 'Настройки',
  intro: 'С кое приложение да се отваря „Упътване“ до джамията.',
  auto: 'Автоматично',
  autoAndroid: 'Картите по подразбиране на телефона',
  autoIos: 'Apple Maps',
  walkDrive: 'Пеша и с кола',
  driveOnly: 'Само с кола',
  notInstalled: 'не е инсталирано',
  organic: 'OpenStreetMap · и без интернет',
  huawei: 'на Huawei',
  download: 'Изтегли',
  note: 'Waze води само с кола – и за джамия на 300 м ще те упъти с колата. Ако избраното приложение липсва, „Упътване“ пита с какво да отвори.',
  sheetTitle: (name: string) => `Упътване до ${name}`,
  sheetSub: (dist: string) => `${dist} · с кое приложение?`,
  walk: 'пеша',
  drive: 'с кола',
  remember: 'Запомни избора',
  rememberHint: 'Смяна: Настройки → Навигация',
  missing: (app: string) => `${app} вече не е на телефона – избери друго приложение.`,
  group: 'Джамии наблизко',
  row: 'Навигация',
  rowNote: 'С кое приложение се отваря „Упътване“ до джамия.',
  ask: 'Пита при упътване',
  close: 'Затвори',
},

methods: {
  Turkey: 'Диянет (Турция)',
  MuslimWorldLeague: 'Muslim World League',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Египетски генерален орган',
  Karachi: 'Университет в Карачи',
  UmmAlQura: 'Umm al-Qura (Мека)',
  NorthAmerica: 'ISNA (Северна Америка)',
  Dubai: 'Дубай (ОАЕ)',

```

```

Kuwait: 'Кувейт',
Qatar: 'Катар',
Singapore: 'Сингапур (MUIS)',
Tehran: 'Техеран (Институт по геофизика)',
France: 'Франция (UOIF, 12°)',
Russia: 'Русия (ДУМ)',
Malaysia: 'Малайзия (JAKIM)',
Indonesia: 'Индонезия (Kemenag)',
} as Record<CalculationMethodId, string>,

highLatRules: {
  seventhofthenight: { name: 'Една седма от нощта', desc: 'Най-често ползваното в Европа' },
  middleofthenight: { name: 'Средата на нощта', desc: 'Фаджр и Иша не минават средата на нощта' },
  twilightangle: { name: 'По ъгъла на здрача', desc: 'Част от нощта според ъгъла на метода' },
} as Record<HighLatRuleId, { name: string; desc: string }>,

place: {
  title: 'Избери място',
  search: 'Търси населено място в България',
  useGps: 'Използвай местоположението ми (GPS)',
  cities: 'Градове',
  noResults: 'Няма такова населено място',
  abroadNote: 'Извън България мястото се определя с GPS.',
  close: 'Затвори',
},

date: {
  weekdaysShort: ['Нд', 'Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб'],
  // съкращенията са с точка; „март“, „май“, „юни“, „юли“ са пълни думи
  monthsShort: ['яну.', 'фев.', 'март', 'апр.', 'май', 'юни', 'юли', 'авг.', 'септ.', 'окт.', 'ное.',
'дек.'],
  monthsFull: ['Януари', 'Февруари', 'Март', 'Април', 'Май', 'Юни', 'Юли', 'Август', 'Септември',
'Октомври', 'Ноември', 'Декември'],
},

hijriMonths: [
  'Мухаррем',
  'Сафер',
  'Ребиул-евел',
  'Ребиул-ахир',
  'Джемазиел-евел',
  'Джемазиел-ахир',
  'Реджеб',
  'Шабан',
  'Рамазан',
  'Шеввал',
  'Зилкаде',
  'Зилхидже',
],
};

export type Strings = typeof bg;

export const en: Strings = {
  prayers: {
    fajr: 'Fajr',
    sunrise: 'Sunrise',
    dhuhr: 'Dhuhr',
    asr: 'Asr',
    maghrib: 'Maghrib',
    isha: 'Isha',
  },

```

```

},

alert: {
  off: 'no notification',
  notify: 'notification',
  adhan: 'alarm',
},

tabs: {
  today: 'Today',
  qibla: 'Qibla',
  mosques: 'Mosques',
  month: 'Month',
  settings: 'Settings',
},

hero: {
  next: 'Next',
  at: (time) => `at ${time}`,
  tomorrowAt: (time) => `tomorrow at ${time}`,
  remaining: (time) => `${time} left`,
},

nowPill: 'NOW',

busy: 'Loading...',
locating: 'Finding your location...',
choosePlace: 'Choose place',
noPlaceBanner: 'Location not found - the times shown are for Sofia. Tap to choose your place.',

ally: {
  city: (city) => `City: ${city}. Change city`,
  locating: 'Finding your location',
  bell: (prayer, mode) => `${prayer}: ${mode}. Tap to change`,
},

toast: (prayer, mode) => `${prayer}: ${mode}`,
alarmToast: (prayer) => `${prayer}: alarm - sounds even in silent mode`,

placeholder: {
  qibla: 'Compass pointing to the Kaaba. Coming in stage 7.',
},

month: {
  dayColumn: 'Day',
  hijriColumn: 'Hijri',
  today: 'Today',
  prev: 'Previous month',
  next: 'Next month',
  friday: 'Friday',
  sourceMufti: 'Source: Grand Mufti of Bulgaria',
  sourceCalc: (method) => `Calculated with the ${method} method`,
},

settings: {
  title: 'Settings',
  groupPlace: 'Location',
  groupTimes: 'Prayer times',
  groupCorrections: 'Adjustments',
  groupApp: 'App',
  about: 'About',
}

```

```

change: 'Change location',
auto: 'Automatic (GPS)',
autoHint: 'Your location is detected when the app opens.',
locating: 'Detecting your location...',
denied: 'Location permission is off.',
openSettings: 'Open phone settings',
unavailable: 'Your location could not be detected. Showing the last known place.',
refresh: 'Update location',
sourceMufti: 'Grand Mufti of Bulgaria calendar',
sourceMuftiNote: 'In Bulgaria the official calendar is used. Method and Asr apply only outside
Bulgaria.',
sourceCalc: 'Calculated for this place',
method: 'Method',
methodAuto: 'Automatic',
methodAutoValue: (method) => `Auto · ${method}`,
methodAutoDesc: (country, method) => `by country: ${country} → ${method}`,
asr: 'Asr',
shafi: "Shafi'i, Maliki, Hanbali",
hanafi: 'Hanafi',
shafiDesc: 'Shadow equal to the object - the "first" Asr, as in the Diyanet calendar',
hanafiDesc: 'Shadow twice the object - about an hour later',
asrNote: "Shafi'i, Maliki and Hanbali calculate Asr the same way; only Hanafi is later.",
polarNote: (lat) =>
  `At ${lat}° latitude there are polar days and nights. On days without sunrise or sunset, times are
  calculated for 65° latitude - the nearest places with a normal day and night. Check with your local
  mosque.`,
  highLat: 'High latitudes',
  highLatNote: (lat) =>
    `At ${lat}° latitude there is no true twilight in summer, so this rule sets Fajr and Isha. Check with
    your local mosque.`,
  offsetsNote: 'If your mosque differs by a few minutes, adjust it here. Applies to all screens.',
  minutes: (n) => (n === 0 ? '0 min' : `${n > 0 ? '+' : ''}${n} min`),
  hijri: 'Hijri date',
  days: (n) => (n === 0 ? '0 days' : `${n > 0 ? '+' : ''}${n} ${Math.abs(n) === 1 ? 'day' : 'days'}`),
  hijriNote: 'If the Hijri date differs from the one announced by the Grand Mufti of Bulgaria.',
  hijriNoteAbroad: 'If the Hijri date differs from the one announced by your local mosque.',
  reset: 'Reset adjustments',
  decrease: (label) => `Decrease: ${label}`,
  increase: (label) => `Increase: ${label}`,
},

widget: {
  next: 'Next',
  tomorrowShort: (time) => `tomorrow ${time}`,
  until: (name) => `until ${name}`,
  left: 'left',
  empty: 'Open Adhan to see the prayer times.',
},

notifications: {
  prayerTitle: (prayer, time) => `${prayer} - ${time}`,
  prayerBody: (prayer, arabic, place) => `It's time for the ${prayer} prayer (${arabic}) · ${place}`,
  sunriseBody: (place) => `Sunrise - the time for Fajr has ended · ${place}`,
  nextLine: (prayer, time, tomorrow) => `Next: ${prayer} at ${time}${tomorrow ? ' (tomorrow)' : ''}`,
  reminderTitle: (prayer, minutes, time) => `${prayer} in ${minutes} min - ${time}`,
  reminderBody: (place) => `Time for wudu and to get ready for prayer · ${place}`,
  refreshTitle: 'Open Adhan',
  refreshBody: 'Open the app to keep prayer notifications coming.',
  channelPrayer: 'Prayers',
  channelAdhan: 'Alarm',
  channelReminder: 'Reminders',
}

```

```

group: 'Notifications',
status: 'Permission',
granted: 'Allowed',
denied: 'Denied',
undetermined: 'Not asked yet',
allow: 'Allow notifications',
deniedNote: "Notifications are turned off in the phone settings – without them prayer notifications and
alarms won't work.",
webNote: 'Notifications work only in the phone app.',
reminder: 'Reminder before prayer',
reminderOff: 'No reminder',
reminderValue: (n) => `${n} min before`,
vibrate: 'Vibration',
exact: 'Exact notifications',
exactOff: 'Off',
exactNote: 'Without the "Alarms & reminders" permission Android may delay notifications by a few
minutes.',
exactAllow: 'Allow "Alarms & reminders"',
scheduled: (n, until) => `${n} notifications scheduled – until ${until}.`,
scheduledNone: 'No notifications scheduled – all bells on "Today" are off.',
testTitle: 'Test notification',
testBody: 'This is how prayer notifications will look.',
bellDenied: "Notifications are not allowed – notifications and alarms won't work.",
allowedToast: '✓ Notifications are allowed',
exactBanner: 'Notifications may be a few minutes late. Tap to allow exact notifications.',
deniedBanner: "Notifications are not allowed – notifications and alarms won't work. Tap to allow
them.",
diagnostics: 'Notification check',
diag: {
  title: 'Notification check',
  subtitle: 'Whether the phone shows Adhan notifications on time – and what to fix if not.',
  refresh: 'Refresh',
  updated: (time) => `Updated at ${time}`,
  status: 'Status',
  permOk: 'Notifications are allowed',
  permBad: 'Notifications are not allowed',
  exactOk: 'Exact notifications ("Alarms & reminders")',
  exactBad: 'Exact notifications are off – they may be late',
  allow: 'Allow',
  scheduledOk: (n, until) => `${n} notifications scheduled – until ${until}`,
  scheduledNone: 'No notifications scheduled – the bells on "Today" are off',
  test: 'Test notification',
  testNotification: 'Notification',
  testIn: (s) => `in ${s} s`,
  testHint: 'Arrives in 10 s. Lock the phone to see and hear it as it will arrive. The alarm sound can
be previewed in Settings → Sounds.',
  recent: 'Recent notifications',
  recentNone: 'No Adhan notifications shown yet.',
  onTime: 'on time',
  late: (s) => (s < 60 ? `+${s} s` : `+${Math.round(s / 6) / 10} min`),
  plannedShown: (planned, shown) => `planned ${planned} · shown ${shown}`,
  upcoming: 'Upcoming notifications',
  kindAlarm: 'Alarm',
  kindNotification: 'Notification',
  kindReminder: 'Reminder',
  kindOther: 'Other',
  tech: 'Technical details',
  device: 'Phone',
  channelLine: (importance, sound, vibrate, usage) =>
    `importance ${importance} · sound ${sound} · vibration ${vibrate ? 'yes' : 'no'} · stream
    ${usage}`,

```

```

        reset: 'Reschedule all notifications',
        resetHint: 'Deletes all Adhan notifications and schedules them again from the current settings. Use
it if something looks wrong.',
        resetDone: '✓ All notifications have been scheduled again.',
        fullScreenOk: 'Full-screen alarm',
        fullScreenBad: 'Full-screen alarm is off',
        batteryOk: 'Runs in the background without limits',
        batteryBad: 'The phone limits background activity',
        fix: 'Fix',
    },
    fullScreen: 'Full-screen alarm',
    fullScreenOn: 'Allowed',
    fullScreenOff: 'Off',
    fullScreenNote: 'Without it the adhan still plays on a locked phone, but you only see a notification -
not the full screen.',
    fullScreenAllow: 'Allow "Full-screen alarm"',
    exactOn: 'Allowed',
    background: 'Background activity',
    backgroundOk: 'Unrestricted',
    backgroundBad: 'Restricted',
    batteryTitle: 'Alarms may be late',
    batteryText: 'The phone may stop the app to save battery.',
    batteryFix: 'Fix',
},

qibla: {
    title: 'Qibla',
    fromNorth: 'from north',
    toMecca: 'to Mecca',
    km: 'km',
    aligned: '✓ You are facing the Qibla',
    turnLabel: (direction) => `Turn ${direction}`,
    starting: 'Starting the compass...',
    quality: { high: 'high', medium: 'medium', low: 'low' },
    accuracy: (q) => `Compass: ${q} accuracy`,
    tip: 'Hold the phone flat, screen up. Metal and magnets nearby confuse the compass.',
    calibrateTitle: 'Calibrate the compass',
    calibrateText: 'Move the phone in a figure-eight in the air 2-3 times. Keep it away from metal, magnets
and magnetic cases.',
    noCompassTitle: 'No compass',
    noCompassText: (deg) =>
        `This phone has no magnetic sensor. The Qibla is ${deg}° from north: face north and turn right by
${deg}°`,
    permissionTitle: 'Permission needed',
    permissionText: 'On iPhone the compass works only with location permission.',
    allowLocation: 'Allow location',
    letters: ['N', 'E', 'S', 'W'],
    allyDial: (bearing) => `Compass. The Qibla is ${bearing}° from north.`,
    sunToday: 'Today at {t} the sun is exactly in the Qibla direction',
    sunTomorrow: 'Tomorrow at {t} the sun will be exactly in the Qibla direction',
    sunNow: 'Right now the sun is exactly in the Qibla direction',
    sunrise: (azimuth, degrees, side) =>
        side === 'same'
            ? `Sunrise at ${azimuth}° - exactly in the Qibla direction`
            : `Sunrise at ${azimuth}° - the Qibla is ${degrees}° to its ${side}`,
},

sounds: {
    group: 'Sounds',
    notifyGroup: 'Reminder and notification',
    notifyRow: 'Notification sound',

```

```

    noteAndroid: 'The sound plays when the prayer bell is set to "alarm". Without "Alarms & reminders" -
the short sound (up to 30 s).',
    noteIos: 'The sound plays when the prayer bell is set to "alarm". On iPhone the sound is up to 30 s.',
    fullOffTitle: 'The full adhan is off',
    fullOffText: 'Without "Alarms & reminders" only the short sound plays (up to 30 s). To get the full
adhan, allow "Alarms & reminders".',
    allow: 'Allow',
    title: (prayer) => `Sound for ${prayer}`,
    notifyTitle: 'Notification sound',
    fullSection: 'Full sound - alarm',
    fullHint: 'Plays to the end; "Stop" ends it.',
    shortSection: 'Short sound - up to 30 s',
    shortHintAndroid: 'When "Alarms & reminders" is off. Your own sound is trimmed to 30 s - at a pause,
with a gentle fade-out.',
    iosSection: 'Sound - up to 30 s',
    iosHint: 'On iPhone a notification sound can be 30 s at most. Longer recordings are trimmed at a pause,
with a gentle fade-out.',
    notifyHint: 'For notifications and reminders before prayer.',
    byDefault: 'default',
    mine: 'MINE',
    add: 'Add your own sound',
    addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · from the phone',
    addShortHint: 'mp3, m4a, mp4, wav... · longer - trimmed to 30 s',
    addNotifyHint: 'up to 10 s · mp3, m4a, mp4, wav...',
    tooLongNotify: (name, dur) => `${name} is ${dur} - a notification sound can be up to 10 s. Pick a
shorter one.`,
    adding: 'Adding...',
    workingShort: 'Trimming the excerpt...',
    workingFull: 'Adding the sound...',
    workingHint: 'Looking for the pause before the 30th second. This may take a few seconds.',
    applyAll: 'Same sound for all prayers',
    apply: 'Apply',
    applied: '✓ Applied to all prayers',
    iosFuture: 'The full adhan on iPhone will come with AlarmKit (iOS 26) - stage 9.',
    added: (name) => `✓ Added: ${name}`,
    addedTrimmed: (name, dur) =>
        `✓ Added: ${name}. The short sound is up to 30 s - the first ${dur} will play, up to a pause, with a
gentle fade-out.`,
    trimmed: (dur, of) => `plays the first ${dur} of ${of}`,
    tooLongFull: (name) => `${name} is longer than 15 min.`,
    tooBig: 'The file is larger than 60 MB.',
    unreadable: 'This file can't be read as a sound.',
    unsupported: 'Your own sounds can only be added in the phone app.',
    deleteTitle: (name) => `Remove "${name}"?`,
    deleteText: 'Prayers using this sound go back to the default sound.',
    delete: 'Remove',
    cancel: 'Cancel',
    play: (name) => `Play ${name}`,
    stop: (name) => `Stop ${name}`,
},

alarm: {
    app: 'Adhan',
    title: (prayer) => `It's time for ${prayer}`,
    stop: 'Stop adhan',
    stopAlarm: 'Stop alarm',
    mute: 'Mute sound',
    muteShort: 'Mute',
    close: 'Close',
    soundName: 'Adhan',
    channel: 'Prayer alarm',

```

```

},

battery: {
  title: 'Background activity',
  lede: 'To save battery, the phone may put Adhan to sleep, and the alarm may be late or not ring at all.
Let the app run without restrictions.',
  yourPhone: 'Your phone',
  open: 'Open Adhan settings',
  openList: '"Battery optimization" list',
  now: 'Now',
  ok: 'unrestricted ✓',
  bad: 'restricted',
  unknown: "can't be checked",
  otherPhone: 'Another phone?',
  more: 'More tips for every brand: dontkillmyapp.com',
  brands: {
    samsung: 'Samsung',
    xiaomi: 'Xiaomi / Redmi / POCO',
    huawei: 'Huawei / Honor',
    oppo: 'OPPO / Realme / OnePlus',
    other: 'Other',
  },
  steps: {
    samsung: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery'],
      ['Choose', 'Unrestricted', 'Not "Optimized" and not "Restricted"'],
      ['Optional:', 'Settings → Battery → Background usage limits', 'Adhan should not be in "Deep
sleeping apps" or "Sleeping apps"'],
    ],
    xiaomi: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery saver', 'Choose "No restrictions"'],
      ['Turn on', 'Autostart'],
      ['Optional: in recent apps, hold the Adhan card and lock it (padlock)'],
    ],
    huawei: [
      ['Open', 'Settings → Battery → App launch'],
      ['Find Adhan and turn off', 'Manage automatically'],
      ['In the window, turn on all three:', 'Auto-launch · Secondary launch · Run in background'],
    ],
    oppo: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery usage'],
      ['Turn on', 'Allow background activity', 'and "Auto launch", if it is there.'],
    ],
    other: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery'],
      ['Choose', 'Unrestricted', 'On some phones it is called "Don\'t optimize"'],
    ],
  },
},

about: {
  title: 'About',
  name: 'Adhan',
  back: 'Settings',
  version: (v: string) => `Version ${v}`,
  tagline: 'Prayer times, reminders and the adhan',
  groupTimes: 'Prayer time sources',
}

```

```

groupPlaces: 'Place data',
groupPrivacy: 'Privacy',
groupMap: 'The map and mosques',
sources: {
  mufti: { name: 'Grand Mufti of Bulgaria', desc: 'Prayer calendar - times in Bulgaria' },
  adhan: { name: 'adhan', desc: 'Calculation outside Bulgaria · MIT license' },
  osm: { name: 'OpenStreetMap', desc: 'Places in Bulgaria · © OpenStreetMap contributors, ODbL' },
  ne: { name: 'Natural Earth', desc: 'Border of Bulgaria · public domain' },
  geonames: { name: 'GeoNames', desc: 'World cities - name and country without internet · CC BY 4.0' },
  praytimes: { name: 'PrayTimes.org', desc: 'Adhan recordings: Makkah, Madinah, Al-Aqsa' },
  overpass: { name: 'OpenStreetMap · Overpass API', desc: 'Mosques nearby · © OpenStreetMap contributors, ODbL' },
  openfreemap: { name: 'OpenFreeMap', desc: 'The map · © OpenMapTiles, data © OpenStreetMap' },
},
privacy:
  'Your location is used on the phone to pick the place and the prayer times. Outside Bulgaria the phone asks its map service for the city name. For "Mosques nearby" only an approximate area (about 1 km) is sent to OpenStreetMap, and the map is loaded from OpenFreeMap.',
},

mosques: {
  title: 'Mosques nearby',
  count: (n: number, dist: string) => `${n} · within ${dist}`,
  next: 'Next:',
  nextAt: (prayer: string, time: string) => `${prayer} at ${time}`,
  nextIn: (left: string) => `in ${left}`,
  walk: (min: number) => `${min} min walk`,
  drive: (min: number) => `~${min} min by car`,
  unnamed: 'Mosque',
  unnamedSub: 'no name in OpenStreetMap',
  locating: 'Finding where you are...',
  loading: 'Looking for mosques nearby...',
  noneNear: (dist: string) => `No mosques within ${dist}`,
  noneFar: (dist: string) => `No mosques even within ${dist}`,
  noneHint: 'The map is OpenStreetMap - in small places not every mosque is on it.',
  searchFar: (dist: string) => `Search within ${dist}`,
  within: (dist: string) => `within ${dist}`,
  offlineTitle: 'No internet.',
  offlineText: (when: string) => `Showing the last search here - from ${when}. Directions will work once you are online.`,
  today: 'today',
  yesterday: 'yesterday',
  errorOffline: 'No internet',
  errorOfflineText: 'Mosques are looked up in OpenStreetMap - a connection is needed.',
  errorServer: 'The server is not responding',
  errorServerText: 'OpenStreetMap (Overpass) is busy. Try again in a minute.',
  retry: 'Try again',
  noGps: (place: string) => `No access to your location - searching around ${place}.`,
  allow: 'Allow',
  myLocation: 'My location',
  locate: 'Go to my location',
  back: 'Back to the list',
  reach: (min: number) => `you'll be there in ${min} min`,
  directions: (app: string) => `Directions with ${app}`,
  directionsAuto: 'Directions',
  withApp: (app: string) => `With ${app} · change in Settings → Navigation`,
  withAuto: 'With the phone's maps · change in Settings → Navigation',
  share: 'Share',
  copyAddress: 'Copy address',
  copyCoords: 'Copy coordinates',
  copied: 'Copied',
}

```

```

noAddress: 'No address in OpenStreetMap',
allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
allyMap: (n: number) => `Map with ${n} mosques`,
attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',
},

nav: {
  title: 'Navigation',
  back: 'Settings',
  intro: 'Which app opens "Directions" to the mosque.',
  auto: 'Automatic',
  autoAndroid: 'The phone's default maps app',
  autoIos: 'Apple Maps',
  walkDrive: 'Walking and driving',
  driveOnly: 'Driving only',
  notInstalled: 'not installed',
  organic: 'OpenStreetMap · works offline',
  huawei: 'on Huawei',
  download: 'Get',
  note: 'Waze only does driving - even for a mosque 300 m away it will route you by car. If the chosen
app is missing, "Directions" asks which one to use.',
  sheetTitle: (name: string) => `Directions to ${name}`,
  sheetSub: (dist: string) => `${dist} · which app?`,
  walk: 'walking',
  drive: 'driving',
  remember: 'Remember my choice',
  rememberHint: 'Change: Settings → Navigation',
  missing: (app: string) => `${app} is no longer on the phone - pick another app.`,
  group: 'Mosques nearby',
  row: 'Navigation',
  rowNote: 'Which app opens "Directions" to a mosque.',
  ask: 'Ask each time',
  close: 'Close',
},

methods: {
  Turkey: 'Diyanet (Turkey)',
  MuslimWorldLeague: 'Muslim World League',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Egyptian General Authority',
  Karachi: 'University of Karachi',
  UmmAlQura: 'Umm al-Qura (Makkah)',
  NorthAmerica: 'ISNA (North America)',
  Dubai: 'Dubai (UAE)',
  Kuwait: 'Kuwait',
  Qatar: 'Qatar',
  Singapore: 'Singapore (MUIS)',
  Tehran: 'Tehran (Institute of Geophysics)',
  France: 'France (UOIF, 12°)',
  Russia: 'Russia (DUM)',
  Malaysia: 'Malaysia (JAKIM)',
  Indonesia: 'Indonesia (Kemenag)',
},

highLatRules: {
  seventhofthenight: { name: 'Seventh of the night', desc: 'Most common in Europe' },
  middleofthenight: { name: 'Middle of the night', desc: 'Fajr and Isha never pass midnight' },
  twilightangle: { name: 'Twilight angle', desc: 'Share of the night based on the method angle' },
},

place: {

```

```

    title: 'Choose location',
    search: 'Search a place in Bulgaria',
    useGps: 'Use my location (GPS)',
    cities: 'Cities',
    noResults: 'No such place',
    abroadNote: 'Outside Bulgaria your location comes from GPS.',
    close: 'Close',
  },

  date: {
    weekdaysShort: ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'],
    monthsShort: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    monthsFull: ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September',
'October', 'November', 'December'],
  },

  hijriMonths: [
    'Muharram',
    'Safar',
    'Rabi al-Awwal',
    'Rabi al-Thani',
    'Jumada al-Ula',
    'Jumada al-Akhirah',
    'Rajab',
    'Shaban',
    'Ramadan',
    'Shawwal',
    'Dhu al-Qadah',
    'Dhu al-Hijjah',
  ],
};

```

tr.ts src/i18n

- И на турски – „Yakındaki camiler“, „Yol tarifi“.

src/i18n/tr.ts

508 реда

```

import type { Strings } from './strings';

/**
 * Türkçe (етап 10). Имената на молитвите и месеците – както в календара на Диянет
 * (İmsak, Güneş, Öğle, İkinci, Akşam, Yatsı). Където името на молитвата влиза в изречение,
 * след него стои „vakti/vaktine“ – така не трябва турски падежни наставки.
 * Настройките на телефона (Samsung, Xiaomi...) – с имената им в турския интерфейс.
 */
export const tr: Strings = {
  prayers: {
    fajr: 'İmsak',
    sunrise: 'Güneş',
    dhuhr: 'Öğle',
    asr: 'İkinci',
    maghrib: 'Akşam',
    isha: 'Yatsı',
  },

  alert: {
    off: 'bildirim yok',
    notify: 'bildirim',
    adhan: 'alarm',
  },
};

```

```

},

tabs: {
  today: 'Bugün',
  qibla: 'Kible',
  mosques: 'Camiler',
  month: 'Ay',
  settings: 'Ayarlar',
},

hero: {
  next: 'Sıradaki',
  at: (time) => `saat ${time}`,
  tomorrowAt: (time) => `yarın ${time}`,
  remaining: (time) => `${time} kaldı`,
},

nowPill: 'ŞİMDİ',

busy: 'Yükleniyor...',
locating: 'Konumunuz bulunuyor...',
choosePlace: 'Yer seçin',
noPlaceBanner: 'Konum bulunamadı - gösterilen vakitler Sofya içindir. Yerinizi seçmek için dokununuz.',

ally: {
  city: (city) => `Şehir: ${city}. Şehri değiştir`,
  locating: 'Konumunuz bulunuyor',
  bell: (prayer, mode) => `${prayer}: ${mode}. Değiştirmek için dokununuz`,
},

toast: (prayer, mode) => `${prayer}: ${mode}`,
alarmToast: (prayer) => `${prayer}: alarm - sessiz modda da çalar`,

placeholder: {
  qibla: 'Kâbe'yi gösteren pusula. 7. aşamada gelecek.',
},

month: {
  dayColumn: 'Gün',
  hijriColumn: 'Hicri',
  today: 'Bugün',
  prev: 'Önceki ay',
  next: 'Sonraki ay',
  friday: 'Cuma',
  sourceMufti: 'Kaynak: Bulgaristan Başmüftülüğü',
  sourceCalc: (method) => `${method} yöntemiyle hesaplandı`,
},

settings: {
  title: 'Ayarlar',
  groupPlace: 'Konum',
  groupTimes: 'Namaz vakitleri',
  groupCorrections: 'Düzeltilmeler',
  groupApp: 'Uygulama',
  about: 'Hakkında',
  change: 'Konumu değiştir',
  auto: 'Otomatik (GPS)',
  autoHint: 'Konumunuz uygulama açıldığında belirlenir.',
  locating: 'Konumunuz belirleniyor...',
  denied: 'Konum izni kapalı.',
  openSettings: 'Telefon ayarlarını aç',
}

```

```

unavailable: 'Konumunuz belirlenemedi. Bilinen son yer gösteriliyor.',
refresh: 'Konumu güncelle',
sourceMufti: 'Bulgaristan Başmüftülüğü takvimi',
sourceMuftiNote: 'Bulgaristan'da resmî takvim kullanılır. Yöntem ve İkinci ayarı yalnızca Bulgaristan
dışında geçerlidir.',
sourceCalc: 'Bu yer için hesaplandı',
method: 'Yöntem',
methodAuto: 'Otomatik',
methodAutoValue: (method) => `Otomatik · ${method}`,
methodAutoDesc: (country, method) => `Ülkeye göre: ${country} → ${method}`,
asr: 'İkinci',
shafi: 'Şâfiî, Mâlikî, Hanbelî',
hanafi: 'Hanefî',
shafiDesc: 'Gölge cismin boyu kadar - Diyanet takvimindeki gibi "birinci" İkinci',
hanafiDesc: 'Gölge cismin iki katı - yaklaşık bir saat sonra',
asrNote: 'Şâfiî, Mâlikî ve Hanbelî İkinci'yi aynı hesaplar; yalnızca Hanefî daha geçtir.',
polarNote: (lat) =>
` ${lat}° enlemde kutup gündüzleri ve geceleri vardır. Güneşin doğmadığı veya batmadığı günlerde
vakitler 65° enleme göre - normal gece ve gündüzün olduğu en yakın yerlere göre - hesaplanır. Bölgenizdeki
camiye danışın.`,
highLat: 'Yüksek enlemler',
highLatNote: (lat) =>
` ${lat}° enlemde yazın gerçek alacakaranlık olmaz; İmsak ve Yatsı bu kurala göre belirlenir.
Bölgenizdeki camiye danışın.`,
offsetsNote: 'Caminiz birkaç dakika farklıysa buradan düzeltin. Tüm ekranlarda geçerlidir.',
minutes: (n) => (n === 0 ? '0 dk' : `${n > 0 ? '+' : ''}${n} dk`),
hijri: 'Hicri tarih',
days: (n) => (n === 0 ? '0 gün' : `${n > 0 ? '+' : ''}${n} gün`),
hijriNote: 'Hicri tarih Bulgaristan Başmüftülüğünün ilan ettiğinden farklıysa.',
hijriNoteAbroad: 'Hicri tarih bölgenizdeki caminin ilan ettiğinden farklıysa.',
reset: 'Düzeltilmeleri sıfırla',
decrease: (label) => `Azalt: ${label}`,
increase: (label) => `Artır: ${label}`,
},

widget: {
next: 'Sıradaki',
tomorrowShort: (time) => `yarın ${time}`,
until: (name) => `${name} vaktine kalan`,
left: 'kaldı',
empty: 'Vakitleri görmek için Ezan'ı açın.',
},

notifications: {
prayerTitle: (prayer, time) => `${prayer} - ${time}`,
prayerBody: (prayer, arabic, place) => `${prayer} vakti girdi (${arabic}) · ${place}`,
sunriseBody: (place) => `Güneş doğdu - İmsak vakti sona erdi · ${place}`,
nextLine: (prayer, time, tomorrow) => `Sıradaki: ${prayer}, ${tomorrow ? 'yarın ' : ''}saat ${time}`,
reminderTitle: (prayer, minutes, time) => `${prayer} vaktine ${minutes} dk - ${time}`,
reminderBody: (place) => `Abdest alıp namaza hazırlanma vakti · ${place}`,
refreshTitle: 'Ezan'ı açın',
refreshBody: 'Namaz bildirimlerinin gelmeye devam etmesi için uygulamayı açın.',
channelPrayer: 'Namazlar',
channelAdhan: 'Alarm',
channelReminder: 'Hatırlatmalar',
group: 'Bildirimler',
status: 'İzin',
granted: 'Verildi',
denied: 'Reddedildi',
undetermined: 'Henüz sorulmadı',
allow: 'Bildirimlere izin ver',

```

```

deniedNote: 'Bildirimler telefon ayarlarında kapalı - bunlar olmadan namaz bildirimleri ve alarmlar çalışmaz.',
webNote: 'Bildirimler yalnızca telefon uygulamasında çalışır.',
reminder: 'Namazdan önce hatırlatma',
reminderOff: 'Hatırlatma yok',
reminderValue: (n) => `${n} dk önce`,
vibrate: 'Titreşim',
exact: 'Tam zamanında bildirimler',
exactOff: 'Kapalı',
exactNote: '"Alarmlar ve hatırlatıcılar" izni olmadan Android bildirimleri birkaç dakika geciktirebilir.',
exactAllow: '"Alarmlar ve hatırlatıcılar"a izin ver',
scheduled: (n, until) => `${n} bildirim planlandı - ${until} tarihine kadar`,
scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki tüm zil simgeleri kapalı.',
testTitle: 'Deneme bildirimi',
testBody: 'Namaz bildirimleri böyle görünecek.',
bellDenied: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz.',
allowedToast: '✓ Bildirimlere izin verildi',
exactBanner: 'Bildirimler birkaç dakika gecikebilir. Tam zamanında bildirimlere izin vermek için dokununuz.',
deniedBanner: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz. İzin vermek için dokununuz.',
diagnostics: 'Bildirim kontrolü',
diag: {
  title: 'Bildirim kontrolü',
  subtitle: 'Telefon Ezan bildirimlerini zamanında gösteriyor mu - göstermiyorsa neyin düzeltilmesi gerekir.',
  refresh: 'Yenile',
  updated: (time) => `Güncellendi: ${time}`,
  status: 'Durum',
  permOk: 'Bildirimlere izin verildi',
  permBad: 'Bildirimlere izin verilmedi',
  exactOk: 'Tam zamanında bildirimler ("Alarmlar ve hatırlatıcılar")',
  exactBad: 'Tam zamanında bildirimler kapalı - gecikebilir',
  allow: 'İzin ver',
  scheduledOk: (n, until) => `${n} bildirim planlandı · ${until} tarihine kadar`,
  scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki zil simgeleri kapalı',
  test: 'Deneme bildirimi',
  testNotification: 'Bildirim',
  testIn: (s) => `${s} sn sonra`,
  testHint: '10 sn sonra gelir. Gerçekte nasıl geleceğini görmek ve duymak için telefonu kilitleyin.',
  Alarm sesi Ayarlar → Sesler bölümünden dinlenebilir.',
  recent: 'Son bildirimler',
  recentNone: 'Henüz Ezan bildirimi gösterilmedi.',
  onTime: 'zamanında',
  late: (s) => (s < 60 ? `+${s} sn` : `+${Math.round(s / 6) / 10} dk`),
  plannedShown: (planned, shown) => `planlanan ${planned} · gösterilen ${shown}`,
  upcoming: 'Yaklaşan bildirimler',
  kindAlarm: 'Alarm',
  kindNotification: 'Bildirim',
  kindReminder: 'Hatırlatma',
  kindOther: 'Diğer',
  tech: 'Teknik ayrıntılar',
  device: 'Telefon',
  channelLine: (importance, sound, vibrate, usage) =>
    `önem ${importance} · ses ${sound} · titreşim ${vibrate ? 'evet' : 'hayır'} · akış ${usage}`,
  reset: 'Tüm bildirimleri yeniden planla',
  resetHint: 'Tüm Ezan bildirimlerini siler ve geçerli ayarlarla yeniden planlar. Bir şey yanlış görünüyorsa kullanın.',
  resetDone: '✓ Tüm bildirimler yeniden planlandı.',
  fullScreenOk: 'Tam ekran alarm',

```

```

        fullScreenBad: 'Tam ekran alarm kapalı',
        batteryOk: 'Arka planda kısıtlamasız çalışıyor',
        batteryBad: 'Telefon arka plan etkinliğini kısıtlıyor',
        fix: 'Düzeltil',
    },
    fullScreen: 'Tam ekran alarm',
    fullScreenOn: 'İzin verildi',
    fullScreenOff: 'Kapalı',
    fullScreenNote: 'Bu olmadan ezan kilitli telefonda yine çalar, ancak tam ekran yerine yalnızca bir
    bildirim görürsünüz.',
    fullScreenAllow: '"Tam ekran alarm"a izin ver',
    exactOn: 'İzin verildi',
    background: 'Arka plan etkinliği',
    backgroundOk: 'Kısıtlamasız',
    backgroundBad: 'Kısıtlı',
    batteryTitle: 'Alarmlar gecikebilir',
    batteryText: 'Telefon pil tasarrufu için uygulamayı durdurabilir.',
    batteryFix: 'Düzeltil',
},

qibla: {
    title: 'Kible',
    fromNorth: 'kuzeyden',
    toMecca: 'Mekke'ye',
    km: 'km',
    aligned: '✓ Kibleye dönüyorsunuz',
    turnLabel: (direction) => (direction === 'right' ? 'Sağa dönün' : 'Sola dönün'),
    starting: 'Pusulaya başlatılıyor...',
    quality: { high: 'yüksek', medium: 'orta', low: 'düşük' },
    accuracy: (q) => `Pusulaya: ${q} doğruluk`,
    tip: 'Telefonu ekranı yukarı bakacak şekilde düz tutun. Yakındaki metal ve mıknatıslar pusulayı
    şaşırtır.',
    calibrateTitle: 'Pusulayı kalibre edin',
    calibrateText: 'Telefonu havada 2-3 kez sekiz çizerek hareket ettirin. Metalden, mıknatıslardan ve
    mıknatıslı kılıflardan uzak tutun.',
    noCompassTitle: 'Pusulaya yok',
    noCompassText: (deg) =>
        `Bu telefonda manyetik sensör yok. Kible kuzeyden ${deg}°: kuzeye dönün ve ${deg}° sağa dönün.`,
    permissionTitle: 'İzin gerekli',
    permissionText: 'iPhone'da pusula yalnızca konum izniyle çalışır.',
    allowLocation: 'Konuma izin ver',
    letters: ['K', 'D', 'G', 'B'],
    allyDial: (bearing) => `Pusulaya. Kible kuzeyden ${bearing}°`,
    sunToday: 'Güneş bugün tam Kible yönünde: saat {t}',
    sunTomorrow: 'Güneş yarın tam Kible yönünde olacak: saat {t}',
    sunNow: 'Şu anda güneş tam Kible yönünde',
    sunrise: (azimuth, degrees, side) =>
        side === 'same'
            ? `Gün doğumu ${azimuth}° - tam Kible yönünde`
            : `Gün doğumu ${azimuth}° - Kible, gün doğumunun ${degrees}° ${side === 'right' ? 'sağında' :
            'solunda'}`,
    },

sounds: {
    group: 'Sesler',
    notifyGroup: 'Hatırlatma ve bildirim',
    notifyRow: 'Bildirim sesi',
    noteAndroid: 'Ses, namazın zili "alarm" olduğunda çalar. "Alarmlar ve hatırlatıcılar" olmadan - kısa
    ses (en fazla 30 sn).',
    noteIos: 'Ses, namazın zili "alarm" olduğunda çalar. iPhone'da ses en fazla 30 sn'dir.',
    fullOffTitle: 'Tam ezan kapalı',

```

```

    fullOffText: "Alarmlar ve hatırlatıcılar" olmadan yalnızca kısa ses çalar (en fazla 30 sn). Tam ezan
    için "Alarmlar ve hatırlatıcılar"a izin verin.',
    allow: 'İzin ver',
    title: (prayer) => `${prayer} sesi`,
    notifyTitle: 'Bildirim sesi',
    fullSection: 'Tam ses - alarm',
    fullHint: 'Sonuna kadar çalar; "Durdur" keser.',
    shortSection: 'Kısa ses - en fazla 30 sn',
    shortHintAndroid: "Alarmlar ve hatırlatıcılar" kapalıyken. Kendi sesiniz bir duraklamada, yumuşak bir
    kısılmayla 30 sn'ye kısaltılır.',
    iosSection: 'Ses - en fazla 30 sn',
    iosHint: 'iPhone'da bildirim sesi en fazla 30 sn olabilir. Daha uzun kayıtlar bir duraklamada, yumuşak
    bir kısılmayla kısaltılır.',
    notifyHint: 'Bildirimler ve namazdan önceki hatırlatmalar için.',
    byDefault: 'varsayılan',
    mine: 'BENİM',
    add: 'Kendi sesini ekle',
    addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · telefondan',
    addShortHint: 'mp3, m4a, mp4, wav... · uzunsa 30 sn'ye kısaltılır',
    addNotifyHint: 'en fazla 10 sn · mp3, m4a, mp4, wav...',
    tooLongNotify: (name, dur) => `${name} ${dur} uzunluğunda - bildirim sesi en fazla 10 sn olabilir.
    Daha kısa bir ses seçin.',
    adding: 'Ekleniyor...',
    workingShort: 'Kesit kesiliyor...',
    workingFull: 'Ses ekleniyor...',
    workingHint: '30. saniyeden önceki duraklama aranıyor. Birkaç saniye sürebilir.',
    applyAll: 'Tüm namazlar için aynı ses',
    apply: 'Uygula',
    applied: '✓ Tüm namazlara uygulandı',
    iosFuture: 'iPhone'da tam ezan AlarmKit (iOS 26) ile gelecek - 9. aşama.',
    added: (name) => `✓ Eklendi: ${name}`,
    addedTrimmed: (name, dur) =>
        `✓ Eklendi: ${name}. Kısa ses en fazla 30 sn'dir - ilk ${dur} bir duraklamaya kadar, yumuşak bir
        kısılmayla çalar.',
    trimmed: (dur, of) => `${of} içinden ilk ${dur} çalar',
    tooLongFull: (name) => `${name} 15 dakikadan uzun.',
    tooBig: 'Dosya 60 MB'tan büyük.',
    unreadable: 'Bu dosya ses olarak okunamıyor.',
    unsupported: 'Kendi seslerinizi yalnızca telefon uygulamasında ekleyebilirsiniz.',
    deleteTitle: (name) => `${name} kaldırılсын mı?',
    deleteText: 'Bu sesi kullanan namazlar varsayılan sese döner.',
    delete: 'Kaldır',
    cancel: 'Vazgeç',
    play: (name) => `${name} - çal`,
    stop: (name) => `${name} - durdur`,
},

alarm: {
    app: 'Ezan',
    title: (prayer) => `${prayer} vakti`,
    stop: 'Ezani durdur',
    stopAlarm: 'Alarmı durdur',
    mute: 'Sesi kapat',
    muteShort: 'Sessiz',
    close: 'Kapat',
    soundName: 'Ezan',
    channel: 'Namaz alarmı',
},

battery: {
    title: 'Arka plan etkinliği',

```

```

    lede: 'Telefon pil tasarrufu için Ezan'ı uyutabilir; alarm gecikebilir ya da hiç çalmayabilir.
    Uygulamanın kısıtlamasız çalışmasına izin verin.',
    yourPhone: 'Telefonunuz',
    open: 'Ezan ayarlarını aç',
    openList: '"Pil optimizasyonu" listesi',
    now: 'Şu an',
    ok: 'kısıtlamasız ✓',
    bad: 'kısıtlı',
    unknown: 'kontrol edilemiyor',
    otherPhone: 'Başka bir telefon mu?',
    more: 'Her marka için daha fazla ipucu: dontkillmyapp.com',
    brands: {
      samsung: 'Samsung',
      xiaomi: 'Xiaomi / Redmi / POCO',
      huawei: 'Huawei / Honor',
      oppo: 'OPPO / Realme / OnePlus',
      other: 'Diğer',
    },
    steps: {
      samsung: [
        ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
        ['Açın:', 'Pil'],
        ['Seçin:', 'Kısıtlanmamış', '"Optimize edildi" ya da "Kısıtlanmış" değil.'],
        ['İsteğe bağlı:', 'Ayarlar → Pil → Arka plan kullanım sınırları', 'Ezan "Derin uykudaki
        uygulamalar" veya "Uykudaki uygulamalar" içinde olmamalı.'],
      ],
      xiaomi: [
        ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
        ['Açın:', 'Pil tasarrufu', '"Kısıtlama yok" seçeneğini seçin.'],
        ['Açın:', 'Otomatik başlatma'],
        ['İsteğe bağlı: son uygulamalarda Ezan kartını basılı tutun ve kilitleyin (asma kilit).'],
      ],
      huawei: [
        ['Açın:', 'Ayarlar → Pil → Uygulama başlatma'],
        ['Ezan'ı bulun ve kapatın:', 'Otomatik olarak yönet'],
        ['Açılan pencerede üçünü de açın:', 'Otomatik başlatma · İkincil başlatma · Arka planda çalıştır'],
      ],
      oppo: [
        ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
        ['Açın:', 'Pil kullanımı'],
        ['Açın:', 'Arka plan etkinliğine izin ver', 've varsa "Otomatik başlatma"yı.'],
      ],
      other: [
        ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
        ['Açın:', 'Pil'],
        ['Seçin:', 'Kısıtlanmamış', 'Bazı telefonlarda adı "Optimize etme"dir.'],
      ],
    },
  },
  about: {
    title: 'Hakkında',
    name: 'Ezan',
    back: 'Ayarlar',
    version: (v: string) => `Sürüm ${v}`,
    tagline: 'Namaz vakitleri, hatırlatmalar ve ezan',
    groupTimes: 'Vakitlerin kaynakları',
    groupPlaces: 'Yer verileri',
    groupPrivacy: 'Gizlilik',
    groupMap: 'Harita ve camiler',
    sources: {

```

```

mufti: { name: 'Bulgaristan Başmüftülüğü', desc: 'Namaz takvimi - Bulgaristan'daki vakitler' },
adhan: { name: 'adhan', desc: 'Bulgaristan dışında hesaplama - MIT lisansı' },
osm: { name: 'OpenStreetMap', desc: 'Bulgaristan'daki yerler - © OpenStreetMap katkıcıları, ODbL' },
ne: { name: 'Natural Earth', desc: 'Bulgaristan sınırı - kamu malı' },
geonames: { name: 'GeoNames', desc: 'Dünya şehirleri - internetsiz ad ve ülke - CC BY 4.0' },
praytimes: { name: 'PrayTimes.org', desc: 'Ezan kayıtları: Mekke, Medine, Mescid-i Aksa' },
overpass: { name: 'OpenStreetMap - Overpass API', desc: 'Yakındaki camiler - © OpenStreetMap katkıcıları, ODbL' },
openfreemap: { name: 'OpenFreeMap', desc: 'Harita - © OpenMapTiles, veriler © OpenStreetMap' },
},
privacy:
  'Konumunuz telefonda, yeri ve vakitleri seçmek için kullanılır. Bulgaristan dışında telefon şehrin adını kendi harita hizmetine sorar. „Yakındaki camiler“ için OpenStreetMap'e yalnızca yaklaşık bölge (yaklaşık 1 km) gönderilir; harita OpenFreeMap'ten yüklenir.',
},

mosques: {
  title: 'Yakındaki camiler',
  count: (n: number, dist: string) => `${n} · ${dist} içinde`,
  next: 'Sıradaki:',
  nextAt: (prayer: string, time: string) => `${prayer} ${time}`,
  nextIn: (left: string) => `${left} kaldı`,
  walk: (min: number) => `yürüyerek ${min} dk`,
  drive: (min: number) => `arabayla ~${min} dk`,
  unnamed: 'Cami',
  unnamedSub: 'OpenStreetMap'te adı yok',
  locating: 'Konumunuz bulunuyor...',
  loading: 'Yakındaki camiler aranıyor...',
  noneNear: (dist: string) => `${dist} içinde cami yok`,
  noneFar: (dist: string) => `${dist} içinde de cami yok`,
  noneHint: 'Harita OpenStreetMap'tir - küçük yerlerde her cami işaretli değildir.',
  searchFar: (dist: string) => `${dist} içinde ara`,
  within: (dist: string) => `${dist} içinde`,
  offlineTitle: 'İnternet yok.',
  offlineText: (when: string) => `Buradaki son arama gösteriliyor (${when}). Yol tarifi bağlantı gelince çalışır.`,
  today: 'bugün',
  yesterday: 'dün',
  errorOffline: 'İnternet yok',
  errorOfflineText: 'Camiler OpenStreetMap'te aranır - bağlantı gerekir.',
  errorServer: 'Sunucu yanıt vermiyor',
  errorServerText: 'OpenStreetMap (Overpass) yoğun. Bir dakika sonra tekrar deneyin.',
  retry: 'Tekrar dene',
  noGps: (place: string) => `Konum izni yok - ${place} çevresinde aranıyor.`,
  allow: 'İzin ver',
  myLocation: 'Konumum',
  locate: 'Konumuma git',
  back: 'Listeye dön',
  reach: (min: number) => `${min} dakikada varırsınız`,
  directions: (app: string) => `${app} ile yol tarifi`,
  directionsAuto: 'Yol tarifi',
  withApp: (app: string) => `${app} ile - Ayarlar → Navigasyon bölümünden değiştirin`,
  withAuto: 'Telefonun haritalarıyla - Ayarlar → Navigasyon bölümünden değiştirin',
  share: 'Paylaş',
  copyAddress: 'Adresi kopyala',
  copyCoords: 'Koordinatları kopyala',
  copied: 'Kopyalandı',
  noAddress: 'OpenStreetMap'te adres yok',
  allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
  allyMap: (n: number) => `Harita: ${n} cami`,
  attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',

```

```

},

nav: {
  title: 'Navigasyon',
  back: 'Ayarlar',
  intro: 'Camiye „Yol tarifi“ hangi uygulamayla açılsın.',
  auto: 'Otomatik',
  autoAndroid: 'Telefonun varsayılan harita uygulaması',
  autoIos: 'Apple Haritalar',
  walkDrive: 'Yürüyerek ve arabayla',
  driveOnly: 'Yalnızca arabayla',
  notInstalled: 'yükülü değil',
  organic: 'OpenStreetMap • internetsiz de çalışır',
  huawei: 'Huawei telefonlarda',
  download: 'İndir',
  note: 'Waze yalnızca araba için yol tarifi verir - 300 m uzaktaki camiye bile arabayla götürür. Seçilen uygulama yoksa „Yol tarifi“ hangisiyle açılacağını sorar.',
  sheetTitle: (name: string) => `Yol tarifi: ${name}`,
  sheetSub: (dist: string) => `${dist} • hangi uygulamayla?`,
  walk: 'Yürüyerek',
  drive: 'arabayla',
  remember: 'Seçimimi hatırla',
  rememberHint: 'Değiştirmek için: Ayarlar → Navigasyon',
  missing: (app: string) => `${app} artık telefonda yok - başka bir uygulama seçin.`,
  group: 'Yakındaki camiler',
  row: 'Navigasyon',
  rowNote: 'Camiye „Yol tarifi“ hangi uygulamayla açılır.',
  ask: 'Her seferinde sor',
  close: 'Kapat',
},

methods: {
  Turkey: 'Diyanet (Türkiye)',
  MuslimWorldLeague: 'Müslüman Dünya Birliği',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Mısır Genel Araştırma Kurumu',
  Karachi: 'Karaçi Üniversitesi',
  UmmAlQura: 'Ümmü'l-Kurâ (Mekke)',
  NorthAmerica: 'ISNA (Kuzey Amerika)',
  Dubai: 'Dubai (BAE)',
  Kuwait: 'Kuveyt',
  Qatar: 'Katar',
  Singapore: 'Singapur (MUIS)',
  Tehran: 'Tahran (Jeofizik Enstitüsü)',
  France: 'Fransa (UOIF, 12°)',
  Russia: 'Rusya (DUM)',
  Malaysia: 'Malezya (JAKIM)',
  Indonesia: 'Endonezya (Kemenag)',
},

highLatRules: {
  seventhofthenight: { name: 'Gecenin yedide biri', desc: 'Avrupa'da en yaygın olanı' },
  middleofthenight: { name: 'Gecenin yarısı', desc: 'İmsak ve Yatsı gece yarısını geçmez' },
  twilightangle: { name: 'Alacakaranlık açısı', desc: 'Yöntemin açısına göre gecenin bir payı' },
},

place: {
  title: 'Konum seçin',
  search: 'Bulgaristan'da yer arayın',
  useGps: 'Konumumu kullan (GPS)',
  cities: 'Şehirler',
}

```

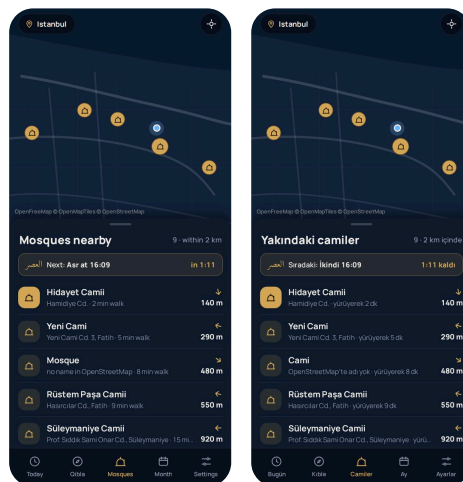
```

noResults: 'Böyle bir yer yok',
abroadNote: 'Bulgaristan dışında konumunuz GPS'ten alınır.',
close: 'Kapat',
},

date: {
  weekdaysShort: ['Paz', 'Pzt', 'Sal', 'Çar', 'Per', 'Cum', 'Cmt'],
  monthsShort: ['Oca', 'Şub', 'Mar', 'Nis', 'May', 'Haz', 'Tem', 'Ağu', 'Eyl', 'Eki', 'Kas', 'Ara'],
  monthsFull: ['Ocak', 'Şubat', 'Mart', 'Nisan', 'Mayıs', 'Haziran', 'Temmuz', 'Ağustos', 'Eylül',
'Ekim', 'Kasım', 'Aralık'],
},

hijriMonths: [
  'Muharrem',
  'Safer',
  'Rebiülevvel',
  'Rebiülahir',
  'Cemaziyelevvel',
  'Cemaziyelahir',
  'Recep',
  'Şaban',
  'Ramazan',
  'Şevval',
  'Zilkade',
  'Zilhicce',
],
},
};

```



Английски и турски

Стъпка 7

Тестове

258 теста общо, 21 нови.

mosques.test.ts src/domain/__tests__

- Закръгленото място в заявката; разчитането на отговора (имена, арабско име, адрес, без дублирани).
- Подреждане по разстояние, пеша/с кола, посока, „350 м“ / „1,4 км“ на трите езика, кеша.

- Връзките за всяко приложение; Waze – само с кола; магазините.

src/domain/__tests__/mosques.test.ts

136 реда

```
import {
  bearingDeg,
  cacheCovers,
  FAR_RADIUS_M,
  formatDistance,
  NEAR_RADIUS_M,
  nearMosques,
  overpassQuery,
  parseOverpass,
  privacyArea,
  travelMinutes,
} from '../mosques';
import { modeFor, navApp, navAppsFor, navUrl, storeUrl } from '../navApps';

// Еминьоню, Истанбул – като в тоскур-а
const ME = { lat: 41.0165, lon: 28.9749 };

const overpass = {
  elements: [
    { type: 'way', id: 1, center: { lat: 41.0171, lon: 28.9715 }, tags: { amenity: 'place_of_worship',
      religion: 'muslim', name: 'Yeni Cami', 'name:ar': 'الجامع الجديد', 'addr:street': 'Yeni Cami Cd.',
      'addr:housenumber': '3', 'addr:district': 'Fatih' } },
    { type: 'way', id: 2, center: { lat: 41.0161, lon: 28.9640 }, tags: { name: 'Süleymaniye Camii',
      'name:bg': 'Сюлеймание' } },
    { type: 'node', id: 3, lat: 41.0054, lon: 28.9768, tags: { name: 'Sultanahmet Camii', 'name:ar':
      'Sultanahmet Camii' } },
    { type: 'node', id: 4, lat: 41.04, lon: 28.99, tags: {} },
    { type: 'node', id: 5, tags: { name: 'без координати' } },
    { type: 'way', id: 1, center: { lat: 41.0171, lon: 28.9715 }, tags: { name: 'Yeni Cami' } },
  ],
};

describe('джамии – заявката и поверителността', () => {
  it('до Overpass отива закръглено място (~1 км) и по-голям радиус', () => {
    const a = privacyArea(41.016512, 28.974931, NEAR_RADIUS_M);
    expect(a).toEqual({ lat: 41.02, lon: 28.97, radiusM: 3500 });
    const q = overpassQuery(41.016512, 28.974931, NEAR_RADIUS_M);
    expect(q).toContain('(around:3500,41.02,28.97)');
    expect(q).toContain('["amenity"="place_of_worship"]["religion"="muslim"]');
    expect(q).not.toContain('41.0165');
  });
});

describe('джамии – отговорът на Overpass', () => {
  const list = parseOverpass(overpass, 'bg');

  it('име на езика на приложението, иначе местното; без име – null', () => {
    expect(list.map((m) => m.name)).toEqual(['Yeni Cami', 'Сюлеймание', 'Sultanahmet Camii', null]);
  });

  it('арабското име, адресът, точката на сградата; без дублирани и без координати', () => {
    expect(list).toHaveLength(4);
    expect(list[0]).toMatchObject({ id: 'way/1', arabic: 'الجامع الجديد', address: 'Yeni Cami Cd. 3,
      Fatih', lat: 41.0171 });
    expect(list[2].arabic).toBeNull(); // същото като името – не се повтаря
    expect(list[3].address).toBeNull();
  });
});
```

```

it('счупен отговор → празен списък', () => {
  expect(parseOverpass(null, 'bg')).toEqual([]);
  expect(parseOverpass({ remark: 'timeout' }, 'en')).toEqual([]);
});

describe('джамии – разстояние, посока, време', () => {
  const all = parseOverpass(overpass, 'en');

  it('подредени по разстояние, само до радиуса', () => {
    const near = nearMosques(all, ME.lat, ME.lon, NEAR_RADIUS_M);
    expect(near.map((m) => m.name)).toEqual(['Yeni Cami', 'Süleymaniye Camii', 'Sultanahmet Camii']);
    expect(near[0].distanceM).toBeGreaterThan(250);
    expect(near[0].distanceM).toBeLessThan(350);
    expect(nearMosques(all, ME.lat, ME.lon, FAR_RADIUS_M)).toHaveLength(4);
  });

  it('пеша до 2,5 км, с кола над това', () => {
    const far = nearMosques(all, ME.lat, ME.lon, FAR_RADIUS_M);
    expect(far[0].mode).toBe('walk');
    expect(far[3].mode).toBe('drive');
    expect(travelMinutes(350, 'walk')).toBe(6);
    expect(travelMinutes(6000, 'drive')).toBe(16);
  });

  it('посоката от север', () => {
    expect(Math.round(bearingDeg(41, 29, 42, 29))).toBe(0);
    expect(Math.round(bearingDeg(41, 29, 41, 30))).toBe(90);
    expect(Math.round(bearingDeg(41, 29, 40, 29))).toBe(180);
  });

  it('разстоянието на трите езика', () => {
    expect(formatDistance(347, 'bg')).toBe('350 м');
    expect(formatDistance(1420, 'bg')).toBe('1,4 км');
    expect(formatDistance(1420, 'en')).toBe('1.4 km');
    expect(formatDistance(1420, 'tr')).toBe('1,4 km');
    expect(formatDistance(12400, 'bg')).toBe('12 км');
  });

  it('кешът важи за същия район, радиус и до 24 ч.', () => {
    const area = privacyArea(ME.lat, ME.lon, NEAR_RADIUS_M);
    const cache = { ...area, at: 0 };
    expect(cacheCovers(cache, ME.lat, ME.lon, NEAR_RADIUS_M, 3600_000)).toBe(true);
    expect(cacheCovers(cache, ME.lat, ME.lon, FAR_RADIUS_M, 3600_000)).toBe(false);
    expect(cacheCovers(cache, 41.2, 29.1, NEAR_RADIUS_M, 3600_000)).toBe(false);
    expect(cacheCovers(cache, ME.lat, ME.lon, NEAR_RADIUS_M, 25 * 3600_000)).toBe(false);
  });

  describe('навигация – приложенията и връзките', () => {
    const to = { lat: 41.0171, lon: 28.9715, name: 'Yeni Cami' };
    const from = { lat: ME.lat, lon: ME.lon, name: 'My location' };

    it('списъкът според телефона', () => {
      expect(navAppsFor('android').map((a) => a.id)).toEqual(['auto', 'google', 'waze', 'here', 'petal', 'organic']);
      expect(navAppsFor('ios').map((a) => a.id)).toEqual(['auto', 'google', 'waze', 'here', 'organic']);
    });

    it('Waze – винаги с кола', () => {

```

```

    expect(modeFor('waze', 'walk')).toBe('drive');
    expect(modeFor('google', 'walk')).toBe('walk');
    expect(navApp('waze').walking).toBe(false);
  });

  it('връзките', () => {
    expect(navUrl('google', 'android', to, 'walk')).toBe('google.navigation:q=41.017100,28.971500&mode=w');
    expect(navUrl('google', 'ios', to, 'drive')).toBe('comgooglemaps://?daddr=41.017100,28.971500&directionsmode=driving');
    expect(navUrl('waze', 'android', to, 'drive')).toBe('https://waze.com/ul?ll=41.017100,28.971500&navigate=yes&utm_source=ezan');
    expect(navUrl('here', 'android', to, 'walk')).toBe('https://share.here.com/r/mylocation/41.017100,28.971500,Yeni%20Cami?m=w');
    expect(navUrl('petal', 'android', to, 'walk')).toContain('petalmaps://route?daddr=41.017100,28.971500&type=walk&coordinateType=0');
    expect(navUrl('organic', 'android', to, 'walk', from)).toBe('om://route?sll=41.016500,28.974900&saddr=My%20location&dll=41.017100,28.971500&daddr=Yeni%20Cami&type=pedestrian');
    expect(navUrl('auto', 'android', to, 'walk')).toBe('geo:0,0?q=41.017100,28.971500(Yeni%20Cami)');
    expect(navUrl('auto', 'ios', to, 'walk')).toBe('https://maps.apple.com/directions?destination=41.017100,28.971500&mode=walking');
  });

  it('магазините', () => {
    expect(storeUrl(navApp('waze'), 'android')).toBe('market://details?id=com.waze');
    expect(storeUrl(navApp('organic'), 'ios')).toBe('https://apps.apple.com/app/id1567437057');
    expect(storeUrl(navApp('auto'), 'android')).toBeNull();
  });
});

```

mosques.test.ts src/services/__tests__

- Сваля и пази в кеша; втори път – без сървъра; в кеша – само нужните тагове.
- Претоварен първи сървър → вторият; „Query timed out“ е грешка, а не „няма джамии“.
- Без интернет: старото търсене с отметка; без кеш – съобщение.

src/services/__tests__/mosques.test.ts

106 реда

```

/* eslint-disable @typescript-eslint/no-require-imports */
import AsyncStorage from '@react-native-async-storage/async-storage';

// jest.mock се изпълнява преди import-ите (babel-jest го премества най-отгоре)
jest.mock('@react-native-async-storage/async-storage', () =>
  require('@react-native-async-storage/async-storage/jest/async-storage-mock'),
);

type Service = typeof import('../mosques');

const ME = { lat: 41.0165, lon: 28.9749 };
const BODY = {
  elements: [
    {
      type: 'way',
      id: 1,
      center: { lat: 41.0171, lon: 28.9715 },
      tags: { name: 'Yeni Cami', 'name:ar': 'الجامع الجديد', 'addr:street': 'Yeni Cami Cd.', wikidata:
'Q1', building: 'mosque' },
    },
  ],

```

```

    },
  };

const ok = (body: unknown) => Promise.resolve({ ok: true, status: 200, json: () => Promise.resolve(body) });
const status = (code: number) => Promise.resolve({ ok: false, status: code, json: () => Promise.resolve({}) });
const offline = () => Promise.reject(new TypeError('Network request failed'));

let fetchMock: jest.Mock;
let service: Service;

beforeEach(async () => {
  await AsyncStorage.clear();
  fetchMock = jest.fn();
  globalThis.fetch = fetchMock as unknown as typeof fetch;
  // кешът в паметта и броячът на заявките - наново за всеки тест
  jest.isolateModules(() => {
    service = require('../mosques');
  });
});

afterEach(() => jest.useRealTimers());

describe('джамии - мрежата и кешът', () => {
  it('сваля, пази в кеша и втория път не ходи до сървъра', async () => {
    fetchMock.mockImplementation(() => ok(BODY));
    const a = await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
    expect(a.mosques[0]).toMatchObject({ name: 'Yeni Cami', arabic: 'الجامع الجديد' });
    expect(a.offline).toBe(false);
    const b = await service.loadMosques(ME.lat, ME.lon, 2000, 'en');
    expect(b.mosques).toHaveLength(1);
    expect(fetchMock).toHaveBeenCalledTimes(1);
  });

  it('до сървъра отиват само закръглени координати', async () => {
    fetchMock.mockImplementation(() => ok(BODY));
    await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
    const body = decodeURIComponent(fetchMock.mock.calls[0][1].body);
    expect(body).toContain('around:3500,41.02,28.97');
    expect(body).not.toContain('41.0165');
  });

  it('в кеша остават само нужните тагове', async () => {
    fetchMock.mockImplementation(() => ok(BODY));
    await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
    const saved = await AsyncStorage.getItem('ezan.mosques.v1');
    expect(saved).toContain('Yeni Cami Cd. ');
    expect(saved).not.toContain('wikidata');
  });

  it('претоварен първи сървър → вторият', async () => {
    fetchMock.mockImplementationOnce(() => status(504)).mockImplementationOnce(() => ok(BODY));
    const r = await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
    expect(r.mosques).toHaveLength(1);
    expect(fetchMock.mock.calls[1][0]).toContain('kumi.systems');
  });

  it('„Query timed out“ с код 200 е грешка, а не „няма джамии“', async () => {
    fetchMock.mockImplementation(() => ok({ elements: [], remark: 'runtime error: Query timed out' }));
    await expect(service.loadMosques(ME.lat, ME.lon, 2000, 'bg')).rejects.toMatchObject({ reason: 'server'

```

```

});
});

it('без интернет и без кеш → грешка „offline“', async () => {
  fetchMock.mockImplementation(offline);
  await expect(service.loadMosques(ME.lat, ME.lon, 2000, 'bg')).rejects.toMatchObject({ reason: 'offline'
});
});

it('без интернет – старото търсене (и по-старо от 24 ч.) с отметка offline', async () => {
  jest.useFakeTimers({ now: new Date('2026-10-01T12:00:00Z'), doNotFake: ['setTimeout', 'clearTimeout']
});
  fetchMock.mockImplementation(() => ok(BODY));
  await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');

  jest.setSystemTime(new Date('2026-10-03T12:00:00Z'));
  fetchMock.mockImplementation(offline);
  const r = await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
  expect(r.offline).toBe(true);
  expect(r.mosques).toHaveLength(1);
  expect(new Date(r.at).toISOString()).toBe('2026-10-01T12:00:00.000Z');
});

it('кешът за 2 км не важи за 10 км', async () => {
  fetchMock.mockImplementation(() => ok(BODY));
  await service.loadMosques(ME.lat, ME.lon, 2000, 'bg');
  await service.loadMosques(ME.lat, ME.lon, 10000, 'bg');
  expect(fetchMock).toHaveBeenCalledTimes(2);
  expect(decodeURIComponent(fetchMock.mock.calls[1][1].body)).toContain('around:11500,');
});
});

```

Трябва да видиш: Tests: 258 passed, 258 total

npm test

Стъпка 8

Проверка на телефона

Samsung и Huawei P70 Ultra:

- Раздел „Джамии“: картата се зарежда, синята точка си ти, списъкът е подреден по разстояние.
- Докосни джамия → „Упътване“ → „Отвори с...“ показва само инсталираните. Избери Waze с „Запомни избора“ → отваря Waze с маршрута.
- Затвори приложението, рестартирай телефона → „Упътване“ отваря направо Waze.
- Настройки → Навигация → Google Maps → „Упътване“ води пеша.
- **Huawei**: картата работи без Google услуги; Petal Maps е в списъка; „Автоматично“ отваря Petal Maps.
- Изтрий за малко избраното приложение (или избери неинсталирано) → „Упътване“ пита отново.
- Самолетен режим → „Джамии“: „Няма интернет. Показвам последното търсене тук – от ...“.
- „Сподели“ → Viber/WhatsApp: името, адресът и връзката към картата. „Копирай адреса“ → „Копирано“.
- В малко село: „Няма джамии до 2 км“ → „Търси до 10 км“.

Готово, когато: в София, Истанбул и непознат град списъкът и картата показват джамиите наоколо, „Упътване“ отваря навигация до избраната, работи и на телефон без Google услуги.

Терминал

```
git add -A  
git status  
git commit -m "Stage 12: Mosques nearby"  
git tag stage-12  
git push  
git push origin stage-12
```